AI_Evidence - Appendix to Assignment #03

Myrzabayev Ruslan - Smart Campus Room Availability Checker

> Contents: A. AI tools, models and dates · B. Original prompt-response exchanges with my review decisions and revisions · C. Raw test output · D. Original screenshots and diagram · E. References and source checks · F. History: earlier Practice 6 Bolt project.
> Bolt prompts and responses are copied verbatim from the Bolt chat. Where Bolt only showed a collapsed list of tool steps, it is written as *[tool steps collapsed in UI]*.

A. AI tools, models and dates

Tool	Model / mode (as displayed)	Used for	Dates (Asia/Almaty)
Bolt.new, A3 project `sb1-gewmu6gh` (Free plan)	"Bolt Agent - Standard" (P-12). Underlying LLM name is **not shown**.	Prototype v1 and fixes v2-v4	24 Oct 2026: v1 22:46, v2 22:50, v3 22:51, v4 22:53
Bolt.new, Practice 6 project `sb1-svxv4k4n`	same	Earlier prototype, history only (section F)	06 Oct, 13 Oct, 24 Oct 2026 (14:06)
Claude (Anthropic), Cowork session in the Chrome side panel	Session configured as `claude-opus-5-5`	Python `can_book` v1 and fix, Python tests, the Vitest acceptance file (E-C1), driving the browser for test runs and screenshots, drafting this documentation	24 Oct 2026

B. Prompt-response exchanges

Mapping to the required workflow: **A Plan** report section 1 (my plan, written first) + P6-1 (Bolt plan I approved); **B Generate** E-B1 (web v1), E-P1 (Python v1); **C Critique** my review of v1 against AC1-AC6 (report 4.2, 5.2) and P6-4/P6-6; **D Repair** E-B2, E-B3, E-B4, E-P2; **E Verify** E-C1 + my own reruns (section C, S-18 to S-27).

E-B1 - Bolt.new - Generate web prototype v1 (24 Oct 2026, Version 1, 22:46)

Prompt (exact, S-01)

> Assignment 3 - prototype v1. Build ONE responsive page: Smart Campus Room Availability Checker. User story: As a student, I want to check a room's availability so that I can choose a valid booking time. The page lets the user select one room from a small list of fictional rooms, set a simulated current time, choose a start and end time, and press Check. It only CHECKS availability - it must not create, confirm or change bookings. Rules (treat as fixed, do not change their meaning): AC1. Candidate times are integer minutes after midnight: 0 <= start < end <= 1440; start must be strictly after now (start == now is rejected). AC2. Duration must not exceed 120 minutes; exactly 120 minutes is permitted. AC3. Reject every request when the selected room is blocked. AC4. Reject overlap with any existing booking; intervals touching at an endpoint do NOT overlap (half-open intervals). AC5. Display an unambiguous Available / Unavailable result; do not mutate existing bookings. AC6. Provide understandable validation feedback and a usable narrow/mobile layout. Use fictional in-memory sample data only (include one room with a 10:00-11:00 booking and one blocked room). Explicitly forbidden: login/authentication, database (no Supabase or storage), backend/server, external APIs, payments, analytics and any

unrelated features. Keep the availability rule in a separate pure function. After building, list the main files/components you created and their role.

Response (exact)

[Thoughts collapsed]
I'll start by reviewing the existing project files to match conventions, then build the room availability checker.
[tool steps collapsed in UI: files read]
Now I'll create the pure availability function, sample data, UI components, and the main page.
[tool steps collapsed in UI: actions taken]
Now the UI components and main page.
[tool steps collapsed in UI: actions taken]
Now the time picker helper and the main page.
[tool steps collapsed in UI: actions taken]
Now let me verify the build passes.
Ran build to verify
Build passes. Here's a summary of what I built.

Smart Campus Room Availability Checker — a single responsive page that lets a student pick a fictional room, set a simulated "now" time, choose a start/end time, and press Check to see if the slot is available. It never creates or modifies bookings.

Main files and their roles:
src/lib/availability.ts — The pure availability function checkAvailability() plus shared types. This is the single source of truth for all six acceptance criteria: time bounds and "start strictly after now" (AC1), 120-min duration cap (AC2), blocked-room rejection (AC3), half-open interval overlap detection (AC4), unambiguous Available/Unavailable result (AC5), and it never mutates bookings (AC5). Also a fmt() helper to turn minutes-after-midnight into HH:MM.
src/data/rooms.ts — Fictional in-memory sample data only. Four rooms, including one with a 10:00–11:00 booking and one blocked room (per AC3).
src/components/RoomPicker.tsx — Room selection list and a view of each room's existing bookings.
src/components/TimeSelect.tsx — A reusable dropdown for picking times in 15-minute increments.
src/components/ResultPanel.tsx — The clear Available / Unavailable result display with icons and the specific reason on rejection.
src/App.tsx — The page: header, room picker, existing bookings, the time form with live validation feedback, the Check button, and the result panel. Fully responsive down to a narrow mobile layout.

The availability logic is isolated in the pure function as required, and validation messages guide the student when start/end/now conflict before they even press Check.

My review decision (critique against AC1-AC6)

Not accepted as final. I checked the claims myself: the logic file is fine (integer checks, `start <= now`, `> 120`, half-open overlap; S-07), but "15-minute increments" contradicts AC1/AC2 testing (S-06), and the "guide before Check" design disables Check and leaves an old result on screen (S-09, S-10). Also the blocked check sits after the time checks (S-08). Three issues logged: V1-01, V1-02, V1-03.

E-B2 - Bolt.new - Repair V1-01 stale result (Version 2, 22:50)

Prompt (exact, S-11)

Issue V1-01 (High, AC1/AC3/AC5). Reproduction in the preview: select Athena Library Room 201, now 09:00, 12:00-14:00, click Check -> Available (correct). Then select Newton Lecture Hall A (blocked) and change Start to 09:00, End to 09:30 without clicking anything else. Actual: the green panel still says "Available - The room is free for 09:00-09:30 (30 min)" while the Check button is disabled. Expected: a blocked room and a start equal to now must never be shown as Available, and the result must belong to the current inputs. Also, for invalid inputs (start == now) the user never gets an Unavailable result, only a small hint, but AC5 needs an unambiguous Available/Unavailable result. Smallest fix, in App.tsx

only: (1) clear the previous result whenever room, now, start or end changes; (2) do not disable the Check button for validation errors - keep the inline hints, but on click always call checkAvailability() and show its Available/Unavailable result with the reason. Do not change availability.ts, TimeSelect.tsx, the rules AC1-AC6 or the layout. Tell me exactly what changed.

Response (exact, S-12)

[Thoughts collapsed] Read src/App.tsx
I'll make the two targeted fixes in App.tsx: clear stale results when inputs change, and keep the Check button always enabled so every click produces an unambiguous Available/Unavailable result.
[tool steps collapsed in UI: actions taken]
Build passes. Here's exactly what changed in App.tsx only:

Imported useEffect alongside useMemo and useState.
Added a useEffect that clears the result (setResult(null)) whenever roomId, now, start, or end changes. This prevents a stale "Available" panel from lingering after you switch rooms or adjust times — the displayed result now always belongs to the current inputs.
Removed disabled and its disabled styling from the Check button. It's now always clickable, so every press calls checkAvailability() and shows an unambiguous Available/Unavailable result with the reason — even for invalid inputs like start equal to now, which now correctly shows Unavailable instead of just a small hint. The inline validation hints are kept as-is.

No changes to availability.ts, TimeSelect.tsx, the rules, or the layout.

My review decision / retest

Accepted after retest: Newton 09:00-09:30 now gives "Unavailable" (v3 run, S-14); switching room after a check clears the panel (S-25); starts-now on Athena gives Unavailable (S-21).

E-B3 - Bolt.new - Repair V1-02 minute precision (Version 3, 22:51)

Prompt (exact)

Issue V1-02 (High, AC1/AC2, testability). src/components/TimeSelect.tsx line 14 builds options in 15-minute steps (STEPS = i * 15). So the required test case 720-841 (12:00-14:01, 121 minutes, must be Unavailable by AC2) cannot be entered at all, and AC1 is defined in integer minutes. Smallest fix in TimeSelect.tsx only: generate options for every minute 00:00..24:00 (0..1440, 1441 options) so 14:01 and 24:00 are selectable. Keep fmt(), the component props, availability.ts and App.tsx logic unchanged. Tell me exactly what changed.

Response (exact, S-13)

[tool steps collapsed in UI: actions taken]
Build passes. Here's exactly what changed in TimeSelect.tsx:

Line 14: replaced the 15-minute step generator (Array.from({ length: 97 }, (_, i) => i * 15), producing 0, 15, 30, … 1440) with a per-minute generator (Array.from({ length: 1441 }, (_, i) => i), producing 0, 1, 2, … 1440). This makes every integer minute from 00:00 to 24:00 selectable, so the required AC2 test case of 12:00–14:01 (121 minutes) and 24:00 can both be entered. fmt(), the component props, availability.ts, and App.tsx logic are all unchanged.

My review decision / retest

Accepted. Code checked (S-17, line 14). 12:00-14:01 → "Booking length is 121 min. Maximum allowed is 120 min." (S-24). Side effect I noticed: "Simulated now" uses the same component, so now = 24:00 is selectable; noted as a limitation, not changed (fixed scope).

E-B4 - Bolt.new - Repair V1-03 blocked reason first (Version 4, 22:53)

Prompt (exact)

> Issue V1-03 (Medium, AC3/AC6 feedback). Reproduction: Newton Lecture Hall A (blocked), now 09:00, 09:00-09:30, Check -> "Unavailable - Start time must be strictly later than the current simulated time." The decision is right but the reason is misleading: the student fixes the time and only then finds out the room is blocked. AC3 says every request for a blocked room is rejected, so it should be reported first. Smallest fix: in src/lib/availability.ts move the existing `if (room.blocked)` block to the top of checkAvailability(), before the AC1 checks. Do not change any other condition, code, reason text, App.tsx or any Available/Unavailable decision. Tell me exactly which lines moved.

Response (exact, S-15)

> [tool steps collapsed in UI: actions taken]
> Now remove the original blocked-room block that's now duplicated further down.
> [tool steps collapsed in UI: actions taken]
> Build passes. Here's exactly what moved in src/lib/availability.ts:
>
> The existing AC3 blocked-room check (the if (room.blocked) block with its code: "room_blocked" return and reason text) was moved from its original position — after the AC1 invalid-time check, the AC1 start-not-after-now check, and the AC2 duration check — to the top of checkAvailability(), before all the AC1 checks. No condition, reason text, or Available/Unavailable decision changed; App.tsx and every other rule are untouched.
>
> Now a request for a blocked room reports "room is blocked" first, so the student learns the decisive reason before being asked to fix their times.

My review decision / retest

Accepted after: code check - block at lines 47-53, no second copy after AC2 (S-16); my own `npm run build` (S-18); Vitest A15 (S-20); UI shows the blocked reason (S-26).

E-C1 - Claude - Acceptance test file for the final Bolt version (24 Oct 2026)

Why: Task 5 asks to rerun the complete acceptance set on the final version, and the Bolt free plan had 53K tokens left (S-15), so instead of another Bolt prompt the test file was drafted with Claude and written into the project from the Bolt terminal (`node -e` writing the file), then run with `npx vitest run`.

File `src/lib/availability.test.ts` (exact)

```
import { describe, it, expect } from "vitest";
import { checkAvailability, type Room } from "./availability";

const NOW = 540;
const mk = (o: Partial<Room> = {}): Room => ({ id: "r1", name: "Test Room", location:
"B", blocked: false, bookings: [{ roomId: "r1", start: 600, end: 660 }], ...o });
const run = (start: number, end: number, room: Room = mk(), now = NOW) => {
  const before = JSON.stringify(room);
  const res = checkAvailability({ now, start, end, room });
  expect(JSON.stringify(room)).toBe(before);
  return res;
};

describe("A3 acceptance set (AC1-AC5)", () => {
  it("A01 touching 660-720 -> Available (AC4)", () => { expect(run(660,
720).available).toBe(true); });
  it("A02 overlap 630-690 -> Unavailable (AC4)", () => { expect(run(630,
```

```
690).available).toBe(false); });
  it("A03 blocked 660-720 -> Unavailable (AC3)", () => { expect(run(660, 720, mk({
blocked: true }),).available).toBe(false); });
  it("A04 exactly 120 720-840 -> Available (AC2)", () => { expect(run(720,
840).available).toBe(true); });
  it("A05 121 min 720-841 -> Unavailable (AC2)", () => { expect(run(720,
841).available).toBe(false); });
  it("A06 starts now 540-570 -> Unavailable (AC1)", () => { expect(run(540,
570).available).toBe(false); });
  it("A07 reversed 720-660 -> Unavailable (AC1)", () => { expect(run(720,
660).available).toBe(false); });
  it("A08 zero length 700-700 -> Unavailable (AC1)", () => { expect(run(700,
700).available).toBe(false); });
  it("A09 day end 1380-1440 -> Available (AC1)", () => { expect(run(1380,
1440).available).toBe(true); });
  it("A10 past day end 1400-1441 -> Unavailable (AC1)", () => { expect(run(1400,
1441).available).toBe(false); });
  it("A11 negative start -10..30 -> Unavailable (AC1)", () => { expect(run(-10, 30,
mk({ bookings: [] }), 0).available).toBe(false); });
  it("A12 non-integer 660.5-720 -> Unavailable (AC1)", () => { expect(run(660.5,
720).available).toBe(false); });
  it("A13 multiple bookings, gap fits 660-720 -> Available (AC4)", () => {
expect(run(660, 720, mk({ bookings: [{ roomId: "r1", start: 600, end: 660 }, { roomId:
"r1", start: 720, end: 780 }, { roomId: "r1", start: 900, end: 960 }]
})).available).toBe(true); });
  it("A14 multiple bookings, hits 3rd 850-910 -> Unavailable (AC4)", () => {
expect(run(850, 910, mk({ bookings: [{ roomId: "r1", start: 600, end: 660 }, { roomId:
"r1", start: 720, end: 780 }, { roomId: "r1", start: 900, end: 960 }]
})).available).toBe(false); });
  it("A15 blocked + starts now -> reason says blocked (AC3, V1-03)", () => { const r =
run(540, 570, mk({ blocked: true })); expect(r.available).toBe(false);
expect(JSON.stringify(r)).toMatch(/blocked/i); });
});
```

My review decision / result

Each expected value comes from the AC and Task 4, not from the code's output. Result on Version 4: 15 passed, 22:56:32 (S-19, S-20). Not run on v1.

E-P1 - Claude - Generate Python v1 (24 Oct 2026)

Prompt

The Python part was done in one Claude (Cowork) session that I started with the assignment PDF and the request to complete Assignment 03 following my teacher's A2 feedback (my message, in Russian: "сделай мне на 100 процентов (assignment 03) и учти вот это (photo)"). The contract from report section 1.4 and AC1-AC5 were the specification.

Response - generated v1, saved unchanged as `v1/booking.py`

```
def can_book(start, end, now, blocked, existing):
    if blocked:
        return False
    if start < 0 or end > 1440 or start >= end:
        return False
    if start <= now:
```

```
        return False
    if end - start > 120:
        return False
    for s, e in existing:
        if start < e and s < end:
            return False
    return True
```

My review decision

Branches correct for AC2-AC5 and the AC1 bounds. Missing: AC1 says *integer* minutes and nothing checks that. Test T20 targets it. v1 result: 20/21, T20 fails (section C).

E-P2 - Claude - Repair P-01 (24 Oct 2026)

Input to the repair step (observed failure)

```
FAIL: test_T20_non_integer_minutes_rejected
    self.assertFalse(can_book(660.5, 720, NOW, False, EXISTING))
AssertionError: True is not false
probe: 660.5-720 -> True | 660-720.0 -> True | True-60 now=0 -> True
```

Change (only this, all other behaviour kept)

```
+ def _is_minute(value):
+     return isinstance(value, int) and not isinstance(value, bool)
+ if not (_is_minute(start) and _is_minute(end)):
+     return False
  constants DAY_START / DAY_END / MAX_DURATION instead of magic numbers, docstring
```

My review decision

Accepted. Rejected the idea of raising `TypeError` for bad input - the contract says return False, and the AC are fixed. Retest: 21/21 (section C).

C. Raw test output

C.1 Python v1 (`BOOKING_IMPL=v1 python3 -m unittest -v test_booking`)

```
test_T07_reversed_times (test_booking.ExtraCases.test_T07_reversed_times) ... ok
test_T08_zero_length (test_booking.ExtraCases.test_T08_zero_length) ... ok
test_T09_end_exactly_1440_is_available
(test_booking.ExtraCases.test_T09_end_exactly_1440_is_available) ... ok
test_T10_end_after_1440 (test_booking.ExtraCases.test_T10_end_after_1440) ... ok
test_T11_negative_start (test_booking.ExtraCases.test_T11_negative_start) ... ok
test_T12_start_in_past (test_booking.ExtraCases.test_T12_start_in_past) ... ok
test_T13_one_minute_after_now (test_booking.ExtraCases.test_T13_one_minute_after_now) ...
ok
test_T14_touching_start_of_booking
(test_booking.ExtraCases.test_T14_touching_start_of_booking) ... ok
test_T15_inside_existing (test_booking.ExtraCases.test_T15_inside_existing) ... ok
test_T16_covers_existing (test_booking.ExtraCases.test_T16_covers_existing) ... ok
test_T17_multiple_bookings_gap_fits
```

```
(test_booking.ExtraCases.test_T17_multiple_bookings_gap_fits) ... ok
test_T18_multiple_bookings_hits_later_one
(test_booking.ExtraCases.test_T18_multiple_bookings_hits_later_one) ... ok
test_T19_inputs_unchanged (test_booking.ExtraCases.test_T19_inputs_unchanged) ... ok
test_T20_non_integer_minutes_rejected
(test_booking.ExtraCases.test_T20_non_integer_minutes_rejected) ... FAIL
test_T21_returns_real_bool (test_booking.ExtraCases.test_T21_returns_real_bool) ... ok
test_T01_touching_end_is_available
(test_booking.RequiredCases.test_T01_touching_end_is_available) ... ok
test_T02_overlap_is_unavailable
(test_booking.RequiredCases.test_T02_overlap_is_unavailable) ... ok
test_T03_blocked_room_is_unavailable
(test_booking.RequiredCases.test_T03_blocked_room_is_unavailable) ... ok
test_T04_exactly_120_is_available
(test_booking.RequiredCases.test_T04_exactly_120_is_available) ... ok
test_T05_121_minutes_is_unavailable
(test_booking.RequiredCases.test_T05_121_minutes_is_unavailable) ... ok
test_T06_starts_now_is_unavailable
(test_booking.RequiredCases.test_T06_starts_now_is_unavailable) ... ok

======================================================================
FAIL: test_T20_non_integer_minutes_rejected
(test_booking.ExtraCases.test_T20_non_integer_minutes_rejected)
----------------------------------------------------------------------
Traceback (most recent call last):
  File "/home/claude/a3/python/test_booking.py", line 95, in
test_T20_non_integer_minutes_rejected
    self.assertFalse(can_book(660.5, 720, NOW, False, EXISTING))
    ~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
AssertionError: True is not false

----------------------------------------------------------------------
Ran 21 tests in 0.002s

FAILED (failures=1)
```

C.2 Python v1 type probe

```
660.5-720 -> True
660-720.0 -> True
True-60 now=0 -> True
```

C.3 Python final (`python3 -m unittest -v test_booking`)

```
test_T07_reversed_times (test_booking.ExtraCases.test_T07_reversed_times) ... ok
test_T08_zero_length (test_booking.ExtraCases.test_T08_zero_length) ... ok
test_T09_end_exactly_1440_is_available
(test_booking.ExtraCases.test_T09_end_exactly_1440_is_available) ... ok
test_T10_end_after_1440 (test_booking.ExtraCases.test_T10_end_after_1440) ... ok
test_T11_negative_start (test_booking.ExtraCases.test_T11_negative_start) ... ok
test_T12_start_in_past (test_booking.ExtraCases.test_T12_start_in_past) ... ok
test_T13_one_minute_after_now (test_booking.ExtraCases.test_T13_one_minute_after_now) ...
ok
test_T14_touching_start_of_booking
(test_booking.ExtraCases.test_T14_touching_start_of_booking) ... ok
test_T15_inside_existing (test_booking.ExtraCases.test_T15_inside_existing) ... ok
test_T16_covers_existing (test_booking.ExtraCases.test_T16_covers_existing) ... ok
test_T17_multiple_bookings_gap_fits
```

```
(test_booking.ExtraCases.test_T17_multiple_bookings_gap_fits) ... ok
test_T18_multiple_bookings_hits_later_one
(test_booking.ExtraCases.test_T18_multiple_bookings_hits_later_one) ... ok
test_T19_inputs_unchanged (test_booking.ExtraCases.test_T19_inputs_unchanged) ... ok
test_T20_non_integer_minutes_rejected
(test_booking.ExtraCases.test_T20_non_integer_minutes_rejected) ... ok
test_T21_returns_real_bool (test_booking.ExtraCases.test_T21_returns_real_bool) ... ok
test_T01_touching_end_is_available
(test_booking.RequiredCases.test_T01_touching_end_is_available) ... ok
test_T02_overlap_is_unavailable
(test_booking.RequiredCases.test_T02_overlap_is_unavailable) ... ok
test_T03_blocked_room_is_unavailable
(test_booking.RequiredCases.test_T03_blocked_room_is_unavailable) ... ok
test_T04_exactly_120_is_available
(test_booking.RequiredCases.test_T04_exactly_120_is_available) ... ok
test_T05_121_minutes_is_unavailable
(test_booking.RequiredCases.test_T05_121_minutes_is_unavailable) ... ok
test_T06_starts_now_is_unavailable
(test_booking.RequiredCases.test_T06_starts_now_is_unavailable) ... ok

----------------------------------------------------------------------
Ran 21 tests in 0.001s

OK
```

C.4 Bolt (A3 project)

`npm run build`: built in 1.82s (S-18). `npx vitest run` on Version 4: Test Files 1 passed, Tests 15 passed, start 22:56:32, duration 2.50s (S-19, S-20). Both typed by me in the Bolt terminal.

D. Original screenshots and diagram (A3 project)

All taken 24 Oct 2026 in the Bolt.new tab. Original files: repository folder `screenshots/main/`; diagram source `diagrams/agent_loop.mmd`.

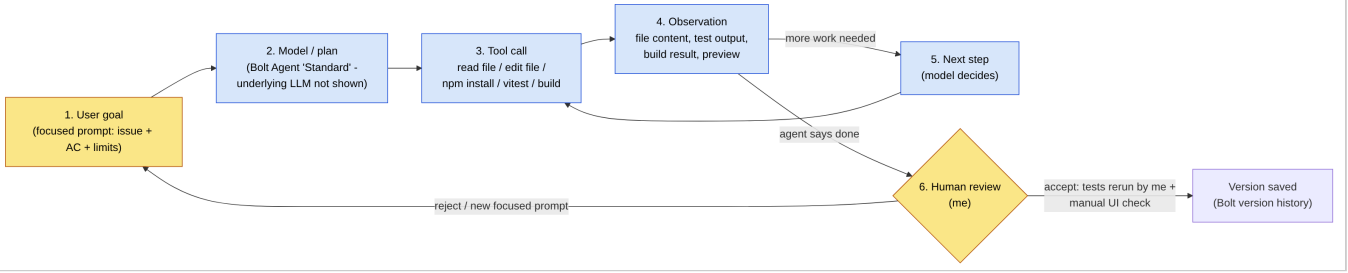
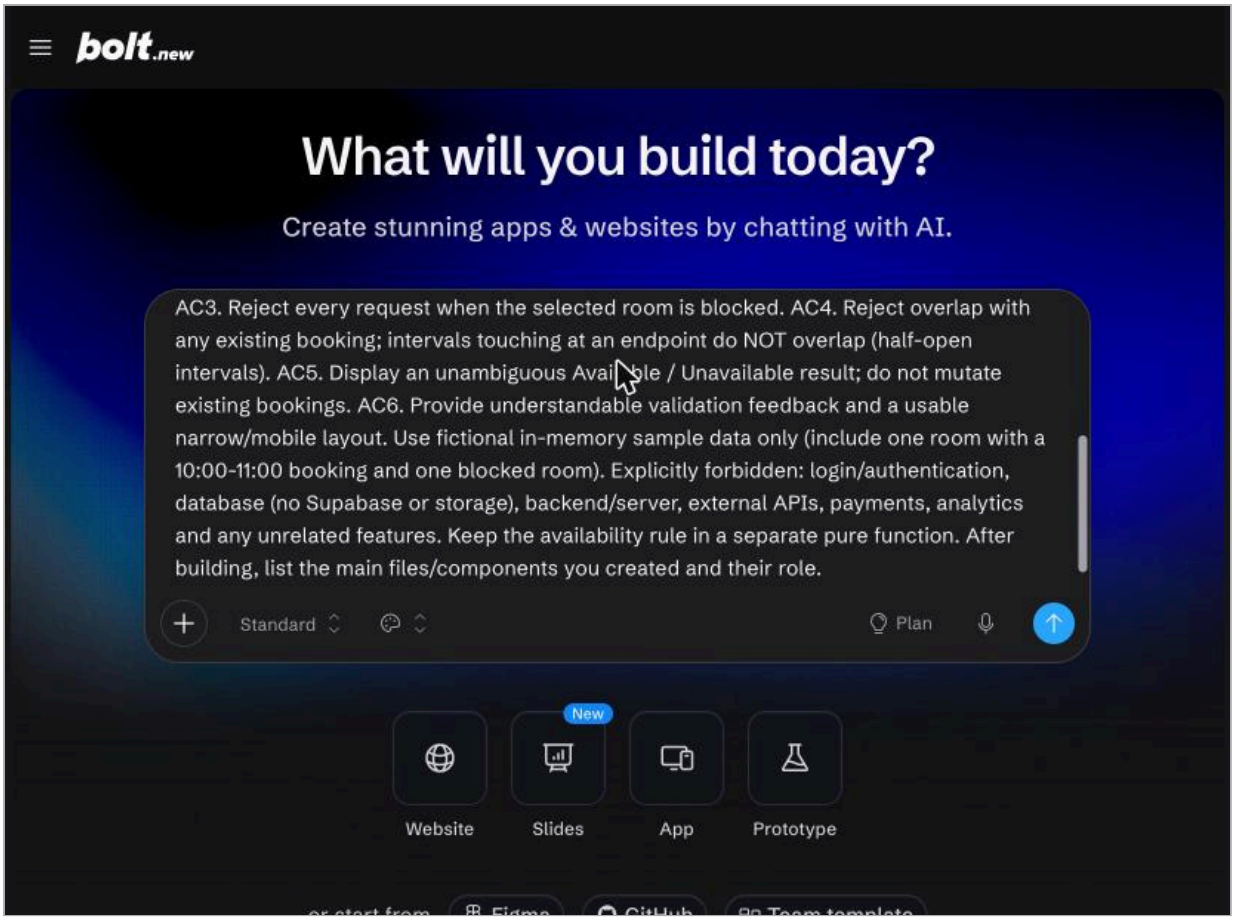
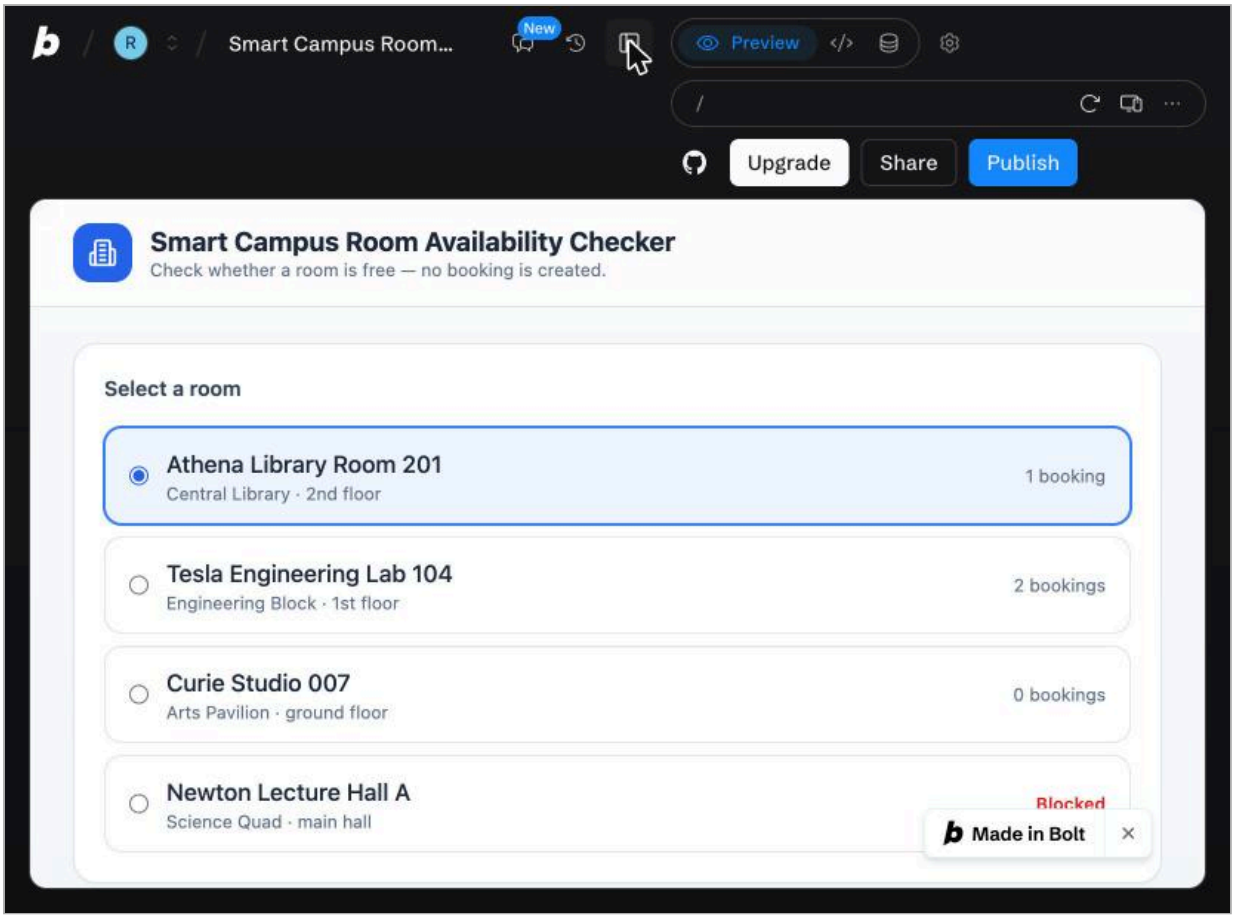


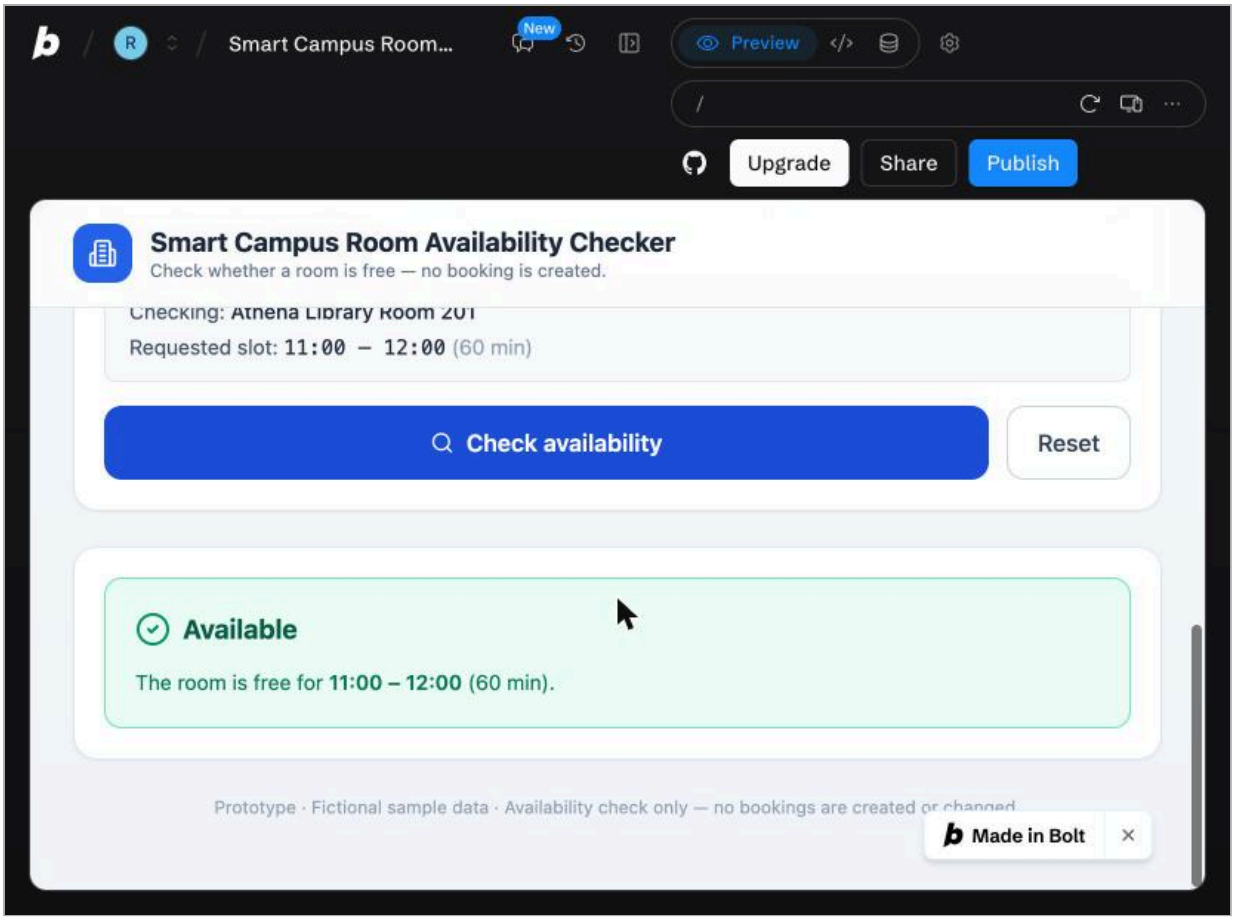
Figure 1 (original): agent loop diagram, rendered from agent_loop.mmd



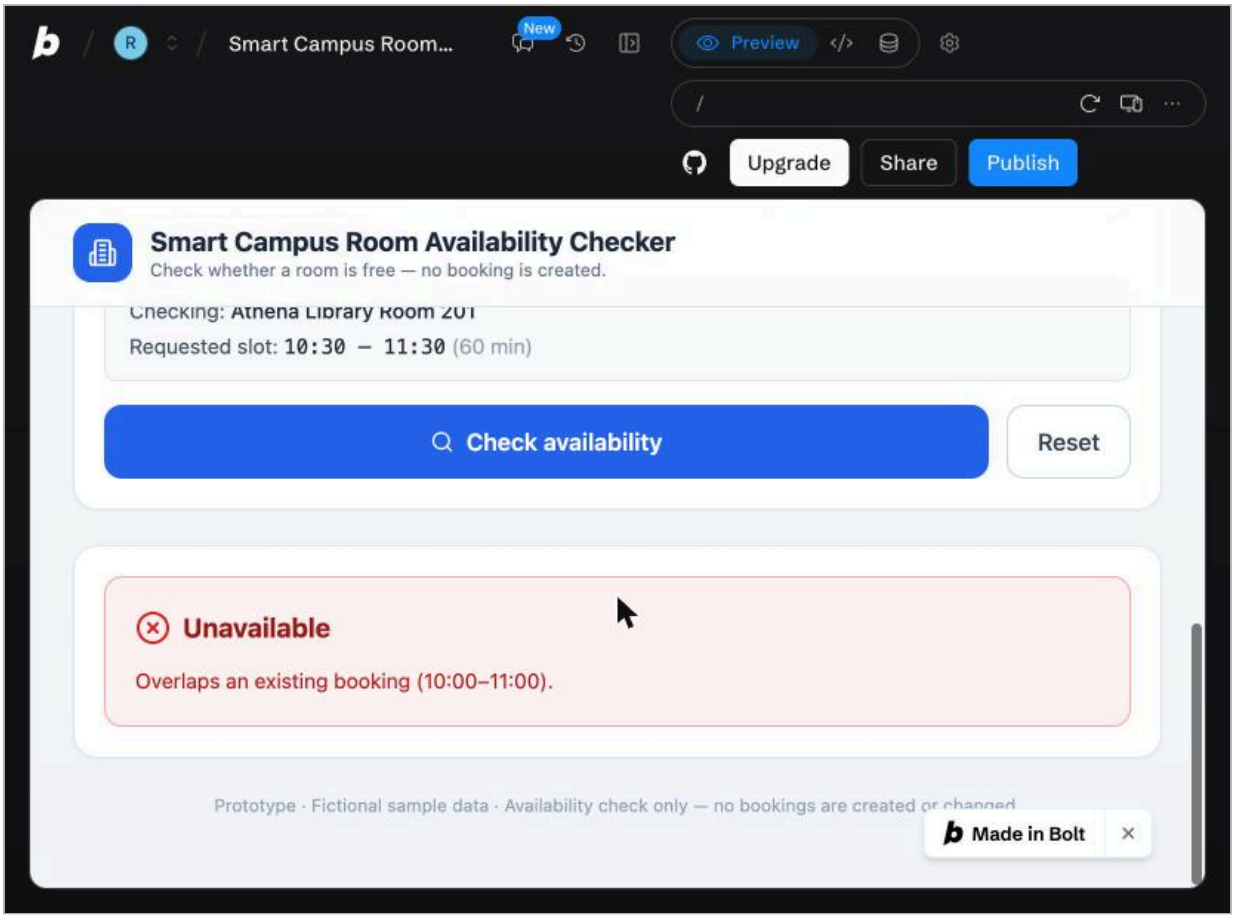
S-01 v1 prompt typed in a new Bolt project — 01_v1_prompt_typed.jpg



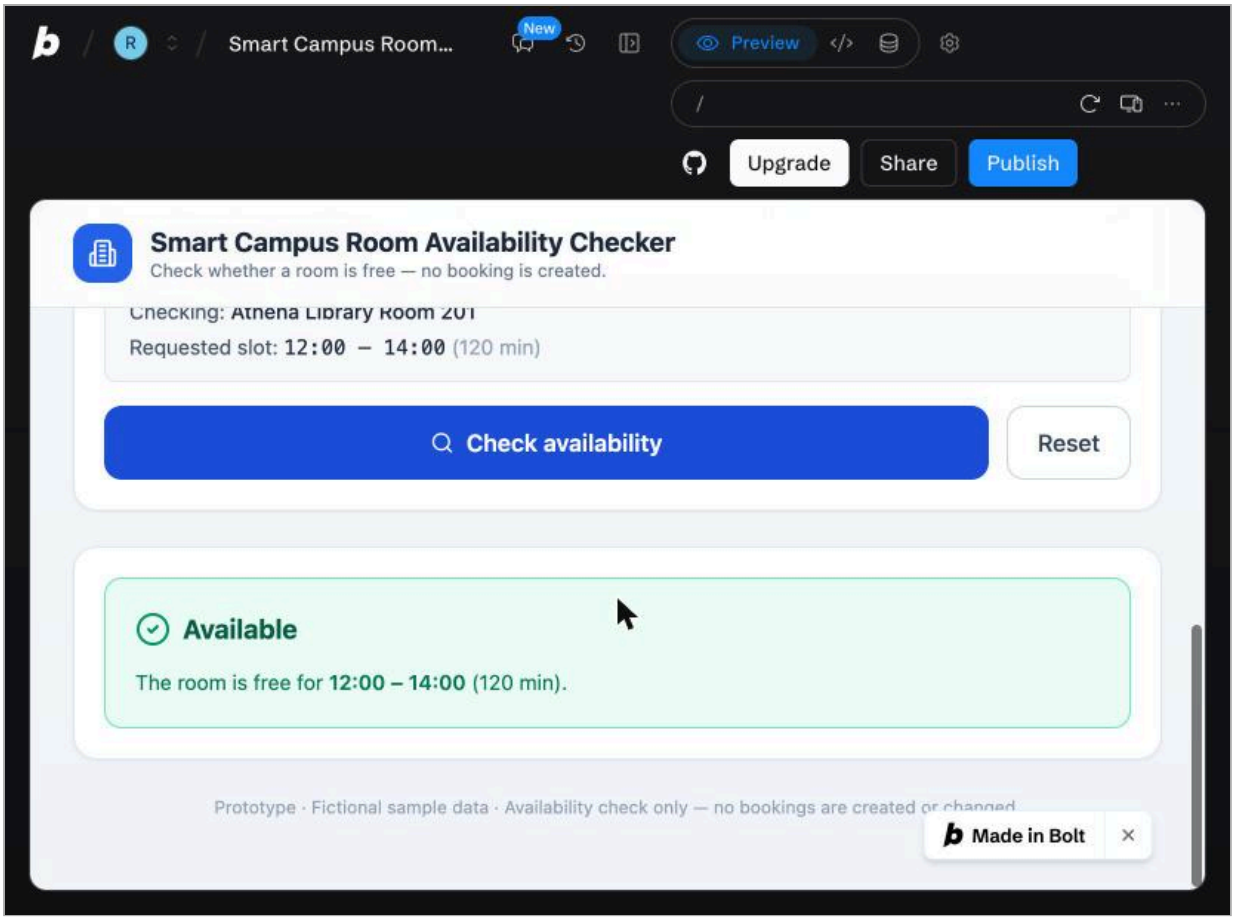
S-02 v1 first usable preview (desktop) — 02_v1_first_preview_desktop.jpg



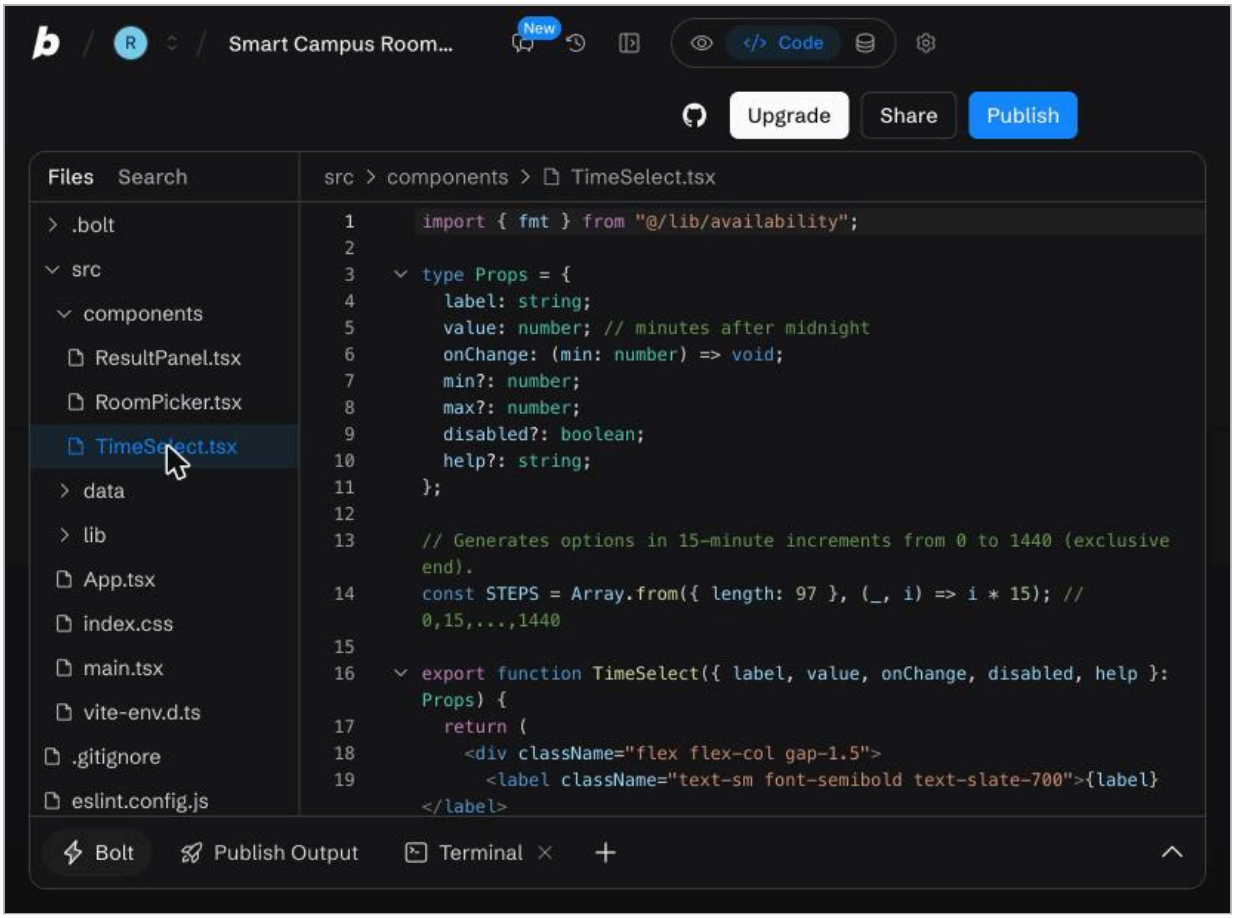
S-03 v1: 11:00-12:00 touching -> Available — 03_v1_T01_touching_available.jpg



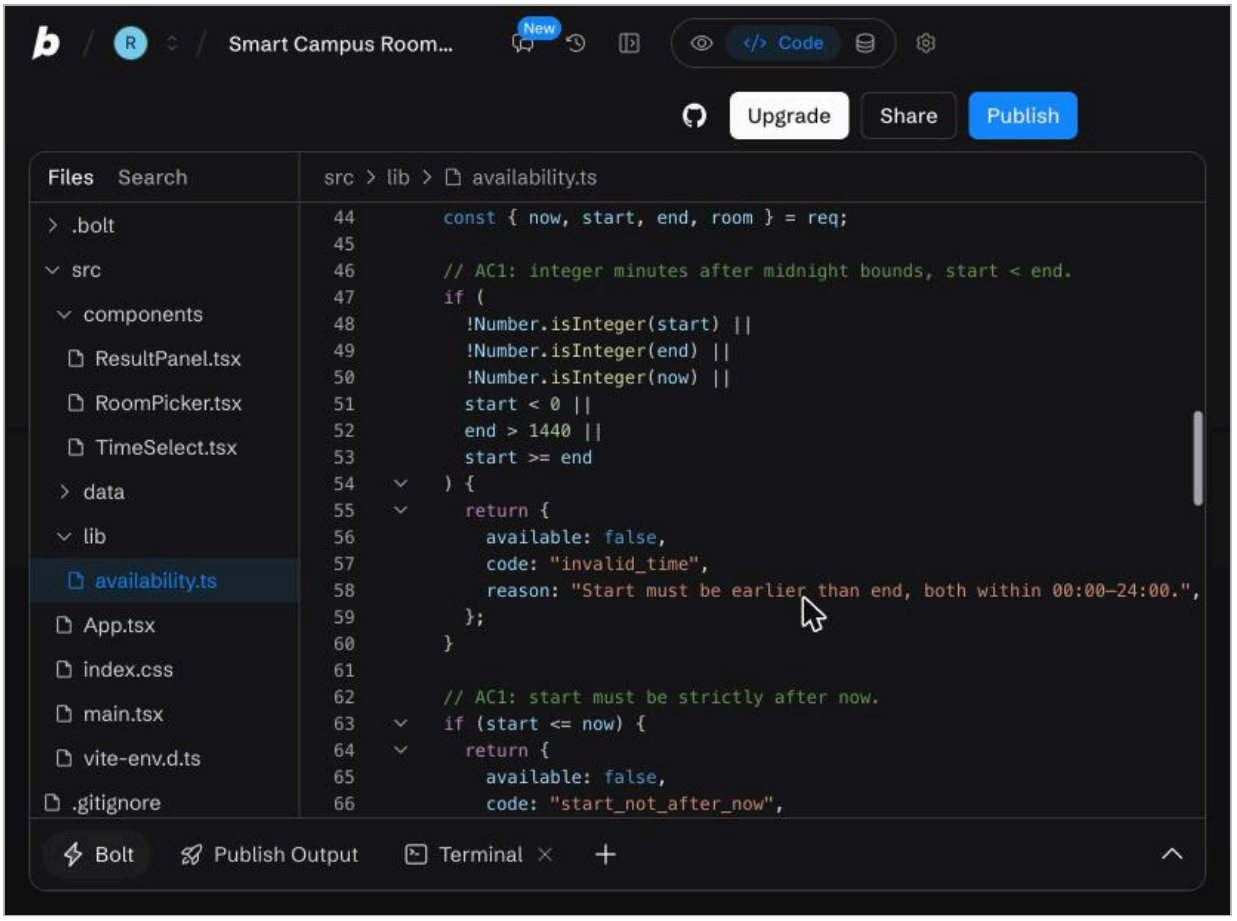
S-04 v1: 10:30-11:30 -> Unavailable (overlap) — 04_v1_T02_overlap_unavailable.jpg



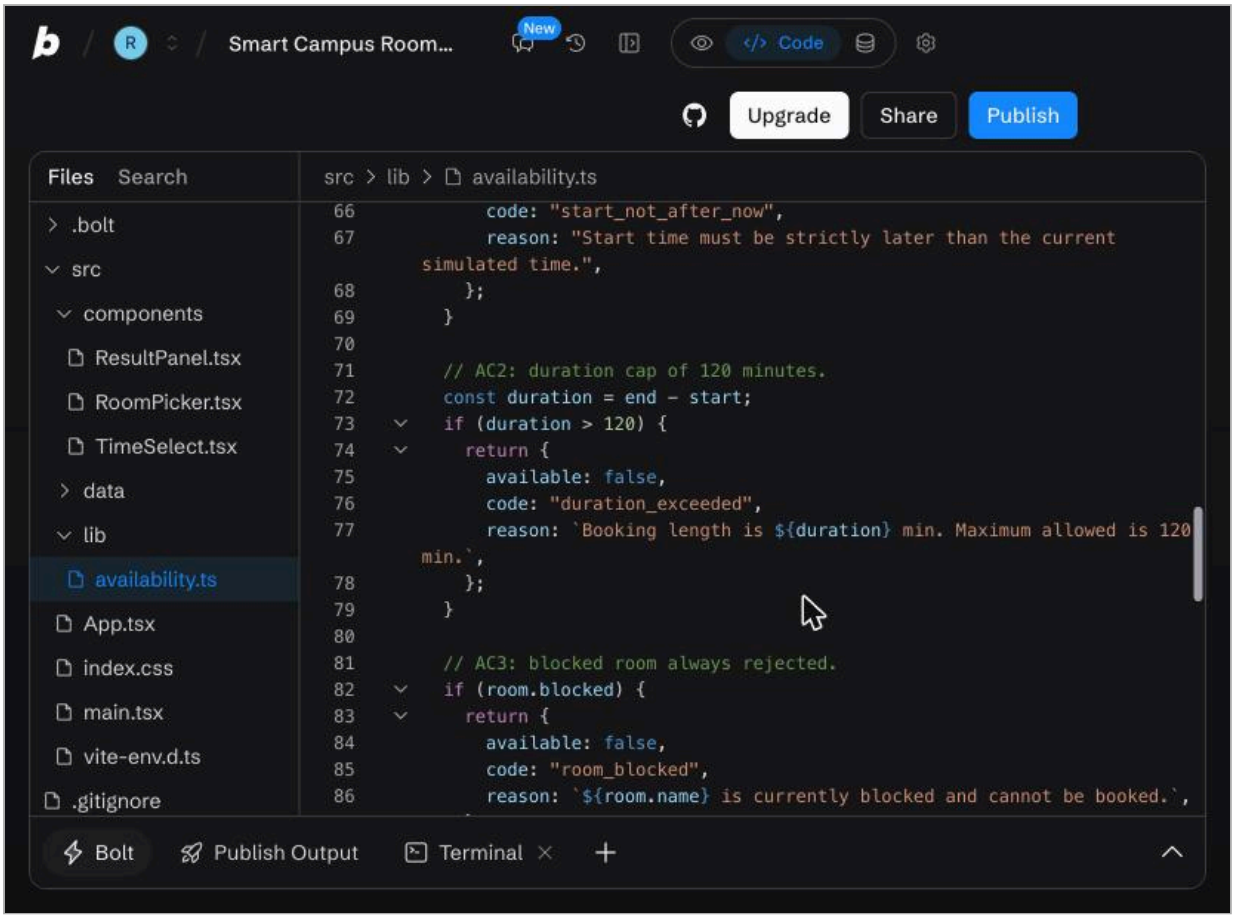
S-05 v1: 12:00-14:00 (120 min) -> Available — 05_v1_T04_120min_available.jpg



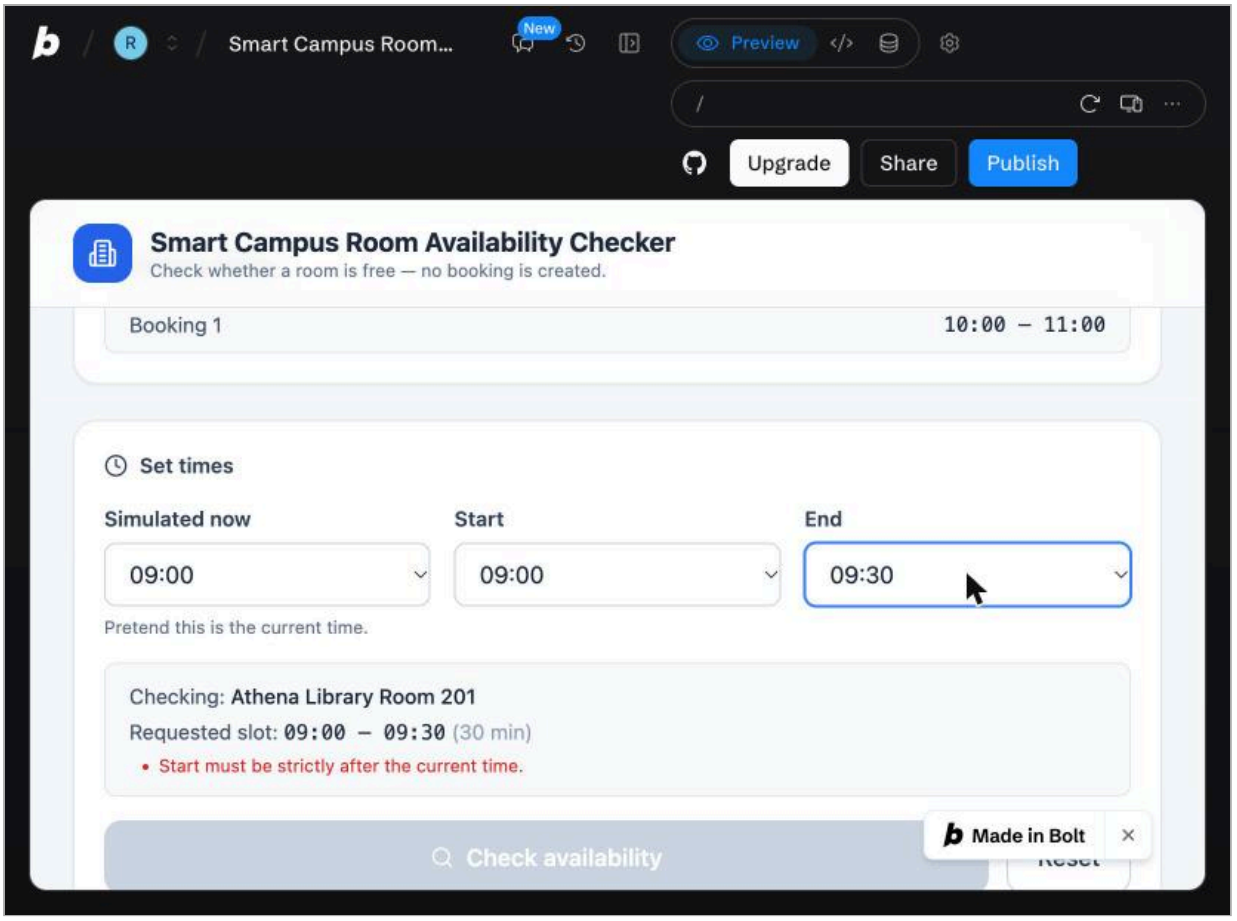
S-06 v1 code: TimeSelect 15-minute steps (line 14) - V1-02 — 06_v1_TimeSelect_15min_steps.jpg



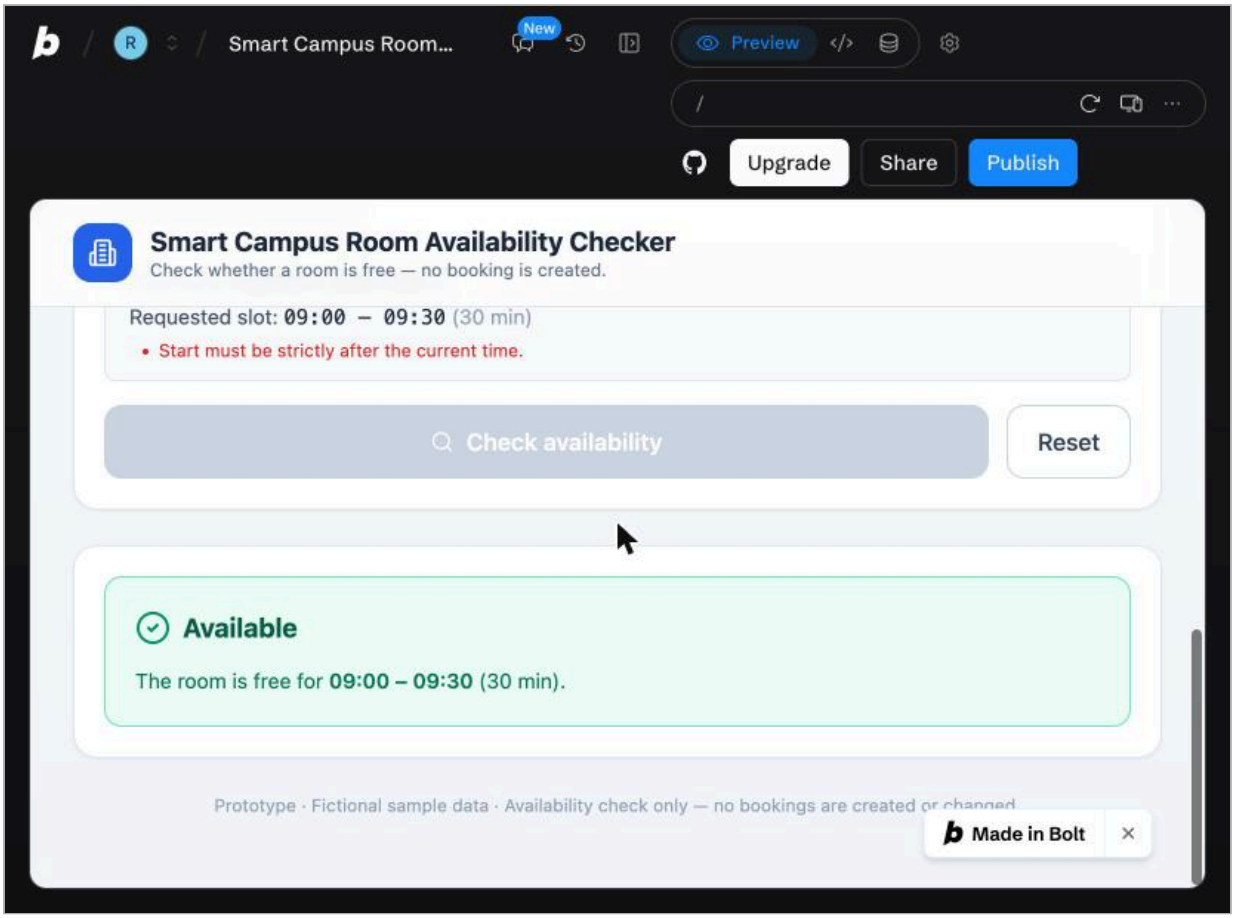
S-07 v1 code: AC1 checks incl. Number.isInteger — 07_v1_code_AC1_integer_checks.jpg



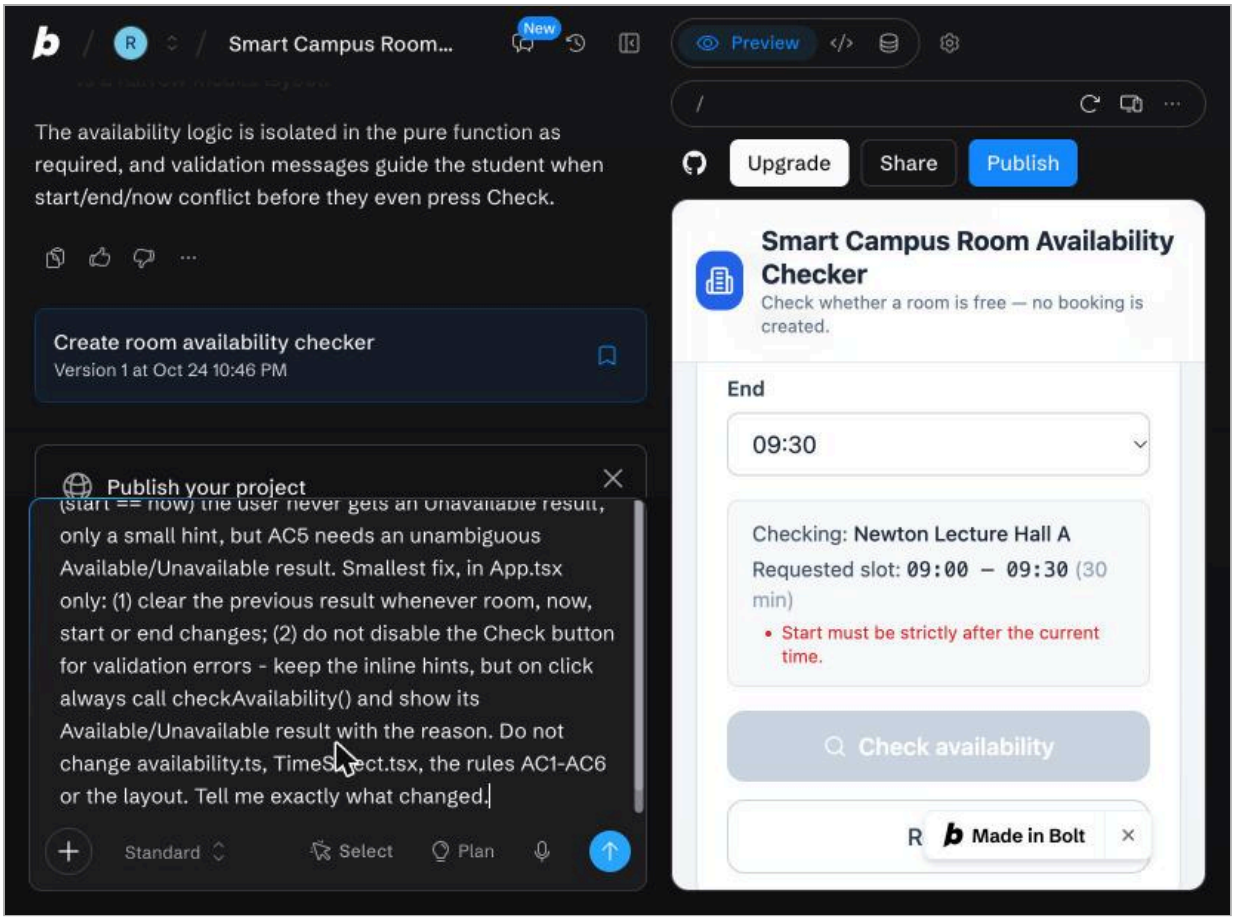
S-08 v1 code: blocked check placed after AC2 - V1-03 — 08_v1_code_blocked_after_AC2.jpg



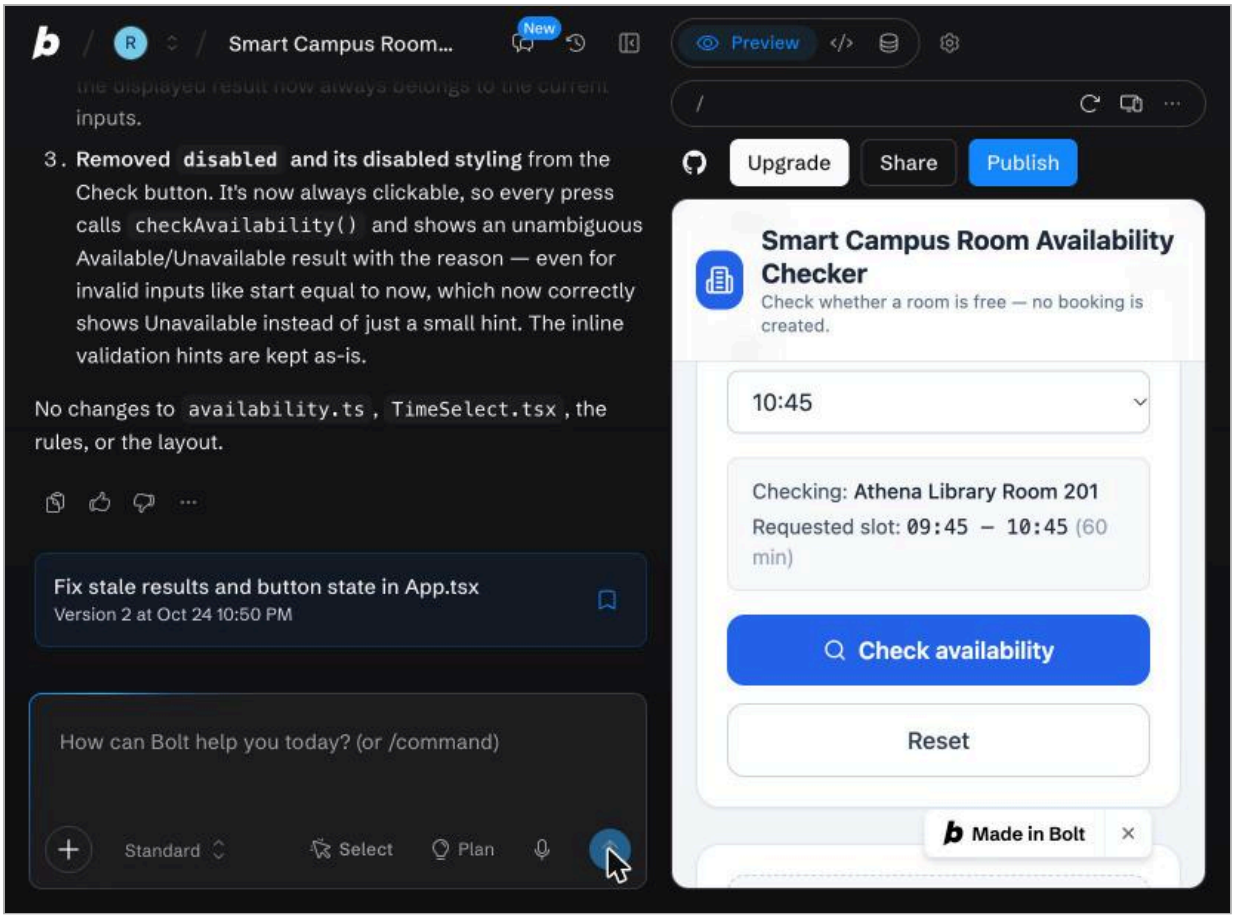
S-09 v1: start = now -> Check disabled, no result - V1-01 — 09_v1_starts_now_check_disabled_no_result.jpg



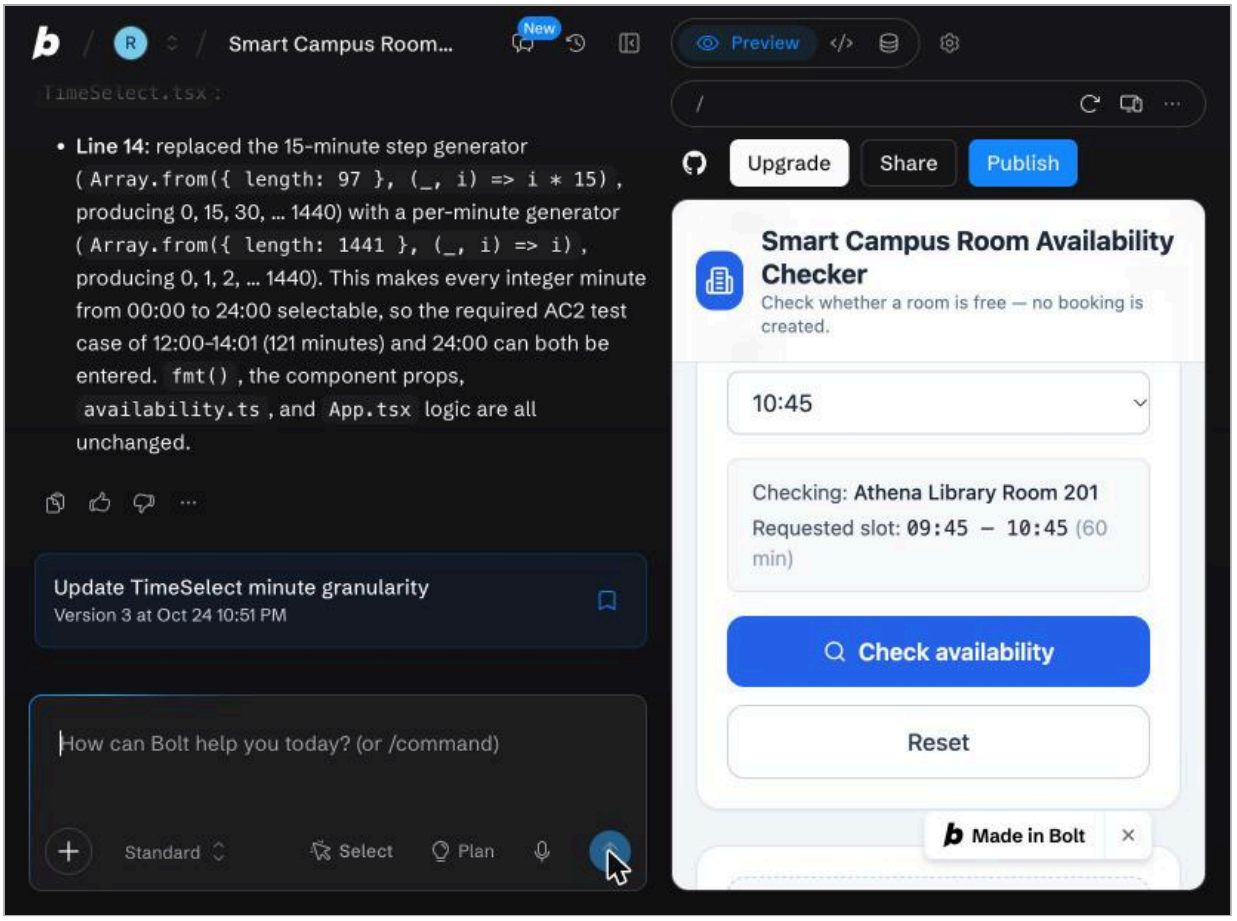
S-10 V1-01 BEFORE: stale "Available" shown for blocked Newton Hall, 09:00-09:30 — 10_V1-01_BEFORE_stale_Available_for_blocked_room.jpg



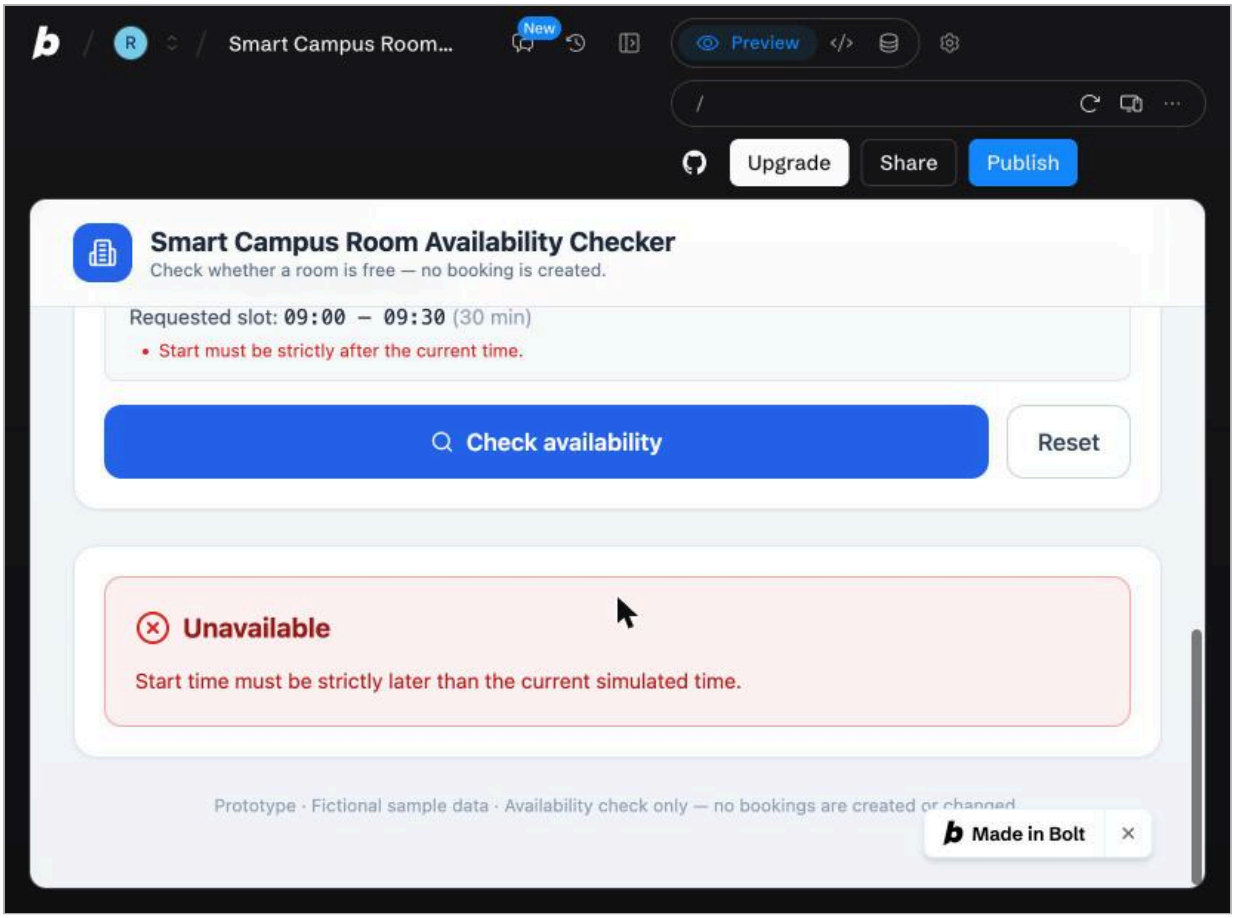
S-11 V1-01 focused prompt — 11_V1-01_prompt.jpg



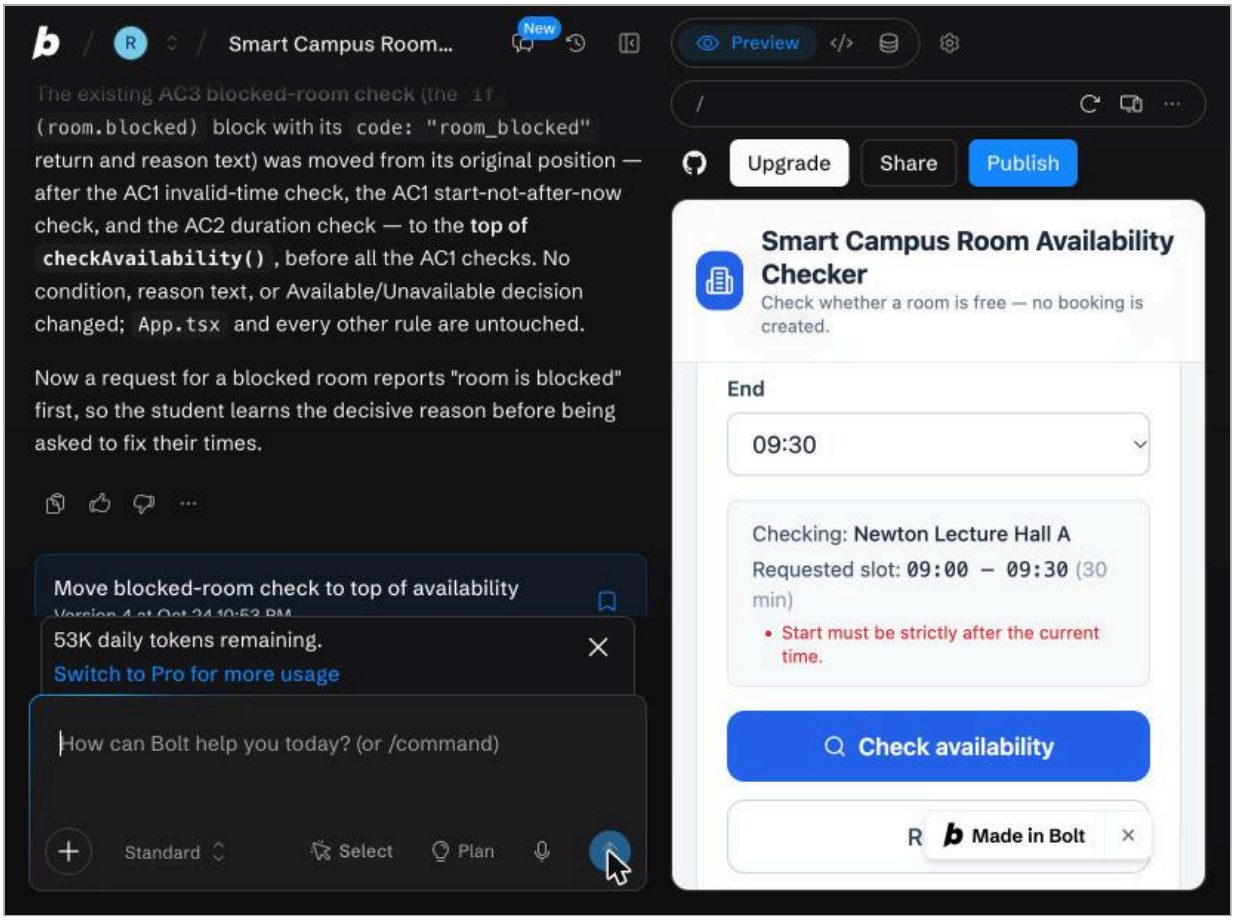
S-12 Bolt response, Version 2 (22:50) — 12_v2_response.jpg



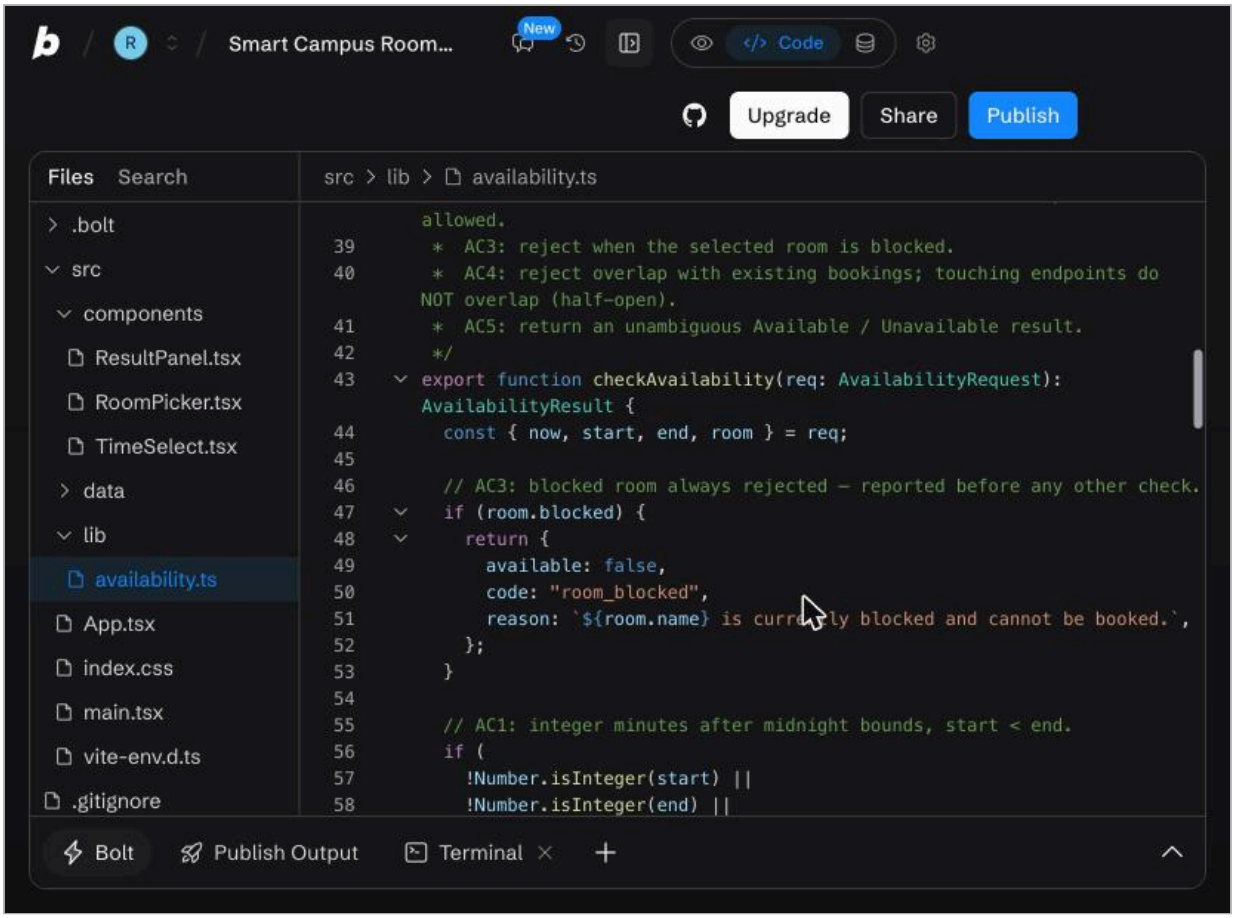
S-13 Bolt response, Version 3 (22:51) — 13_v3_response.jpg



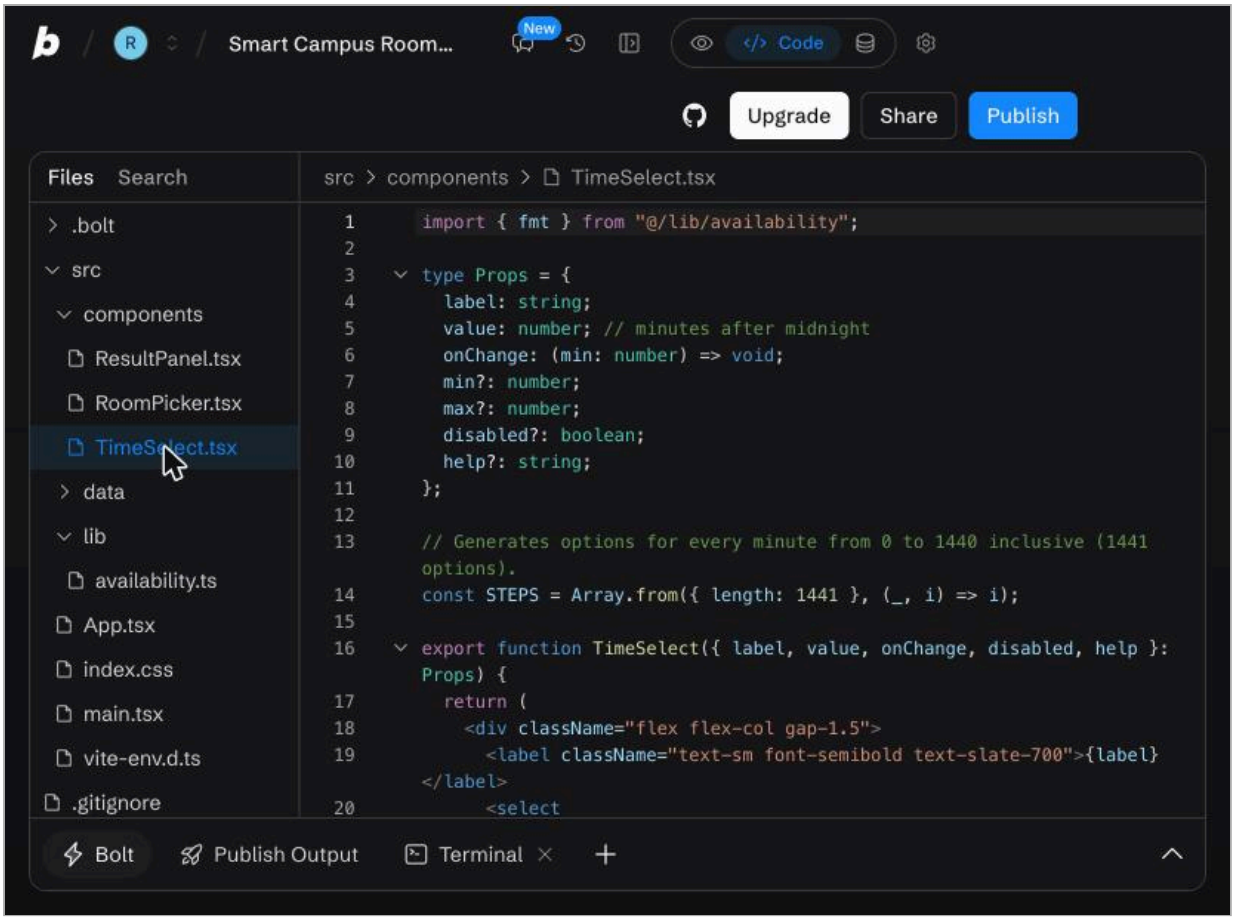
S-14 V1-03 BEFORE (v3): blocked room -> time reason — 14_V1-03_BEFORE_blocked_room_time_reason.jpg



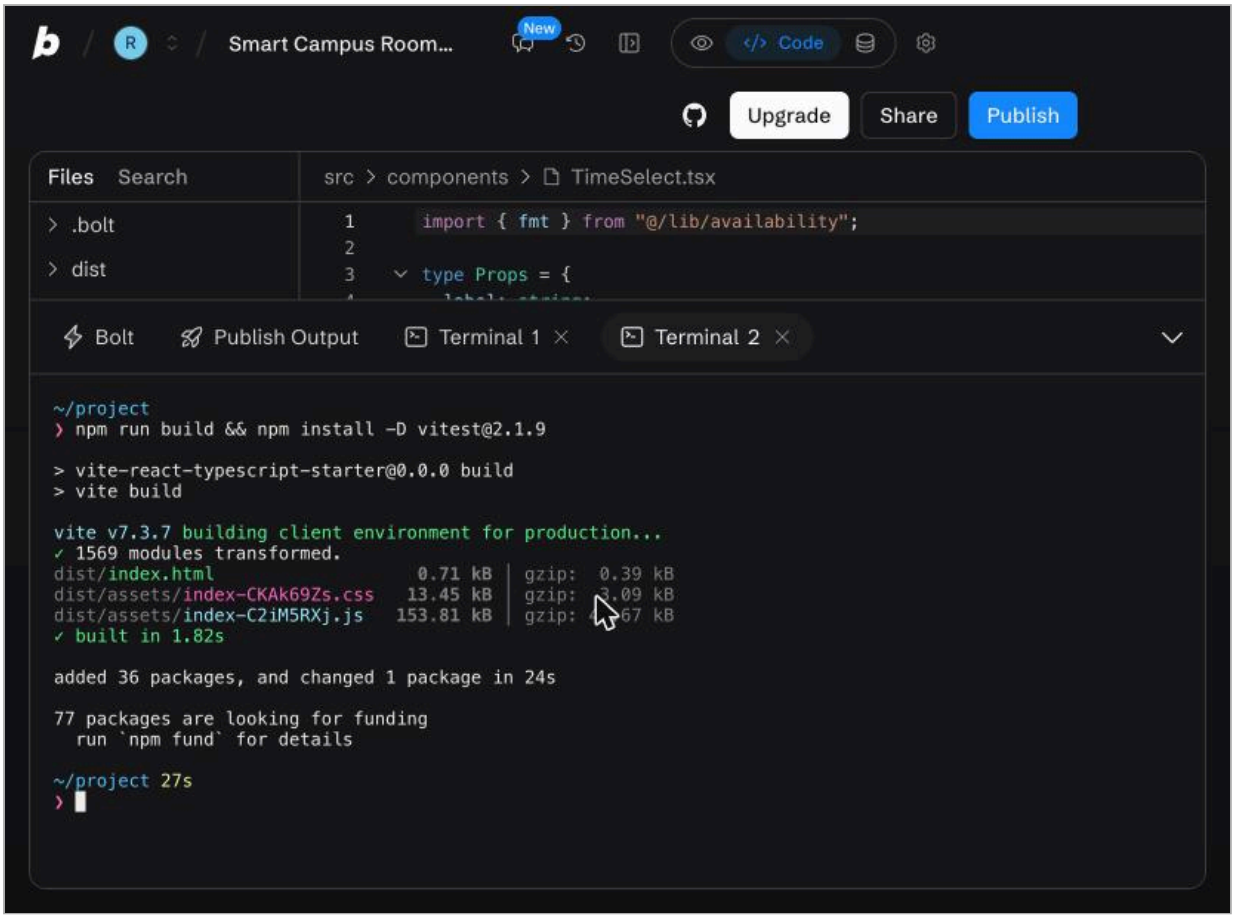
S-15 Bolt response, Version 4 (22:53) + daily token notice — 15_v4_response_token_notice.jpg



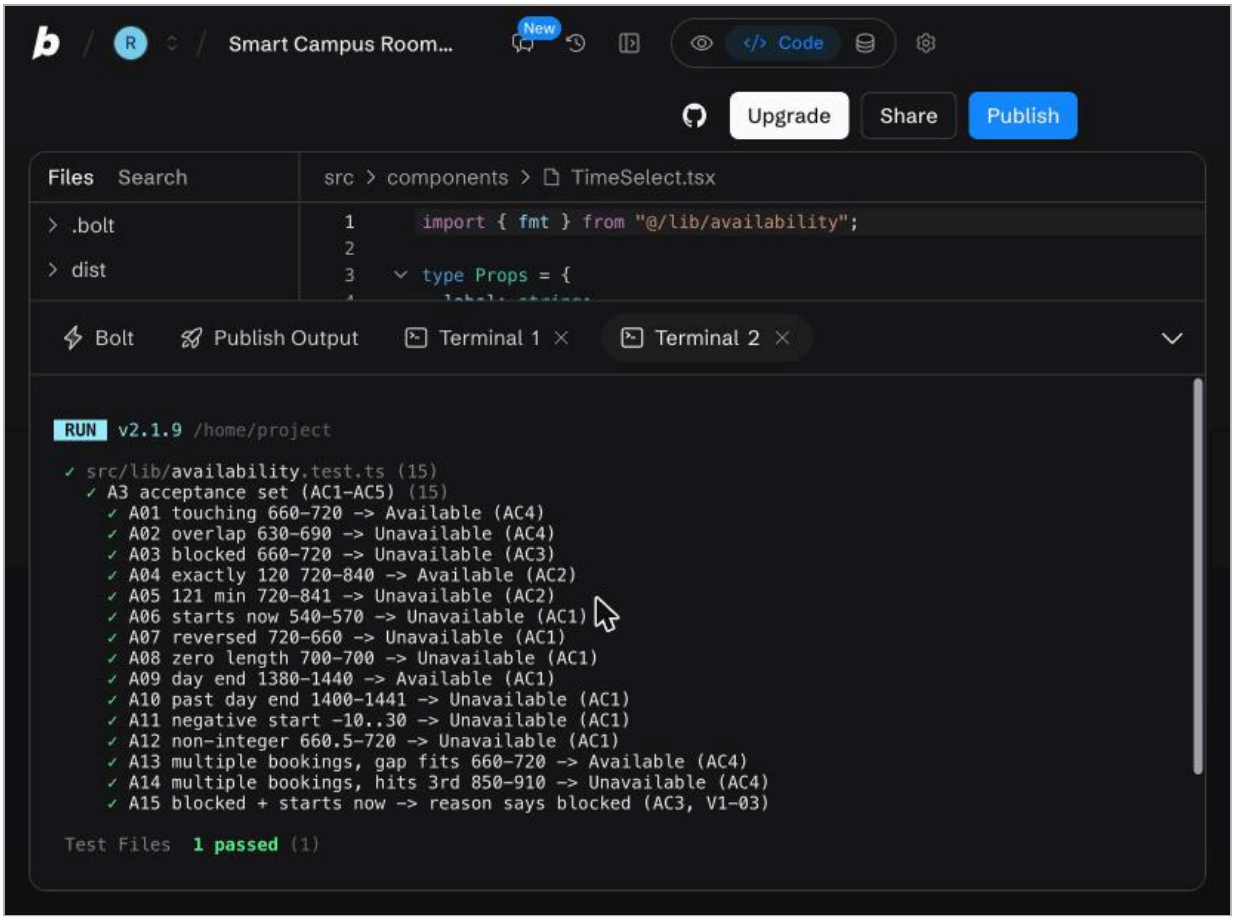
S-16 v4 code: blocked check first (lines 46-53) — 16_v4_code_blocked_first.jpg



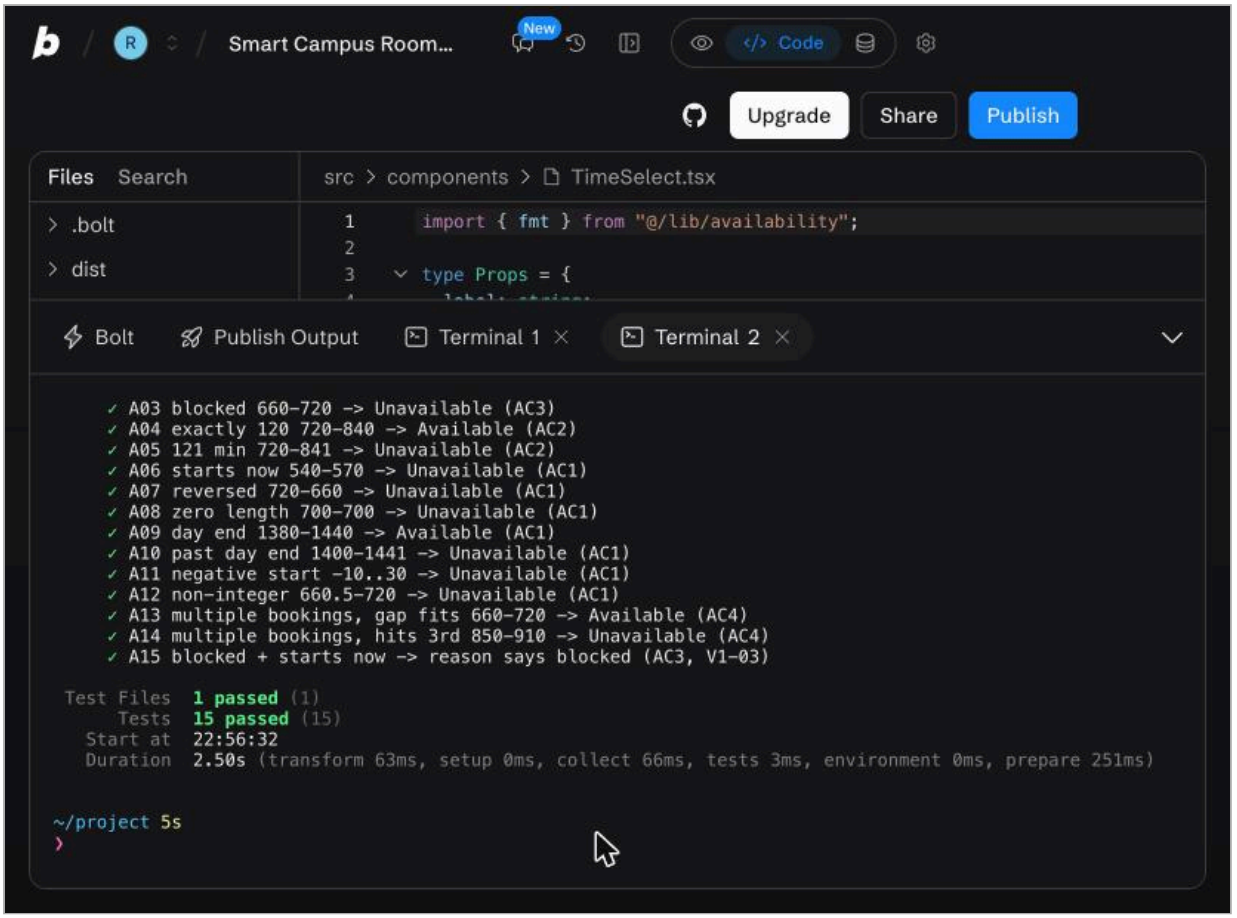
S-17 v4 code: TimeSelect every minute (1441 options) — 17_v4_TimeSelect_every_minute.jpg



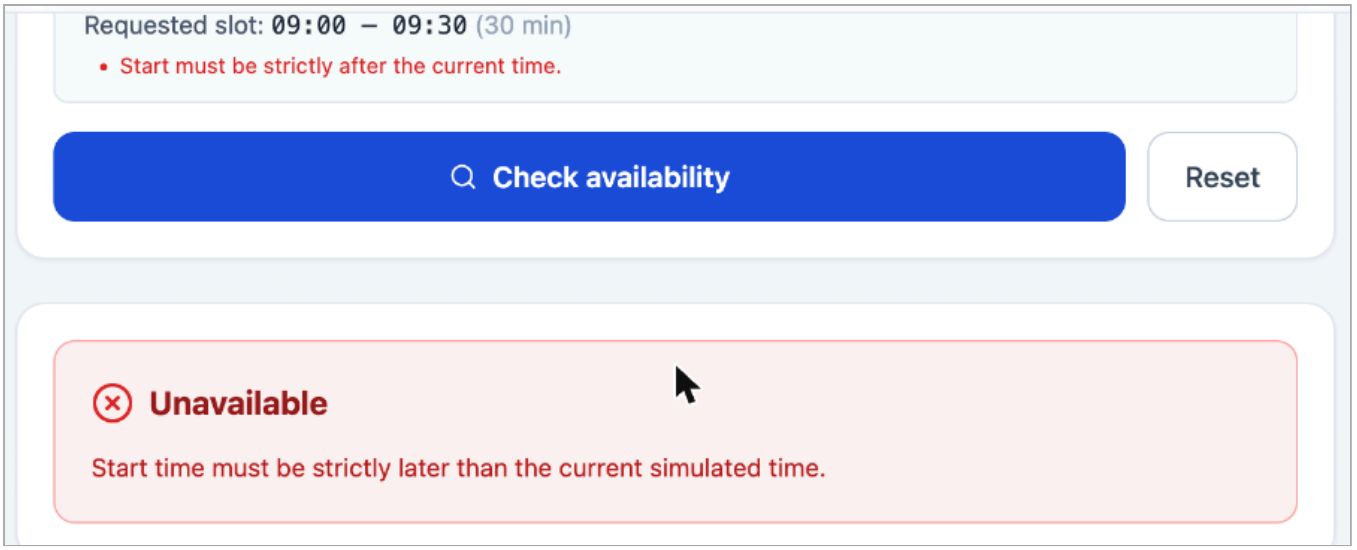
S-18 My run: npm run build OK, vitest installed — 18_build_ok_vitest_installed.jpg



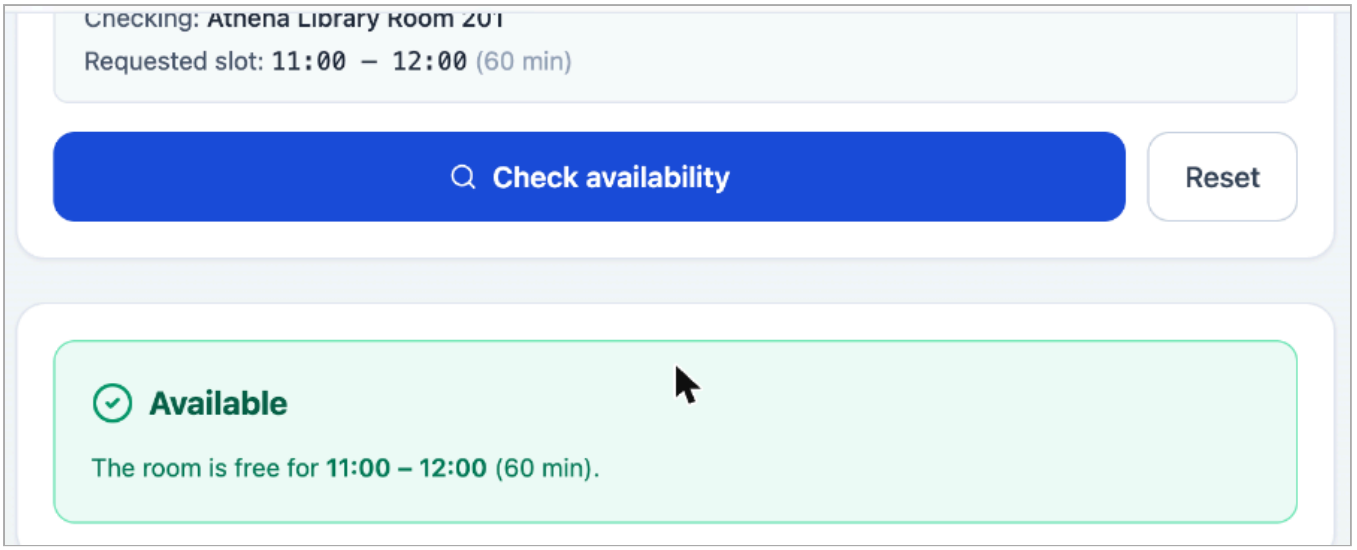
S-19 My run: Vitest A01-A15 list — 19_vitest_final_top.jpg



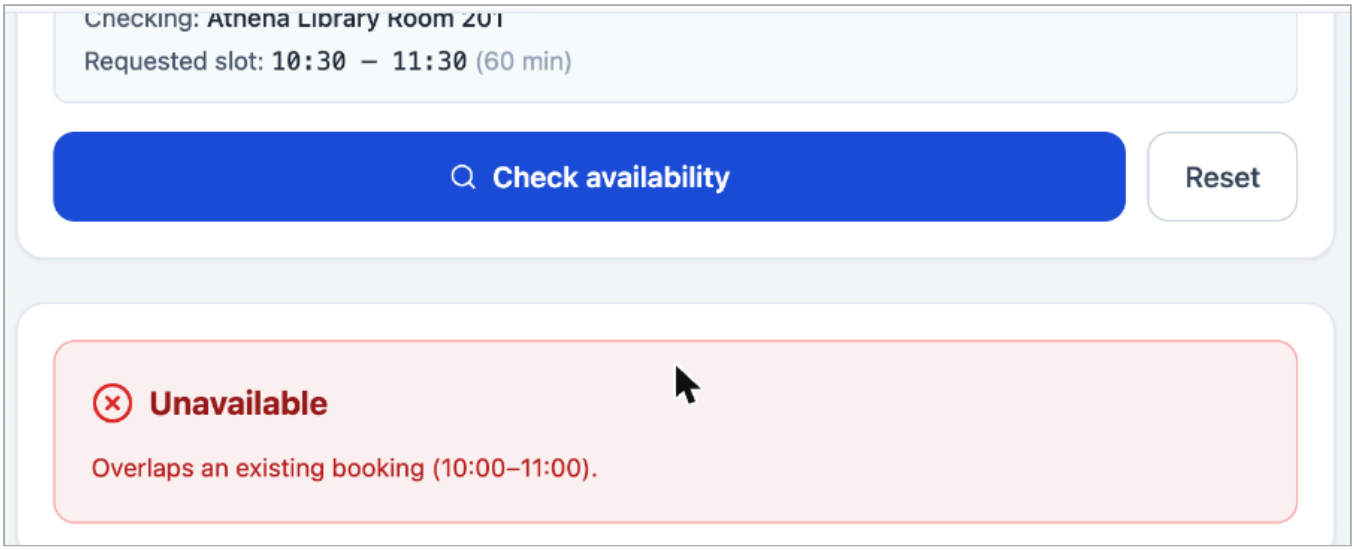
S-20 My run: Vitest 15 passed (22:56:32) — 20_vitest_final_15_passed.jpg



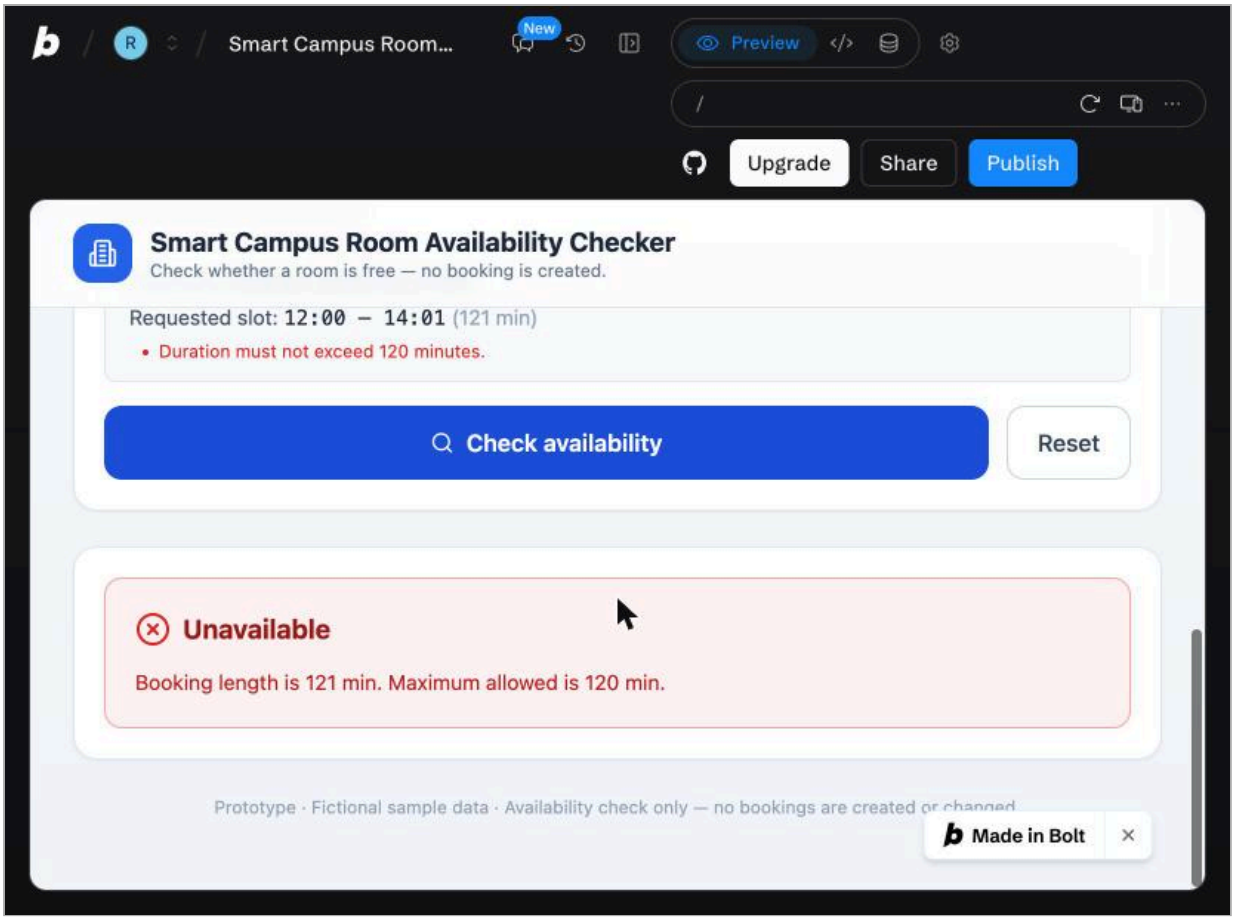
S-21 Final: 09:00-09:30 -> Unavailable (starts now) — 21_final_T06_starts_now_unavailable.png



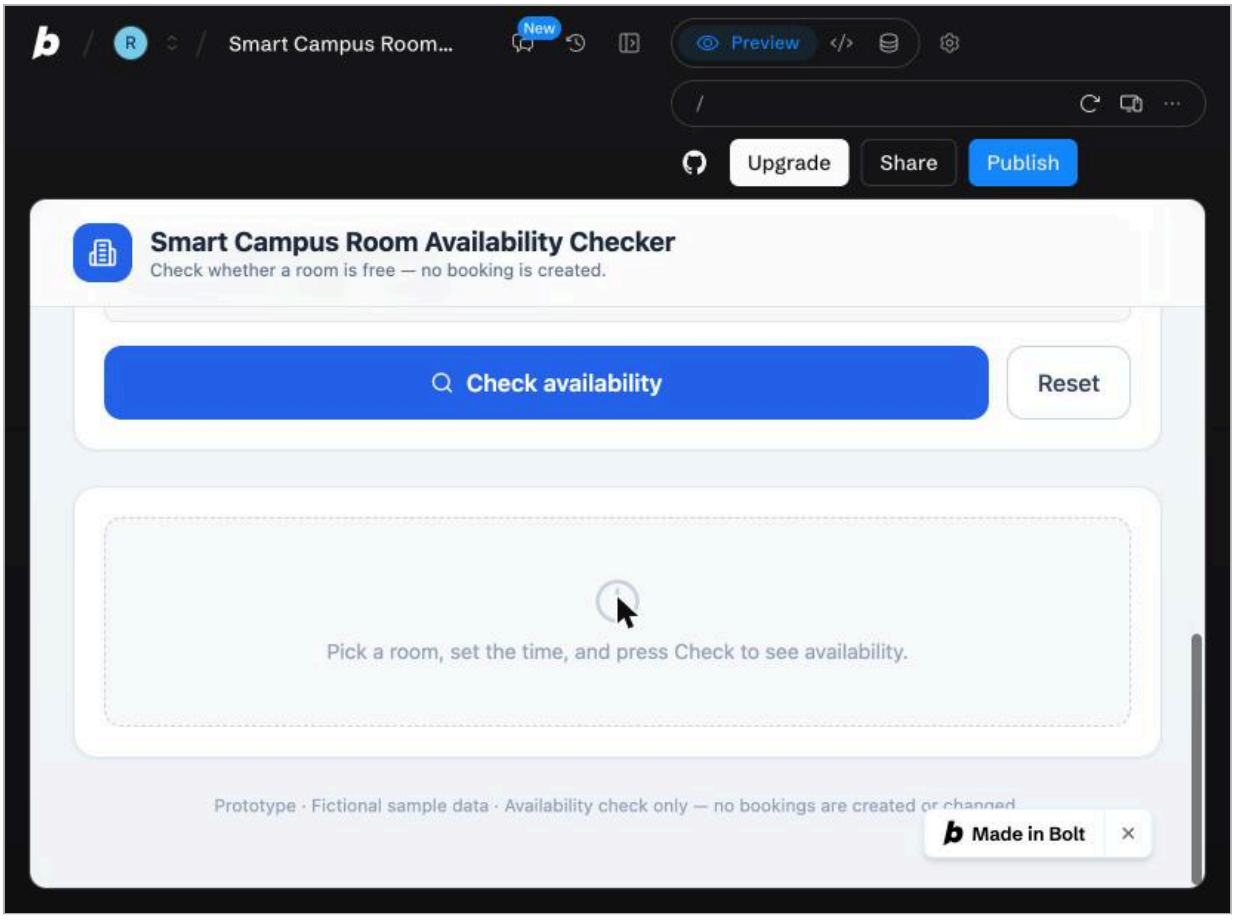
S-22 Final: 11:00-12:00 -> Available (touching) — 22_final_T01_touching_available.png



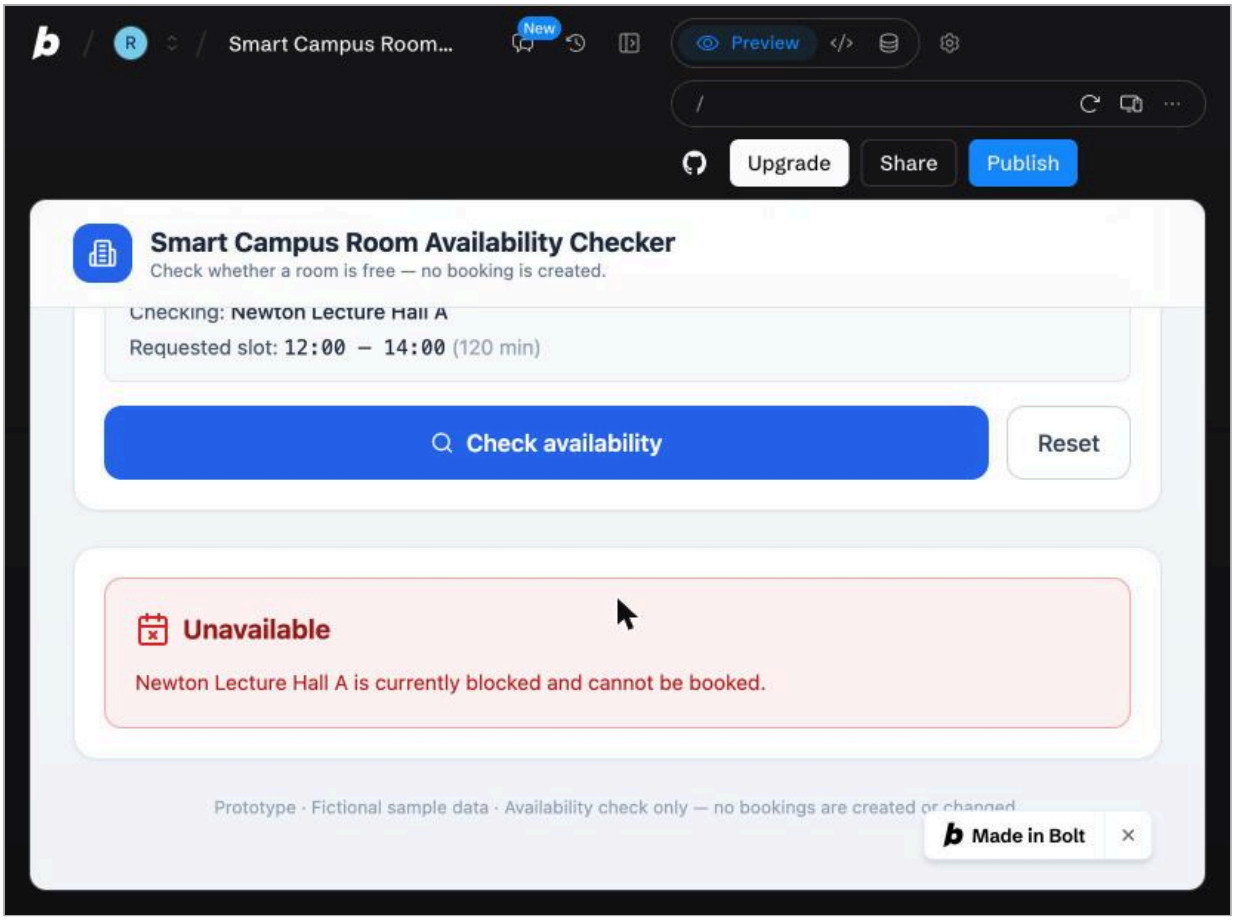
S-23 Final: 10:30-11:30 -> Unavailable (overlap) — 23_final_T02_overlap_unavailable.png



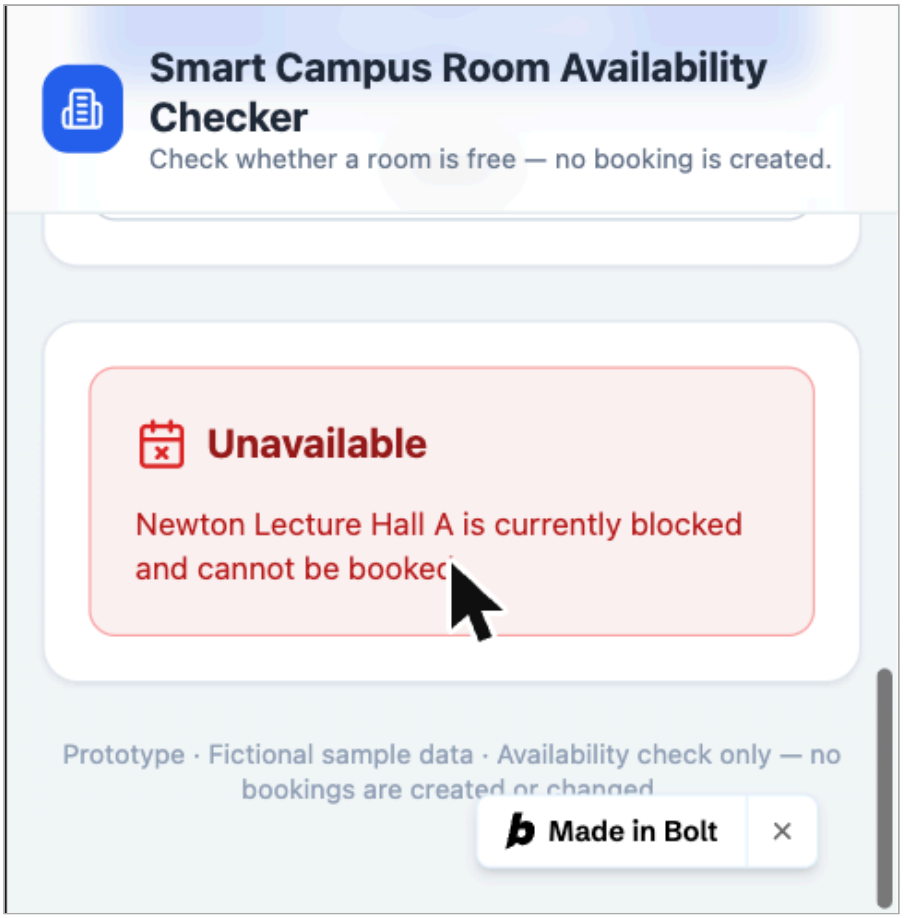
S-24 Final: 12:00-14:01 -> Unavailable (121 min) - V1-02 AFTER — 24_final_T05_121min_unavailable.jpg



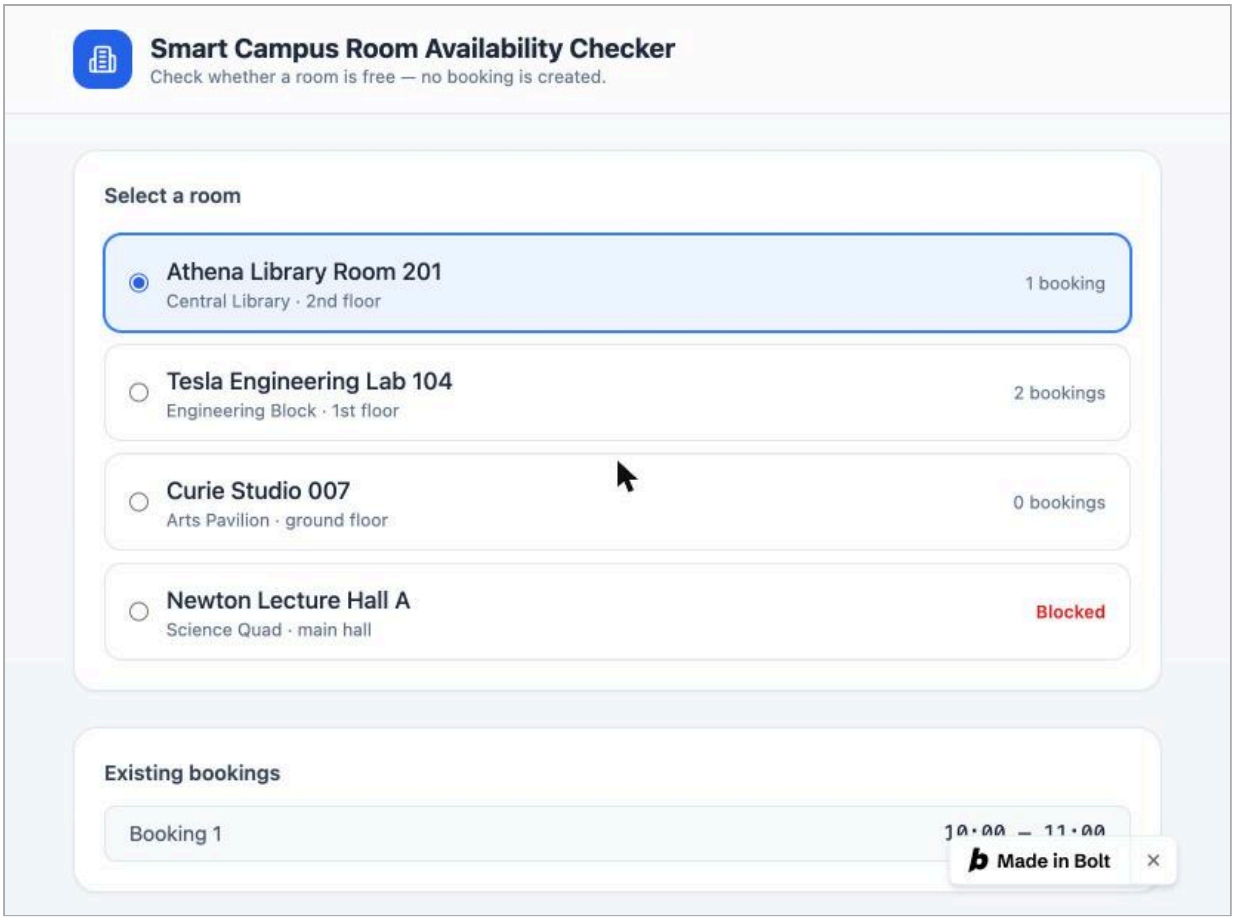
S-25 V1-01 AFTER: result cleared after switching room — 25_V1-01_AFTER_result_cleared_on_room_change.jpg



S-26 V1-03 AFTER: blocked reason shown first — 26_V1-03_AFTER_blocked_reason_first.jpg



S-27 Final: phone preview (iPhone 17, 402x874) — 27_final_mobile_iphone17.png



S-28 Published demo smart-campus-room-av-kyfa.bolt.host — 28_published_demo_bolt_host.jpg

E. References and source checks

E.1 References

1. Lesson #05 - Software Engineering - Lecture and Practice (dependability, AI-assisted coding, `can_book` tests).
2. Lesson #06 - Software Engineering - Lecture and Practice (coding agents, agent loop, Bolt.new, cleanup).
3. Assignment #02 format and teacher feedback (appendix requirement).
4. Python documentation: `unittest`; numeric types (`bool` is a subtype of `int`).
5. Vitest documentation: CLI `vitest run`.

E.2 Source checks - AI claims verified independently

Claim (source)	How I checked	Result
availability.ts is "the single source of truth for all six acceptance criteria" (E-B1)	Read the file (S-07, S-08); UI runs	Logic correct, but AC5/AC2 broken by the UI (V1-01, V1-02); AC3 reason order wrong (V1-03)
"validation messages guide the student ... before they even press Check" (E-B1)	UI run starts-now (S-09) and room switch (S-10)	**Harmful**: Check disabled, stale Available shown
"TimeSelect ... 15-minute increments" (E-B1)	Code line 14 (S-06)	True - and that is the V1-02 defect
"Build passes" (E-B1 to E-B4)	My own `npm run build` on v4	True (S-18)
"result now always belongs to the current inputs" (E-B2)	Check, then switch room	True (S-25)

"every integer minute from 00:00 to 24:00 selectable" (E-B3)	Code (S-17) and 12:00-14:01 in UI (S-24)	True; side effect: now = 24:00 also selectable
"No condition, reason text, or decision changed" (E-B4)	Code at lines 47-53 and after AC2 (S-16); Vitest A01-A15	True - 15/15 pass
Python v1 meets AC1 (E-P1)	T20 + probe	**False** for non-integer input
`bool` passes `isinstance(x, int)`	Python docs + probe `can_book(True, 60, 0, False, [])` → True in v1	True, hence the explicit bool exclusion
Underlying Bolt model	Mode selector (P-12)	Only "Bolt Agent - Standard" is shown; model not claimed

F. History - earlier Practice 6 Bolt project (`sb1-svxv4k4n`)

Before A3 I built the same checker in Practice 6 (Versions 1-6, 06 and 13 Oct 2026) and on 24 Oct (14:06) did one more cleanup there (Version 7). For A3 I then started the clean project above so that v1 is a real first generation. These exchanges are kept because they show the same workflow (plan → generate → critique → repair → verify) and my test-review work (mutation trials). Screenshots P-xx are in `screenshots/practice6_project/`.

P6-1 - Bolt.new - Generate v1 (06 Oct 2026, Version 1)

Prompt (exact)

Build a small responsive Smart Campus Room Availability Checker.
User story: As a student, I want to check a room's availability so that I can choose a valid booking time.
Requirements:

AC1: Accept a candidate interval only when 0 ≤ start < end ≤ 1440 and start is after the supplied current time.
AC2: Reject a booking longer than 120 minutes.
AC3: Reject the request when the selected room is blocked.
AC4: Reject any interval that overlaps an existing booking; touching endpoints are allowed.
AC5: Show a clear Available / Unavailable decision without changing the existing booking data.
AC6: The page remains usable at desktop and narrow/mobile width and gives understandable validation feedback. Use fictional sample data only. Keep the implementation simple and within one-page scope. Do not add authentication, backend services, analytics, or unrelated features. After building, briefly list the main files/components you created.

Response (exact)

I'll build a Smart Campus Room Availability Checker with in-memory fictional sample data, clean validation logic, and a responsive design. Let me start by creating the core files.

Plan
Create availability logic module (AC1-AC5)
Create fictional sample room data
Build main UI component with responsive design
Wire up App.tsx and verify build
Plan completed

Build passed successfully. Here's what I created:

Main files:
src/lib/availability.ts — The core logic that enforces all acceptance criteria: validates the interval (0 ≤ start < end ≤ 1440), checks that start is after the current time, rejects bookings over 120 minutes, rejects blocked rooms, and checks for overlapping bookings (touching endpoints allowed). Returns a clear Available/Unavailable decision without modifying any booking data.
src/lib/sampleData.ts — Four fictional campus rooms with sample bookings and one blocked room.

src/App.tsx — The full responsive UI: room selection cards, a simulated current-time input, start/end time dropdowns, a duration indicator, a check button, and a clear Available/Unavailable result banner. Layout adapts from a single column on mobile to a two-column layout on desktop.

How it works: You pick a room, set a simulated "current time," choose a start and end time, and hit "Check Availability." The app tells you whether the slot is available and, if not, explains exactly why — all using fictional data with no backend or real bookings involved.

My review decision

Plan approved (it matched my scope). Output **not accepted as final**: "enforces all acceptance criteria" is a claim, so I tested it in the preview and found B-01 and B-02. Gap in my prompt: the word "database" was not forbidden explicitly; I checked there is no DB code.

P6-2 - Bolt.new - Repair B-01 "24:00" (06 Oct 2026, Version 2)

Prompt (exact)

In the current prototype, selecting Room = Business Seminar Room, Current Time = 09:00, Start = 23:00, End = 24:00 and clicking Check Availability produces "Unavailable - Please enter valid start and end times.", but AC1 requires accepting intervals where 0 <= start < end <= 1440, so 23:00-24:00 (1380-1440) must be Available. Root cause appears to be labelToMinutes() rejecting "24:00" (h > 23). Make the smallest change that fixes this behavior (accept exactly "24:00" as 1440, still reject anything above it). Preserve the other acceptance criteria AC1-AC6. Explain which file/component you changed and why.

Response (exact)

Good catch. The fix is in labelToMinutes — I need to allow 24:00 as a special case mapping to 1440.
[tool steps collapsed in UI: actions taken]
Build passes. I changed one line in src/lib/availability.ts — the labelToMinutes function. It now checks for the exact string "24:00" and returns 1440 before the normal h > 23 rejection kicks in. Everything else stays the same: "24:01" and higher are still rejected, and all other criteria (interval bounds, current-time check, 120-minute max, blocked rooms, overlap with touching endpoints allowed) remain intact.

Now selecting Business Seminar Room with current time 09:00 and a 23:00–24:00 slot correctly shows Available.

My review decision / revision

Accepted after checking the code line (P-24, line 35: `if (h === 24 && m === 0) return DAY_MINUTES;`) and retesting 23:00-24:00 in the UI. Later locked by Vitest T16 and T20.

P6-3 - Bolt.new - Repair B-02 minute precision (06 Oct 2026, Version 3)

Prompt (exact)

In the current prototype, the Start Time and End Time dropdowns only offer 15-minute steps (e.g. 14:00, 14:15). So the required test "request 720-841" (12:00-14:01, which AC2 must reject as 121 minutes) cannot be entered at all; the closest possible value is 14:15. AC1/AC2 are defined in minutes, so the user must be able to enter any minute from 00:00 to 24:00. Make the smallest change that fixes this: let the user pick start and end times with 1-minute precision (end time must still allow exactly 24:00), keep the existing validation logic in availability.ts unchanged. Preserve AC1-AC6 and the mobile layout. Explain which file/component you changed and why.

Response (exact)

I'll replace the 15-minute-step <select> dropdowns with <input type="time"> fields that allow 1-minute precision, plus a special "24:00" option for the end time since native time inputs cap at 23:59.

[tool steps collapsed in UI: actions taken]
Build passes. I changed only src/App.tsx:

Replaced the old 15-minute-step TIME_SLOTS array with two new arrays: END_TIME_SLOTS (every minute from 00:00 to 24:00, 1441 entries) and START_TIME_SLOTS (same minus "24:00", since start must be strictly less than end per AC1). Updated the Start Time dropdown to use START_TIME_SLOTS and the End Time dropdown to use END_TIME_SLOTS.

The validation logic in availability.ts is untouched. You can now select 12:00 as start and 14:01 as end, which correctly triggers the "exceeds 120 min max" rejection from AC2. The end dropdown still includes exactly "24:00" as the final option, and the mobile layout is preserved.

My review decision / revision

Accepted, but I noted a contradiction: the first sentence says `<input type="time">`, the summary says dropdowns with 1441 entries. The UI really uses `<select>` (P-07, P-08). I trust the code and UI, not the first sentence. 12:00-14:01 now gives Unavailable (P-08).

P6-4 - Bolt.new - Critique feedback messages B-03 (06 Oct 2026, Version 4)

Prompt (exact)

Review only the validation feedback messages returned by checkAvailability() in src/lib/availability.ts for unnecessary duplication, confusing naming, or UI feedback problems. Observed: with Start = 23:00 and End = 14:01 the result says "Interval must satisfy 0 <= start < end <= 24:00." - this is a math formula, not understandable feedback for a student (AC6). Suggest one small cleanup. Do not add features or redesign the whole application. Implement it only if it preserves AC1-AC6 (the Available/Unavailable decisions must stay exactly the same, only the wording of messages may change).

Response (exact)

Let me review the current messages.
Read src/lib/availability.ts
The one cleanup I'd suggest: the AC1 message is a math formula (0 ≤ start < end ≤ 24:00) that doesn't tell the student what is actually wrong. With Start=23:00, End=14:01, the real problem is "end is before start" — but the student sees an algebra inequality. I'll split it into plain-language messages for each sub-condition. The boolean decisions stay identical; only the wording changes.
[tool steps collapsed in UI: actions taken]
Build passes. I changed only src/lib/availability.ts — the AC1 validation branch. Instead of one math-formula message, it now gives plain-language feedback for each specific problem:

End after 24:00 → "End time cannot be later than 24:00."
End not after start (e.g. Start=23:00, End=14:01) → "End time must be later than the start time."
Start before 00:00 → "Start time cannot be earlier than 00:00."

The Available/Unavailable decisions are unchanged — every interval that was rejected before is still rejected for the same AC1 reason, and every accepted interval is still accepted. Only the wording improved for readability (AC6).

My review decision / revision

Accepted. Decisions unchanged was verified later by the Vitest suite (all AC1 cases still reject, with the new reason texts asserted in v6).

P6-5 - Bolt.new - Generate tests (13 Oct 2026, Version 5)

Prompt (exact; first attempt was cut off, so I resent the full prompt)

(Full prompt, previous one was cut.) Generate tests for the existing booking function using Vitest (add vitest as a devDependency and a "test": "vitest run" script if needed). Use AC1-AC6 and the reviewed case table below. FUNCTION UNDER TEST: checkAvailability(room, start, end, currentTime) from src/lib/availability.ts. It returns { available: boolean, reason?: string }. Treat available === true as Available and false as Unavailable. All times are minutes after midnight. DEFAULT FIXTURE (fresh object per test): now = 540, room = { id: 'r1', name: 'Test Room', building: 'B', capacity: 10, blocked: false, bookings: [{ start: 600, end: 660 }] }. CASES (start-end -> expected available): T01 660-720 -> true (AC4 touching after); T02 540-570 -> false (AC1 starts now); T03 720-720 -> false (AC1 zero duration); T04 750-720 -> false (AC1 reversed); T05 1380-1441 -> false (AC1 past day end); T06 720-840 -> true (AC2 exactly 120); T07 720-841 -> false (AC2 121 min); T08 570-600 -> true (AC4 touching before); T09 630-690 -> false (AC4 partial overlap); T10 570-690 -> false (AC4 contains booking); T11 615-645 -> false (AC4 inside booking); T12 600-660 -> false (AC4 identical); T13 660-720 with blocked = true -> false (AC3); T14 630-690 with bookings = [] -> true (AC4); T15 AC5: after calling checkAvailability for one available and one unavailable case, room.bookings is deep-equal to the original and the room object is unchanged; T16 1380-1440 -> true (AC1 end exactly 24:00 allowed); T17 541-571 -> true (AC1 one minute after now allowed). Import the real function used by the page. Keep its code unchanged. Use fixed time and fresh fixtures. Assert exact decisions and unchanged booking data. Put the file at src/lib/availability.test.ts and start every test name with its ID (e.g. 'T01 ...'). Return the test file, run command, and AC mapping. Flag unclear requirements. We will run the tests ourselves.

Response (exact, AC mapping table shortened to its header)

I'll add Vitest, create the test file, and verify it runs. Let me start by reading the current availability module and installing vitest in parallel.
[tool steps collapsed in UI: actions taken]
Vitest 5 needs Vite 6+, but this project uses Vite 5. I'll install Vitest 2.x which is compatible.
[tool steps collapsed in UI: commands executed]
Good, Vitest installed. Now let me add the test script and create the test file.
[tool steps collapsed in UI: actions taken]
All 17 tests pass. Here's the summary:
Files changed:
package.json — Added vitest@^2.1.9 as a devDependency and a "test": "vitest run" script.
src/lib/availability.test.ts — New test file with 17 cases.
Run command: npm test (or npx vitest run)
AC mapping: Test | AC | What it checks (T01-T17 listed, matching my case table)
AC6 (responsive layout and understandable feedback) is a UI-level requirement and isn't unit-testable through checkAvailability alone — it's covered by the message-wording review done in the previous step.
No unclear requirements — the case table was precise and all expected decisions matched the function's actual behavior.
Ran production build to verify project compiles
Build passes with no issues. The project compiles cleanly — the test file is excluded from the Vite build (it only processes src/ entry points referenced by the app), so it doesn't affect the production bundle.

My review decision / revision

I ran the tests myself (17 passed) and then ran mutation trials on a copy of `availability.ts`. M4-M6 survived, so "all tests pass" did not mean the tests were good. **Rejected** the claim that AC6 is "covered by the wording review" - I check AC6 manually in the UI and phone preview. Also noted that the agent chose and installed a package version itself.

P6-6 - Bolt.new - Critique and repair the tests B-04 (13 Oct 2026, Version 6)

Prompt (exact)

Review the test suite in src/lib/availability.test.ts against AC1-AC6 and our case table. Identify incorrect expectations, weak assertions, duplicated cases, and missing boundaries. For each proposed change, explain the requirement and the defect it could detect. Improve the tests without changing correct expectations to match buggy behavior. Use the actual output below. Separate test repairs from implementation fixes (do NOT edit availability.ts or App.tsx). ACTUAL OUTPUT: npx vitest run

-> 17 passed (17). Mutation trials we ran on a temporary copy of availability.ts: M1 (duration >= 120) -> T06 failed; M2 (touching counts as overlap) -> T01, T08 failed; M3 (blocked check removed) -> T13 failed. But these 3 mutants SURVIVED with 17 passed: M4 overlap check only looks at the first booking (room.bookings.slice(0, 1).find); M5 labelToMinutes no longer accepts "24:00" (the form uses labelToMinutes before checkAvailability, this was a real bug in Practice 6); M6 blocked-room reason text replaced by "Error." (AC6 understandable feedback). MY REVIEW NOTES: (1) rejection tests only assert available === false, so a request can be rejected by the wrong rule and still pass - also assert that reason matches the expected rule (e.g. /blocked/i, /overlaps/i, /120 minutes/, /after the current time/, /later than/); (2) only T15 checks that bookings are unchanged - add the unchanged-bookings check to every test via a small helper; (3) add T18: bookings [600-660, 780-840], request 750-800 -> false (overlap with the second booking); (4) add T19: request -1 to 30 -> false (AC1 start >= 0); (5) add T20 integration of form input: labelToMinutes('24:00') === 1440, labelToMinutes('14:01') === 841, labelToMinutes('24:01') === null. Keep T01-T17 IDs and expected decisions. Return the updated file, the list of changes, and which mutant each change can detect. We will run the tests ourselves.

Response (exact, tables flattened)

Let me read the current test file and the source to get exact reason strings for the assertions.
[tool steps collapsed in UI: files read]
Now I'll rewrite the test file with all the requested improvements: reason-matching assertions on rejections, an expectUnchanged helper applied to every test, T18 (second-booking overlap), T19 (negative start), and T20 (labelToMinutes integration).
[tool steps collapsed in UI: actions taken]
All 20 tests pass and the build succeeds. availability.ts and App.tsx were not touched.
List of changes:
1 Added reason-matching assertions to every rejection test (T02–T05, T07, T09–T13, T19) ... Mutant killed: M6
2 Added expectUnchanged helper and called it in every test (T01–T19), not just T15 ...
3 Added T18 — room with two bookings [600-660, 780-840], request 750-800 → unavailable ... M4
4 Added T19 — request -1 to 30 → unavailable, reason matches /earlier than 00:00/ ...
5 Added T20 — labelToMinutes('24:00') === 1440, labelToMinutes('14:01') === 841, labelToMinutes('24:01') === null ... M5
Mutant coverage summary: M1 T06 (existing); M2 T01, T08 (existing); M3 T13 (existing); M4 T18 (new); M5 T20 (new); M6 T13 reason assertion (new)

My review decision / revision

Accepted after my own run: 20 passed (re-run on 24 Oct, P-03). Expected decisions T01-T17 unchanged, as I asked.

P6-7 - Bolt.new - A3 cleanup I-01 (24 Oct 2026, Version 7)

Prompt (exact, P-13)

Assignment 3 cleanup, issue I-01 (AC3/AC6 feedback). Observed in the preview: Room = Engineering Lab (blocked), Current Time = 09:00, Start = 09:00, End = 09:30 -> result is "Unavailable - Start time must be after the current time (09:00)." The decision is correct, but the reason is misleading: the student fixes the time and only then learns the room is blocked. AC3 says every request for a blocked room is rejected, so the blocked rule should be reported first. Make the smallest change: in src/lib/availability.ts move the existing `if (room.blocked)` check to the top of checkAvailability(), before the AC1 checks. Do not change any other condition, message text, App.tsx, or the Available/Unavailable decision for any input. Add one test T21 to src/lib/availability.test.ts: blocked room, now=540, request 540-570 -> available false and reason matches /blocked/i (keep T01-T20 unchanged). Then run npx vitest run and tell me exactly which lines changed.

Response (exact, P-14)

[Thoughts collapsed] [2 files read]
I'll move the blocked-room check to the top and add test T21.
[tool steps collapsed in UI: actions taken]
All 21 tests pass. Let me verify the final state of the changed lines.
Read src/lib/availability.ts

All 21 tests pass. Here's exactly what changed.

src/lib/availability.ts
Lines 46–49 (added): The if (room.blocked) check now sits at the very top of checkAvailability(), before the AC1 interval validation, so a blocked room is always reported first.
The previous blocked check that lived between the AC2 and AC4 checks (old lines 73–75) was removed — no other condition, message, or decision changed.

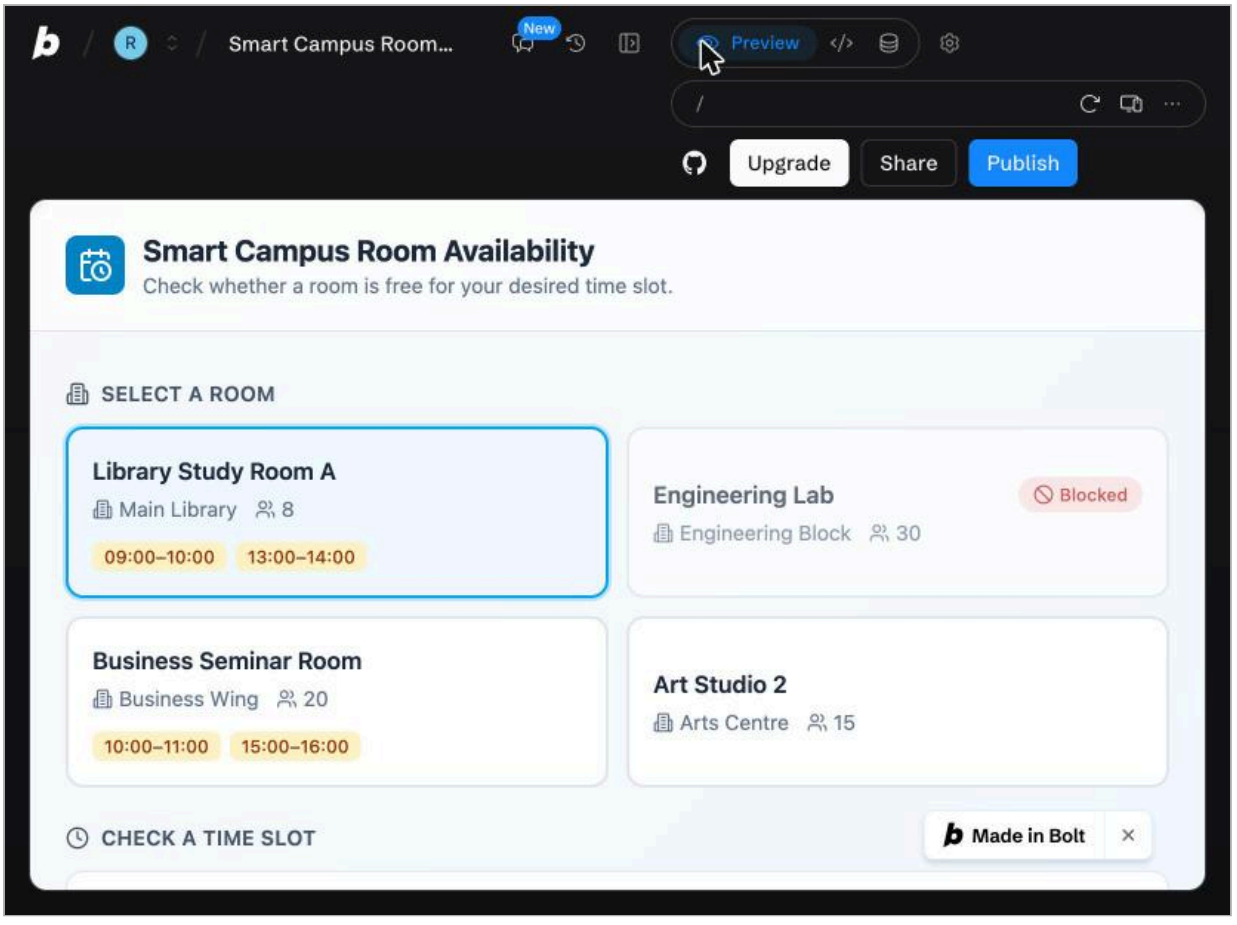
src/lib/availability.test.ts
Lines 222–230 (added): Test T21 — a blocked room requested at 540–570 (start equals current time, which would otherwise trigger the AC1 "start must be after current time" rule) now returns available: false with a reason matching /blocked/i. T01–T20 are untouched.

Now when a student picks the blocked Engineering Lab at 09:00–09:30, the result reads "This room is currently blocked and cannot be booked" instead of the misleading time error — so they learn the room is blocked first, matching AC3 and AC6.

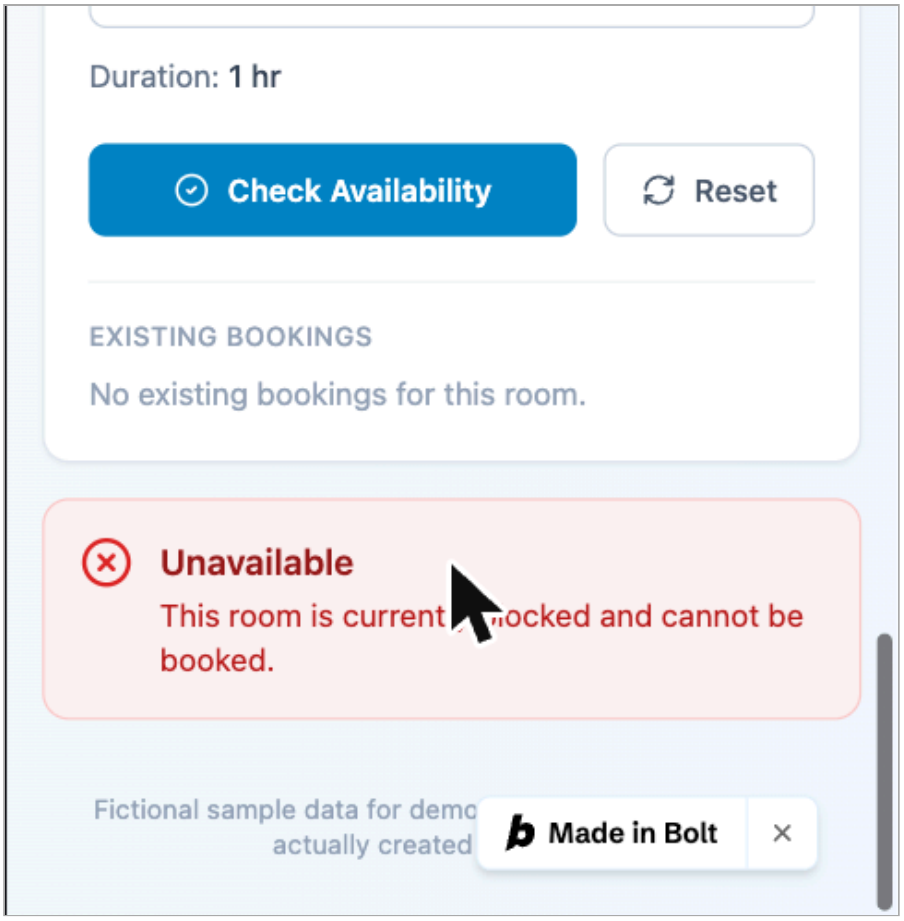
My review decision / revision

Accepted after checking: (1) code - blocked check at lines 46-49, old check after AC2 no longer present, overlap block and `return {available: true}` unchanged, file still 88 lines (P-15); (2) my own `npx vitest run` - 21 passed at 14:08:07 (P-16, P-17); (3) UI rerun of the reproduction (P-18) and the full Task 4 set (P-19 to P-23).

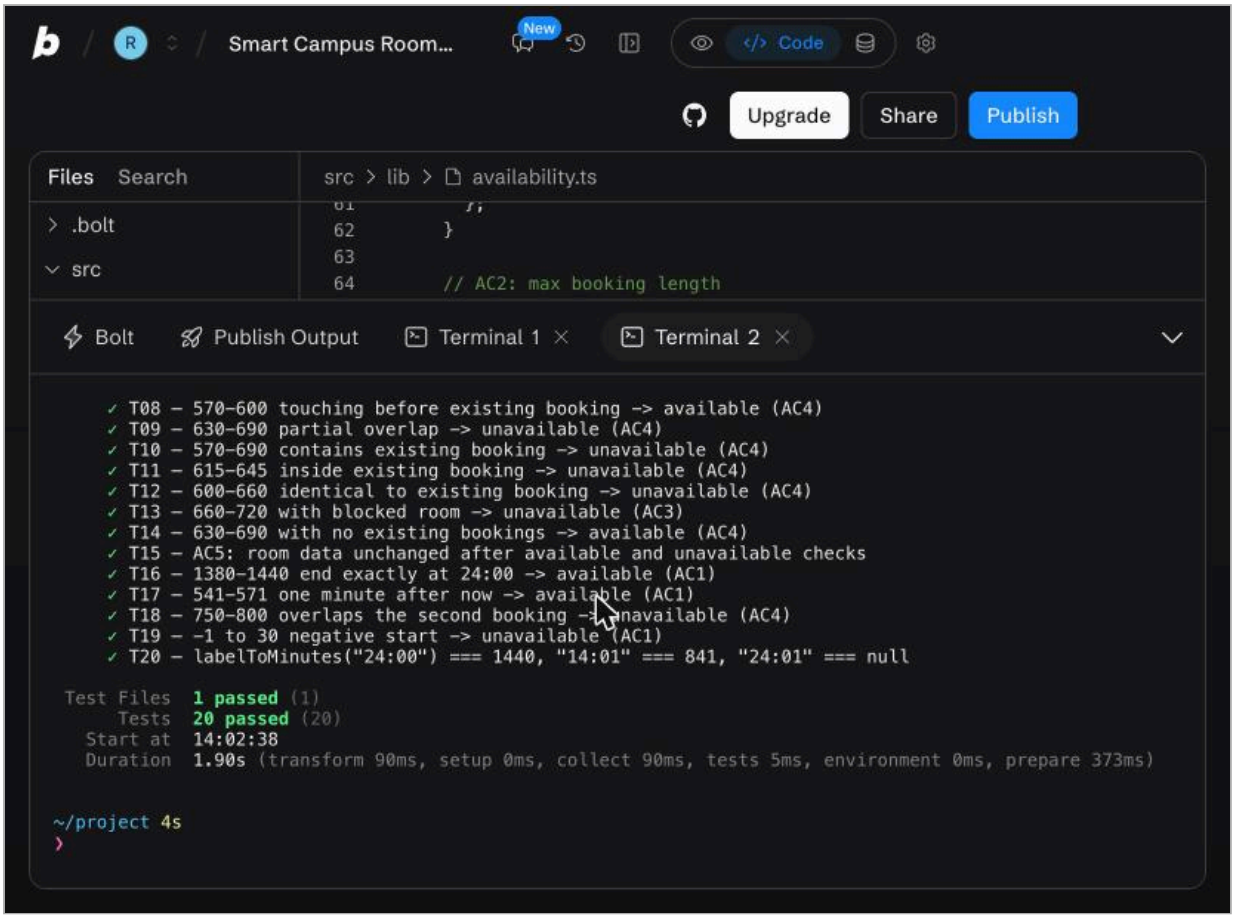
F.1 Practice 6 screenshots



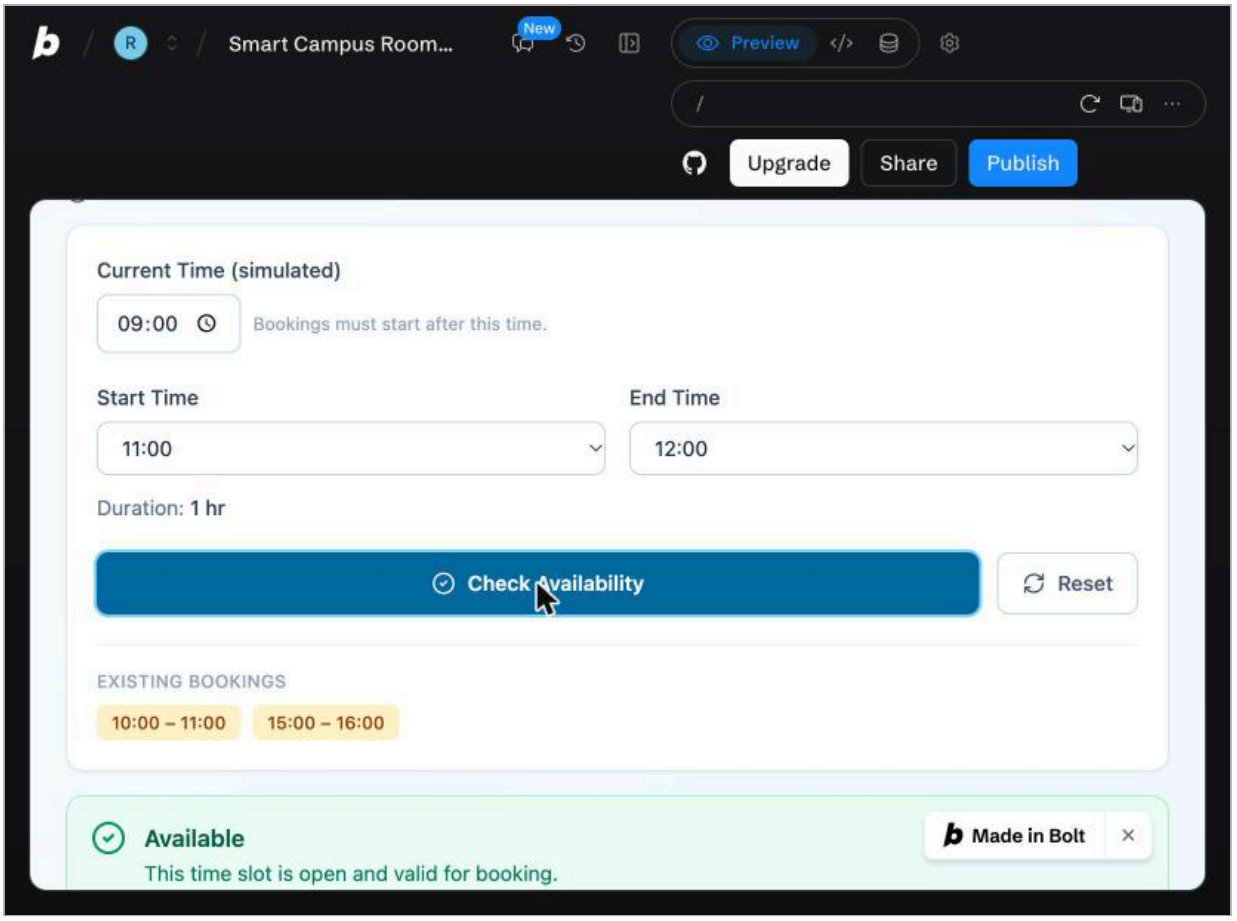
P-01 Prototype desktop — P01_prototype_desktop.jpg



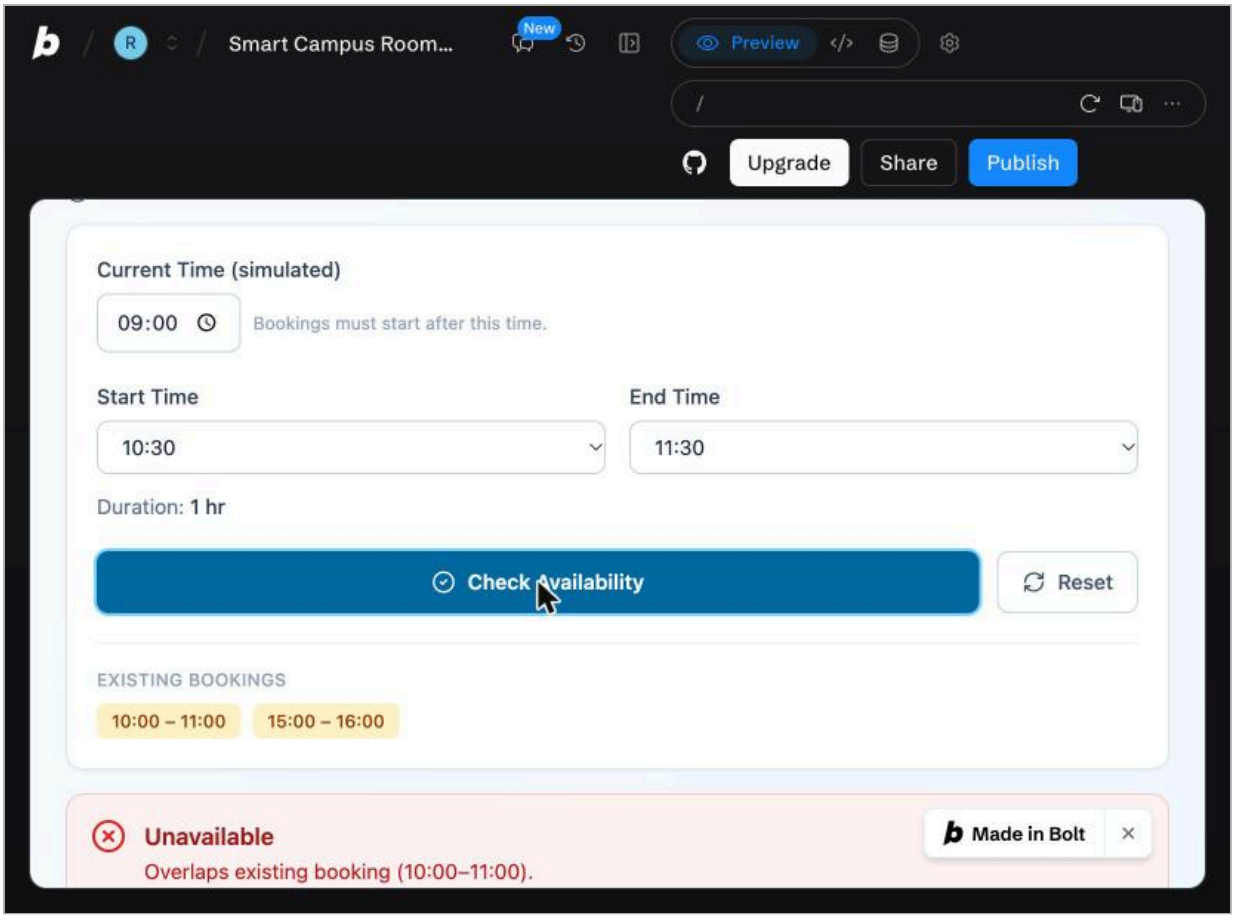
P-02 Phone preview — P02_prototype_mobile_iphone.png



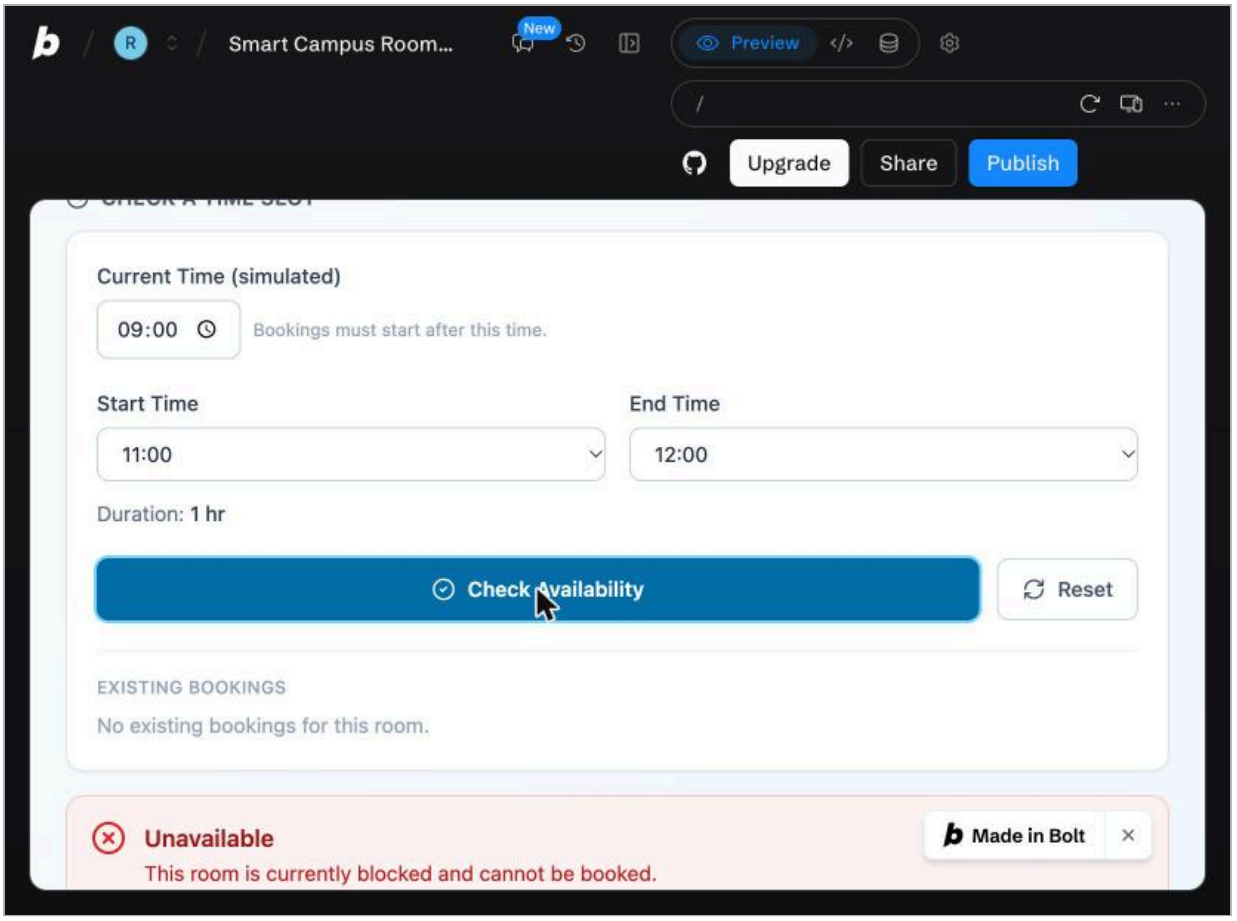
P-03 Vitest 20 passed (14:02) — P03_vitest_before_20_passed.jpg



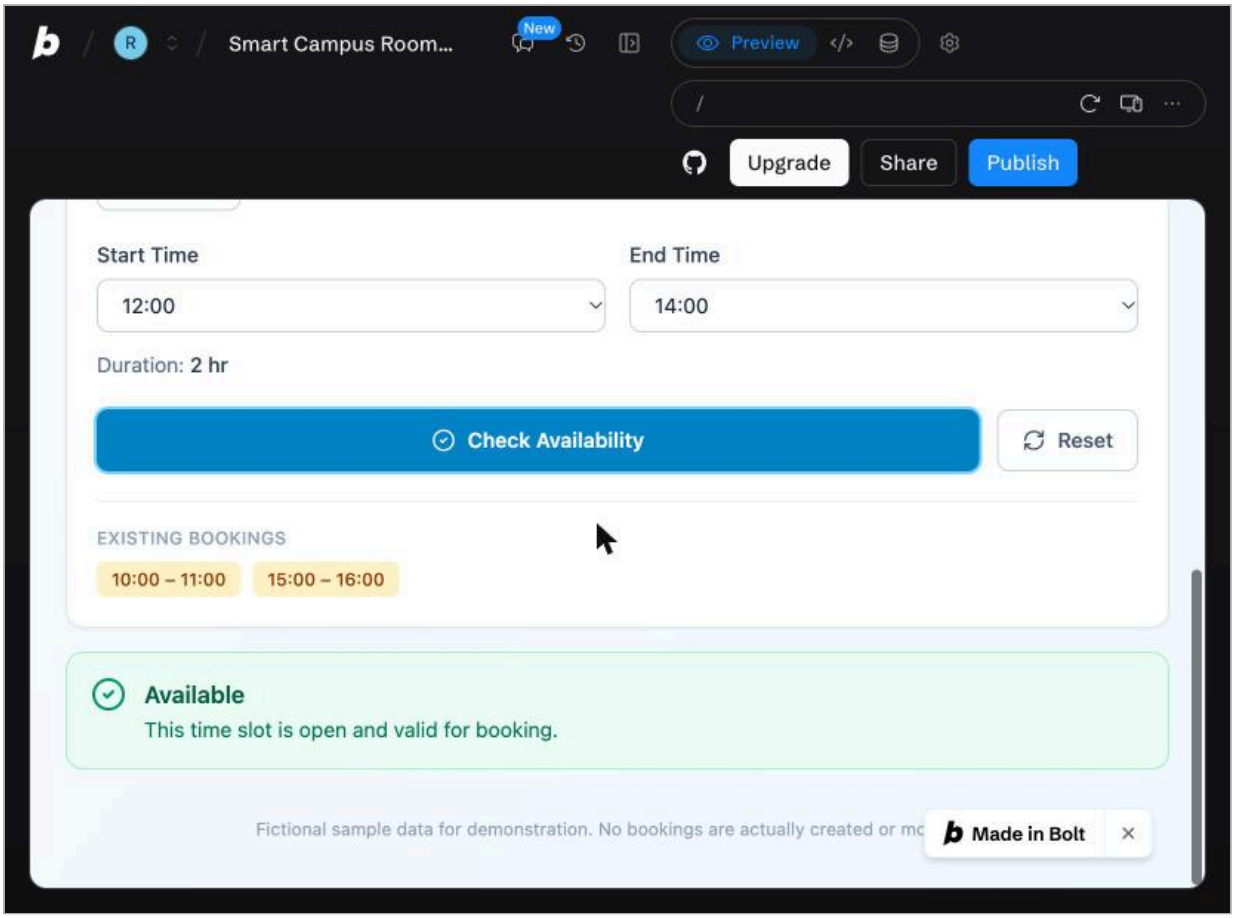
P-04 T01 touching — P04_v6_T01_touching_available.jpg



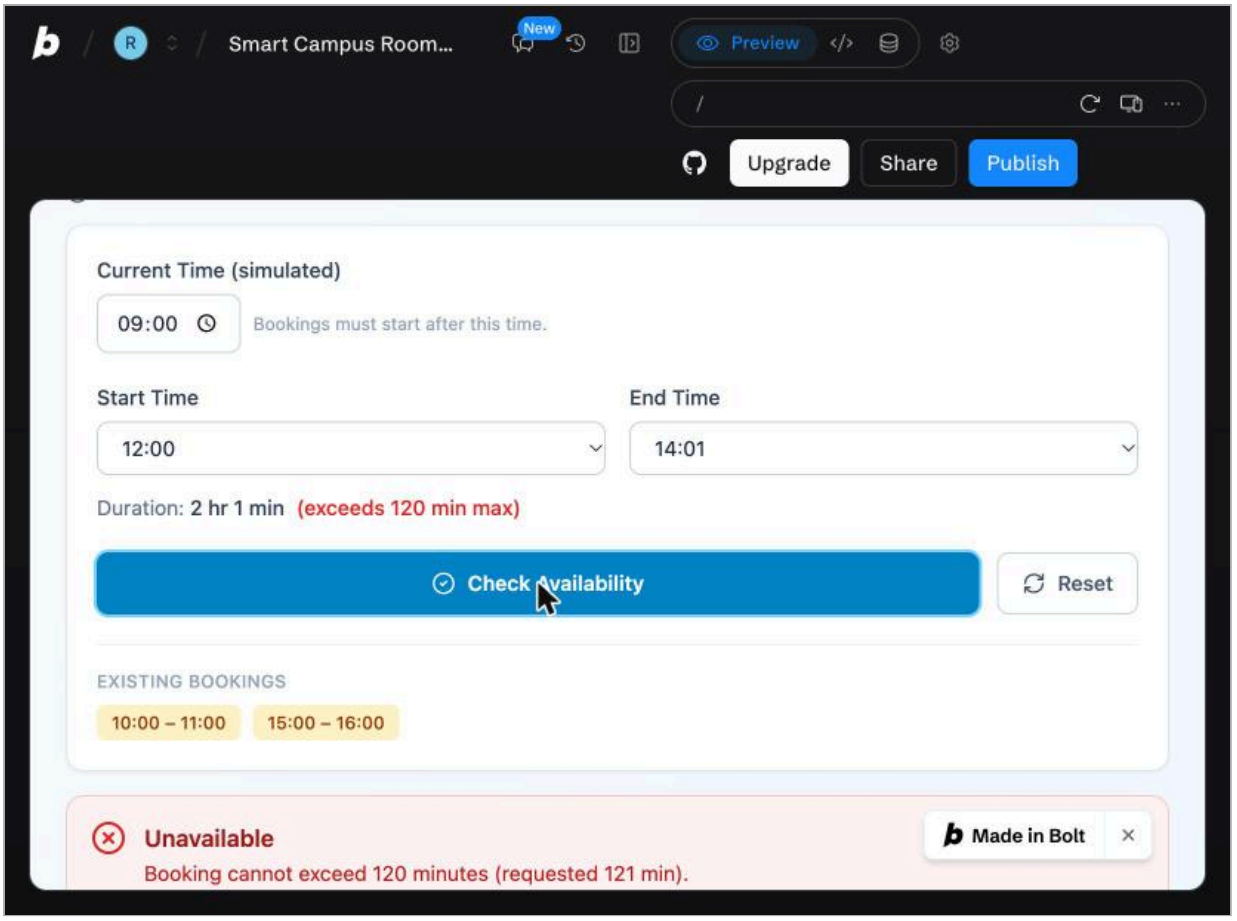
P-05 T02 overlap — P05_v6_T02_overlap_unavailable.jpg



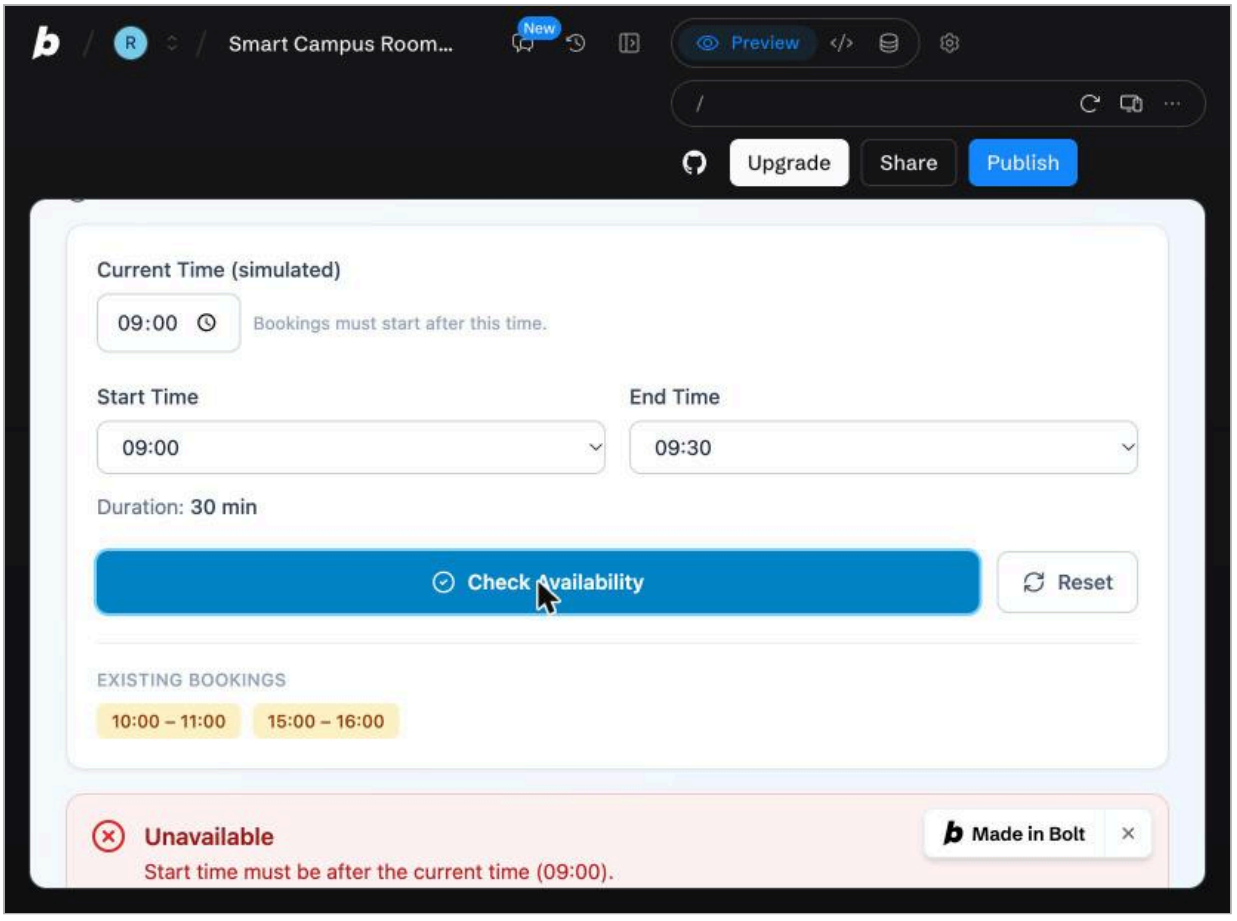
P-06 T03 blocked — P06_v6_T03_blocked_unavailable.jpg



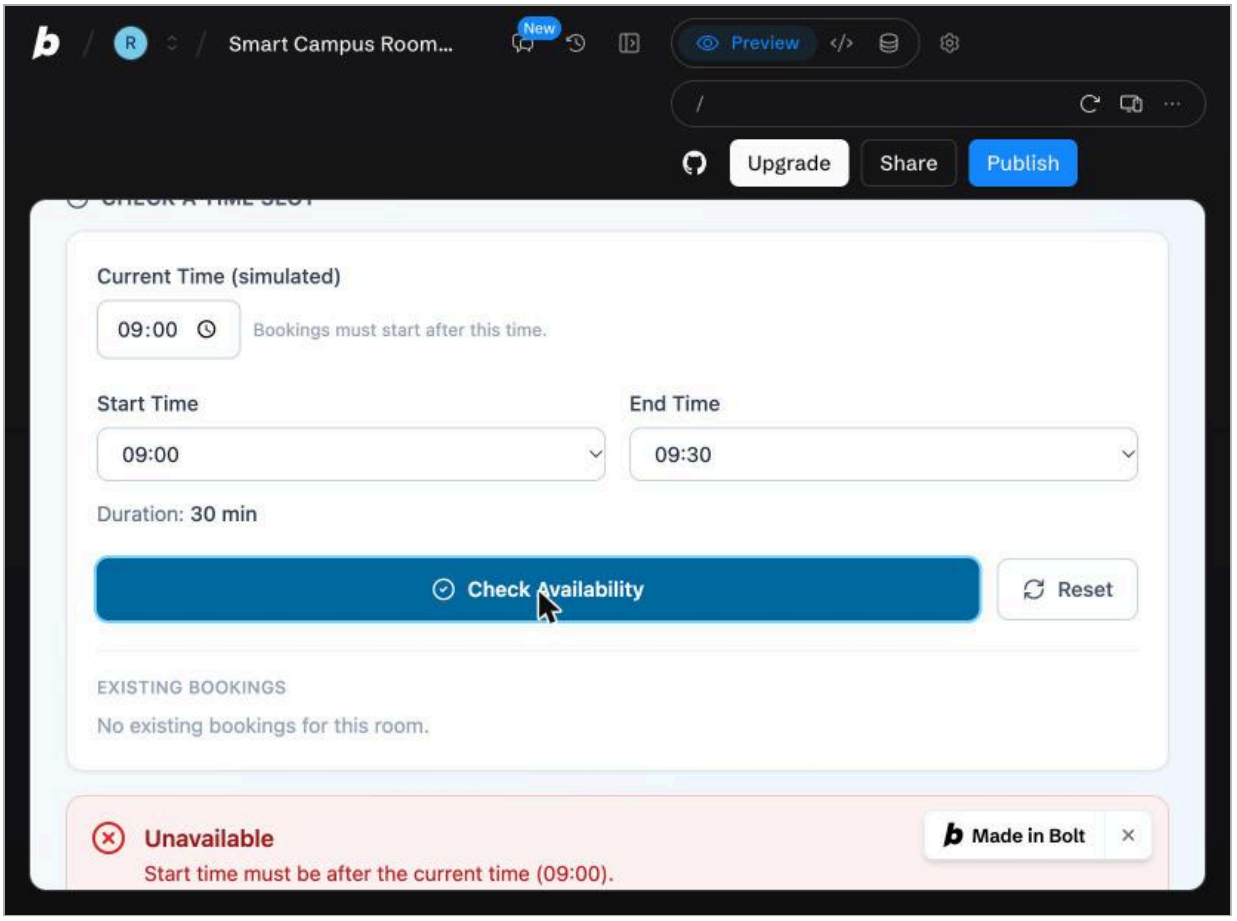
P-07 T04 120 min — P07_v6_T04_120min_available.jpg



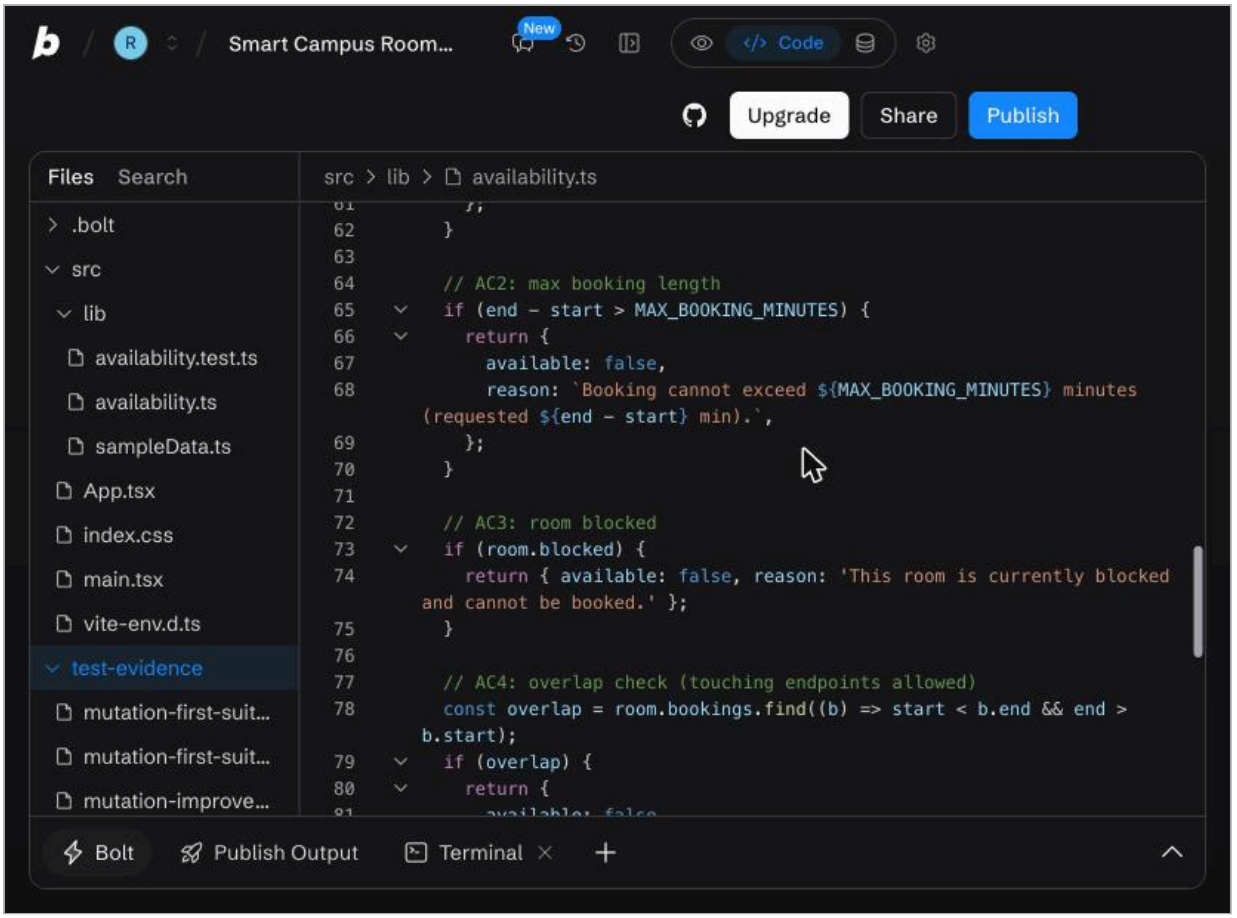
P-08 T05 121 min — P08_v6_T05_121min_unavailable.jpg



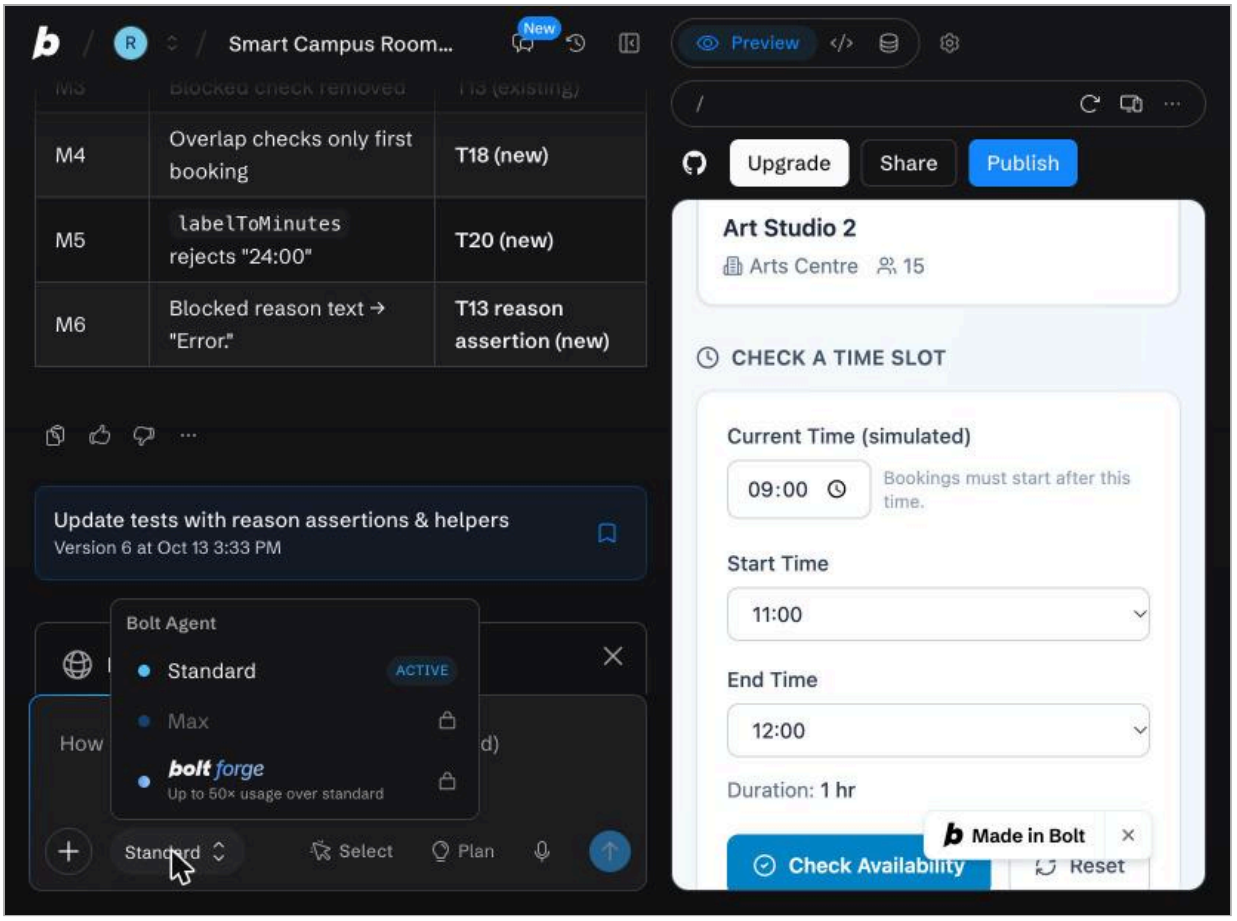
P-09 T06 starts now — P09_v6_T06_starts_now_unavailable.jpg



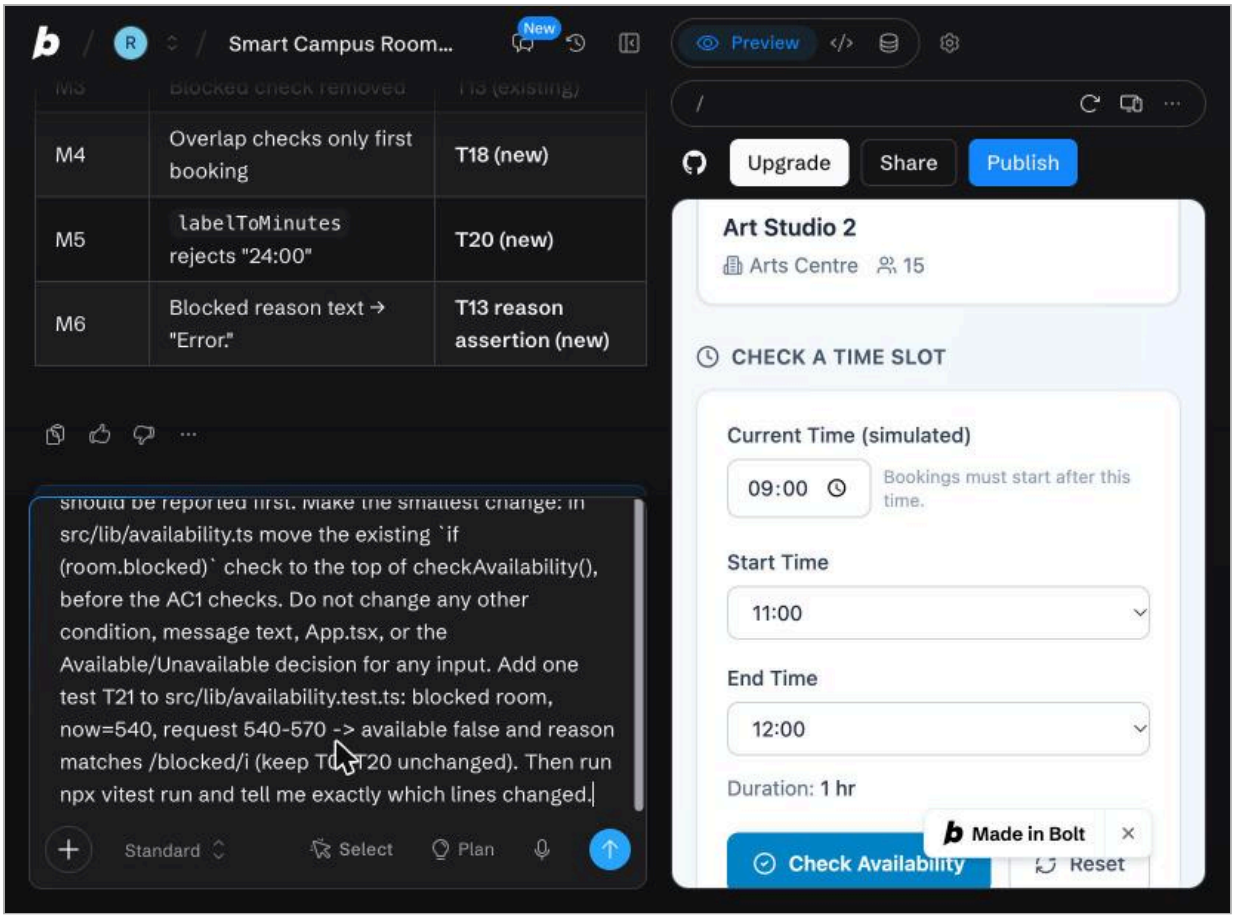
P-10 I-01 before — P10_I01_BEFORE_blocked_room_shows_time_error.jpg



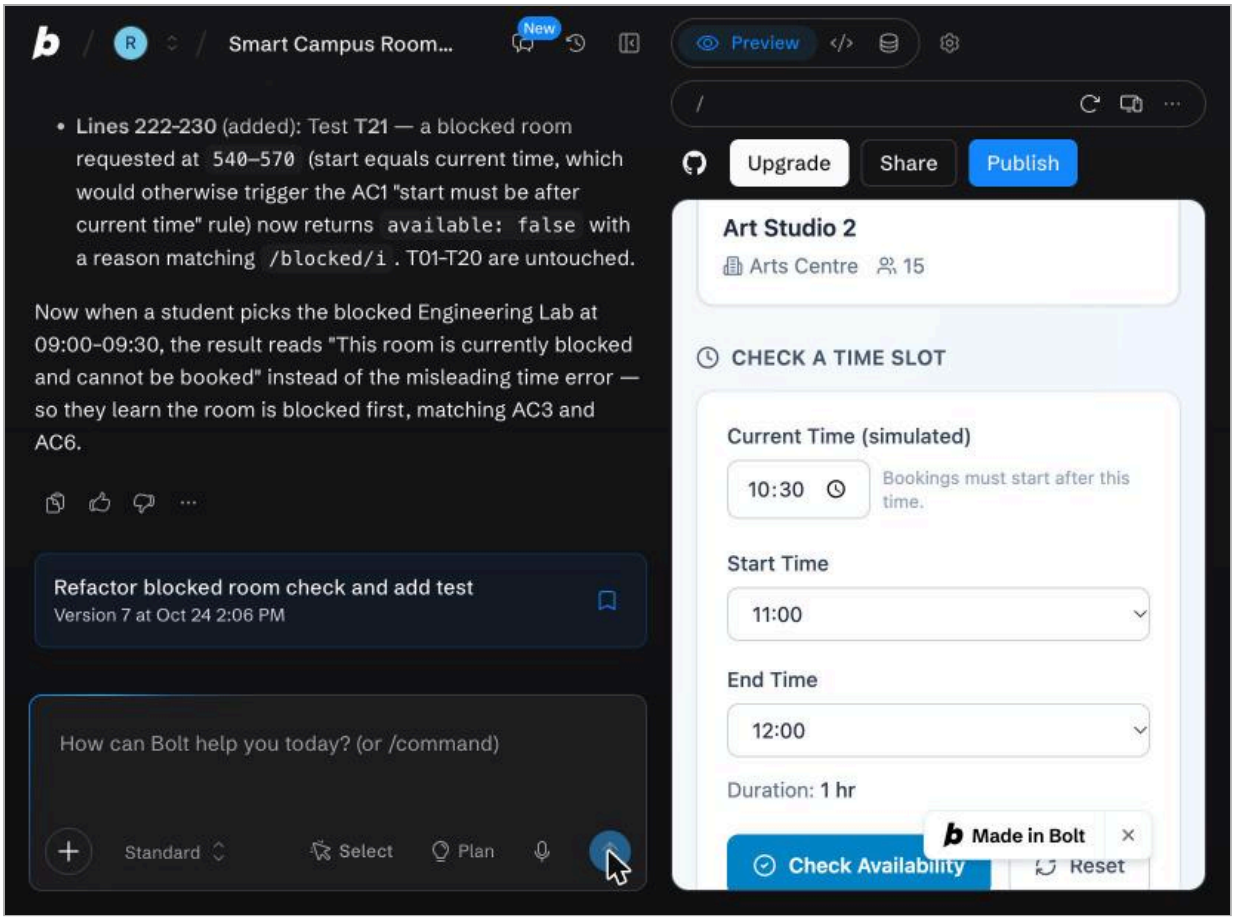
P-11 I-01 code before — P11_I01_BEFORE_code_blocked_check_after_AC2.jpg



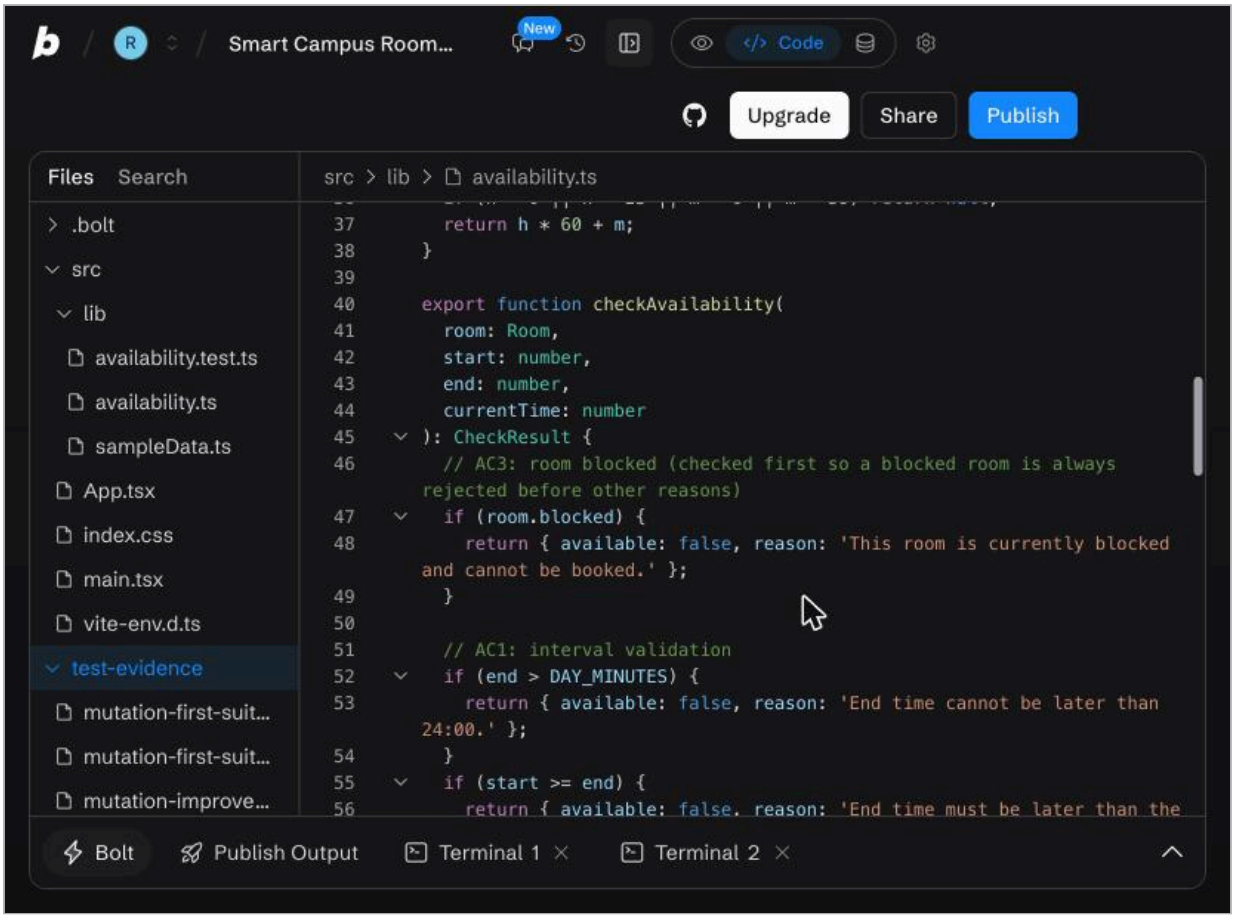
P-12 Bolt Agent mode selector "Standard" — P12_bolt_agent_mode_standard.jpg



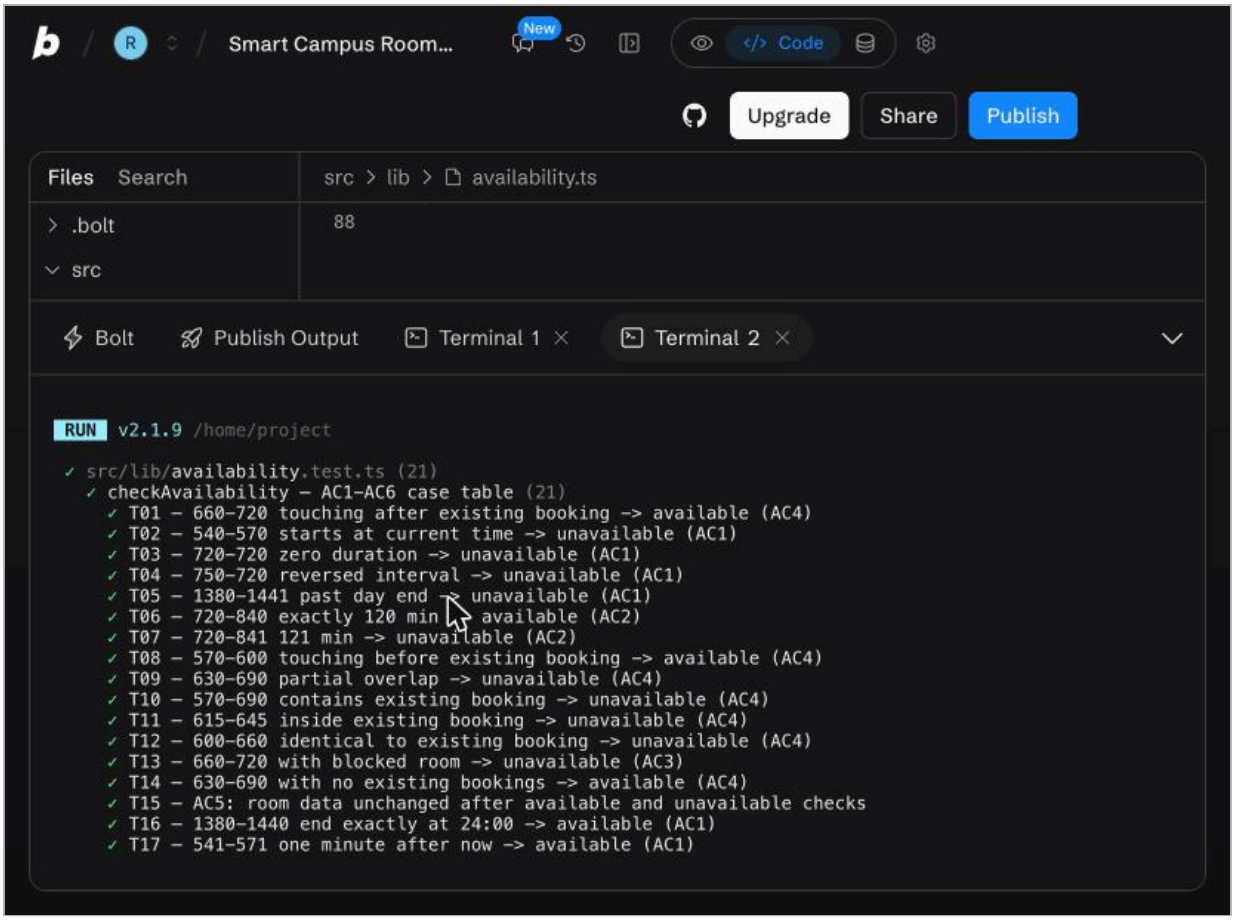
P-13 I-01 prompt — P13_I01_prompt_typed.jpg



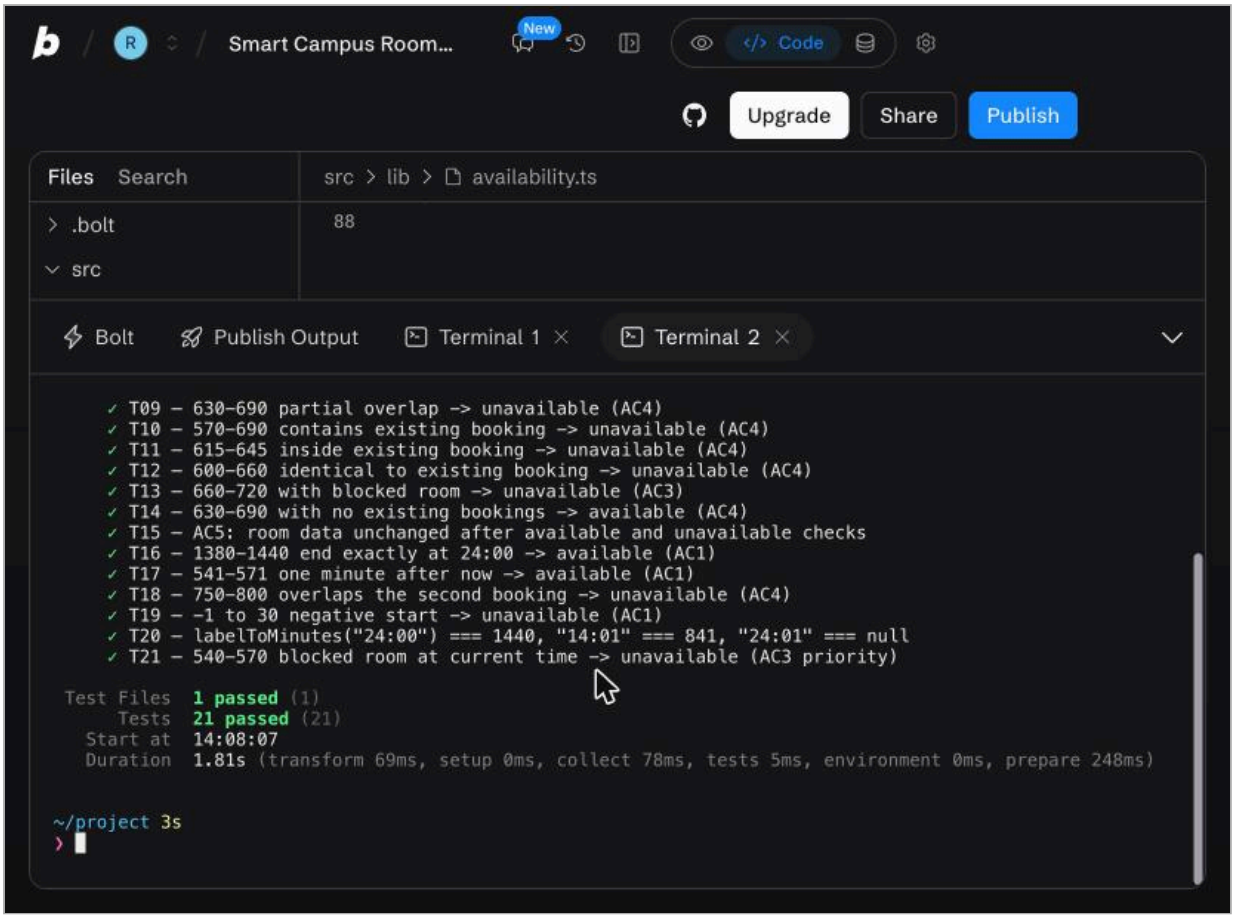
P-14 Bolt response v7 — P14_I01_bolt_response_v7.jpg



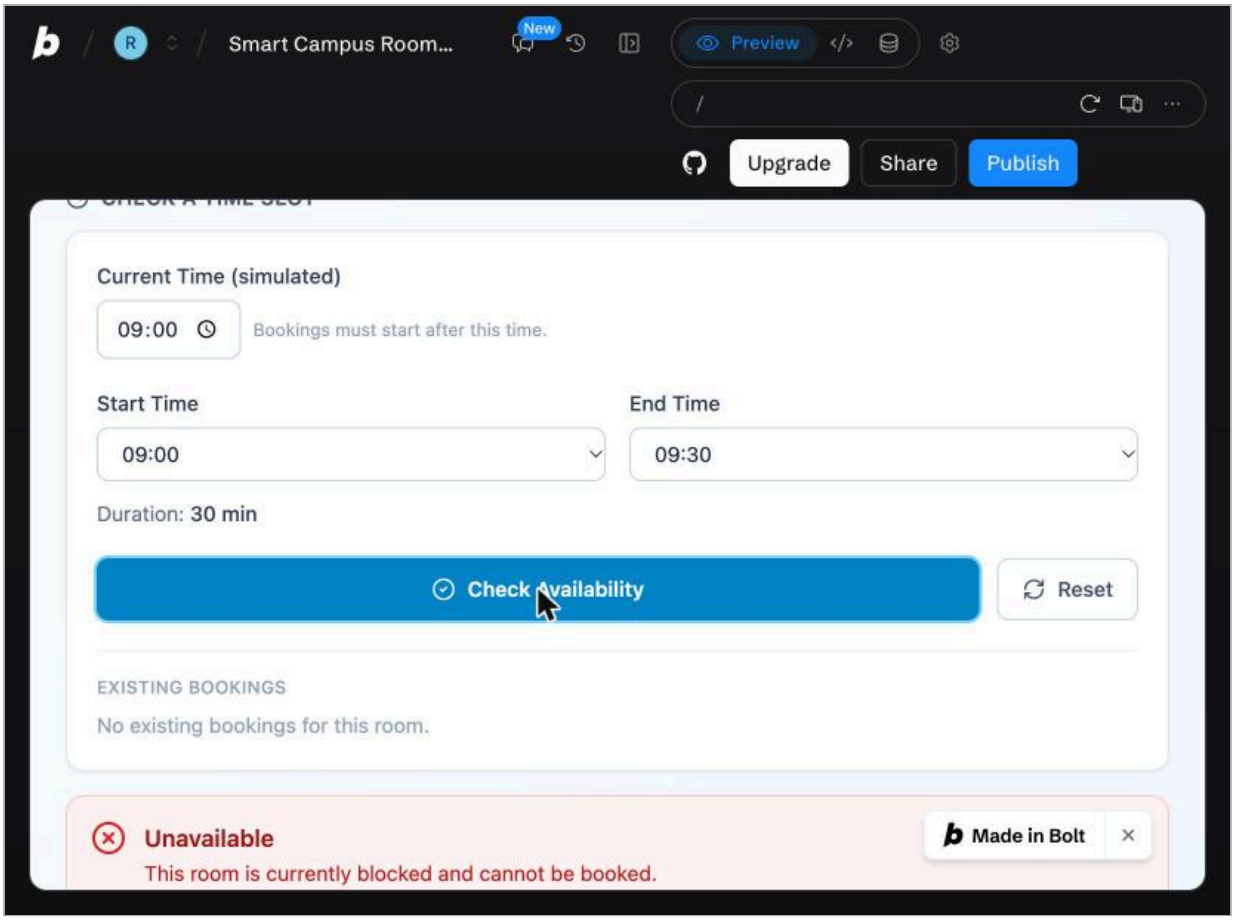
P-15 I-01 code after — P15_I01_AFTER_code_blocked_check_first.jpg



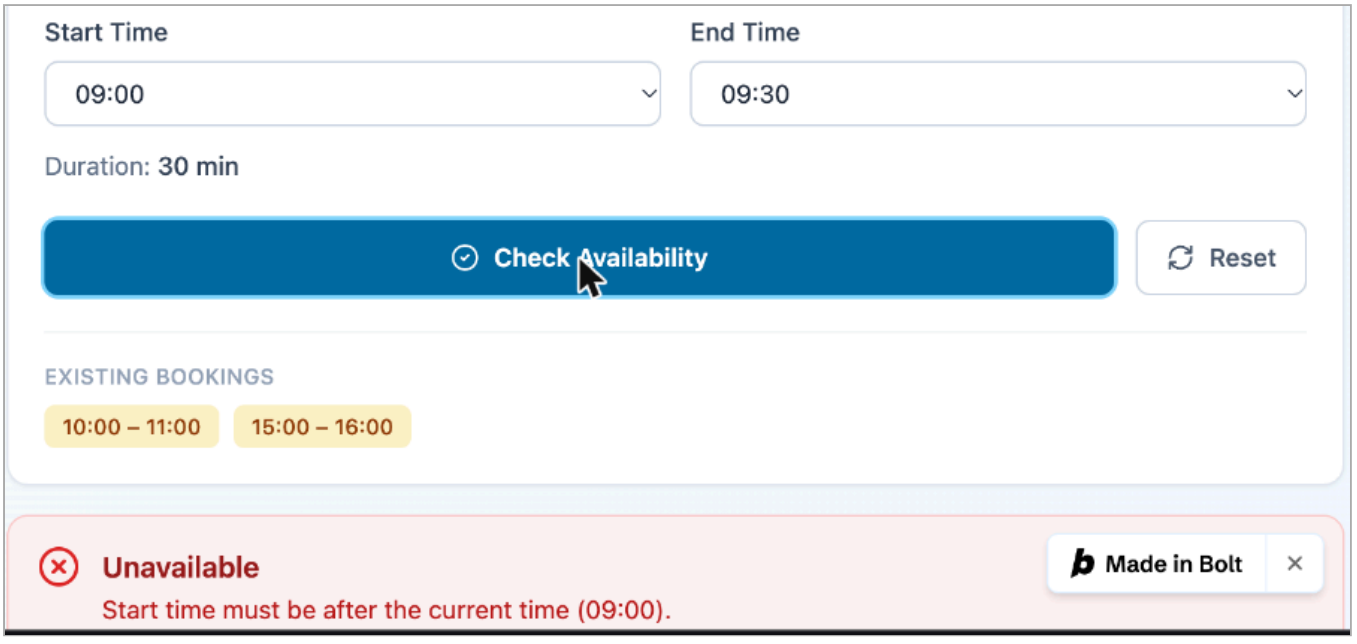
P-16 Vitest after (top) — P16_vitest_after_top.jpg



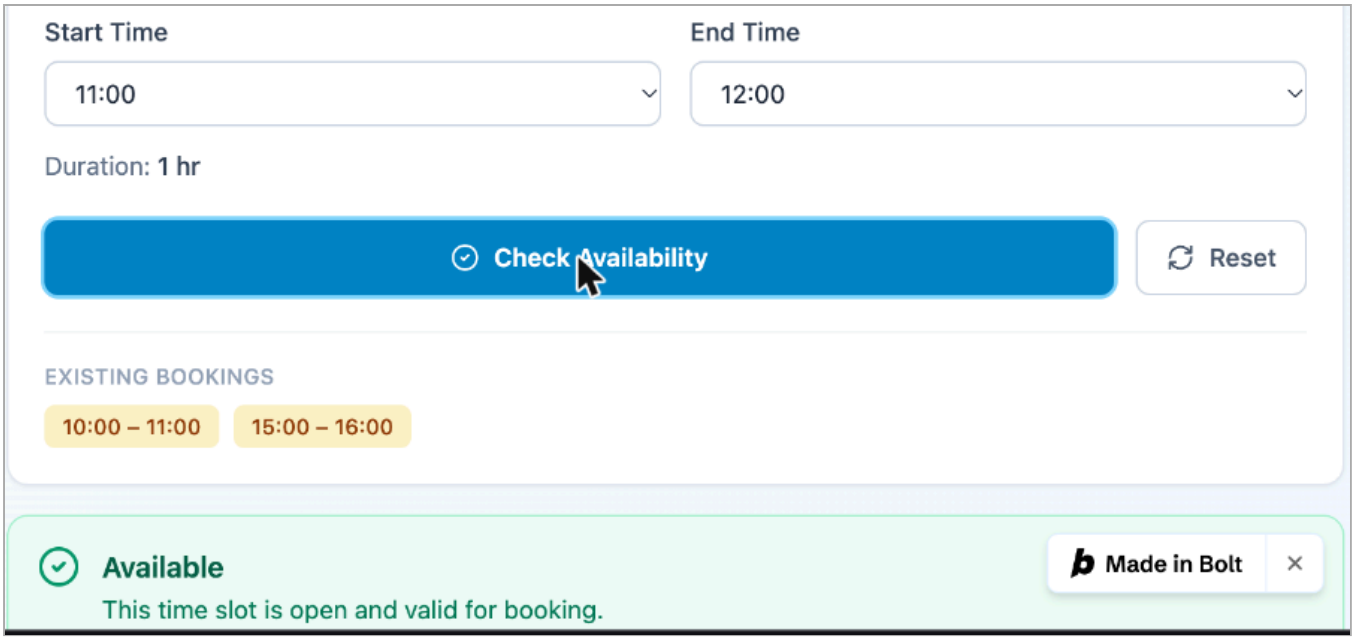
P-17 Vitest 21 passed (14:08) — P17_vitest_after_21_passed.jpg



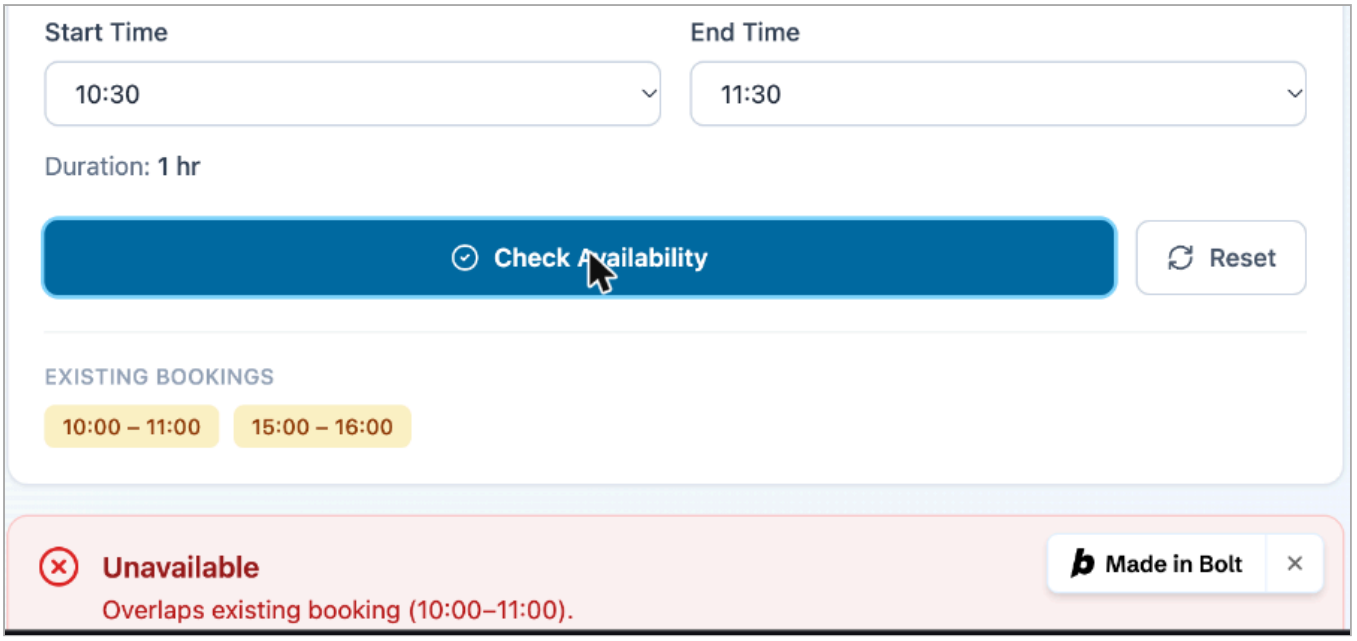
P-18 I-01 after — P18_I01_AFTER_blocked_room_message.jpg



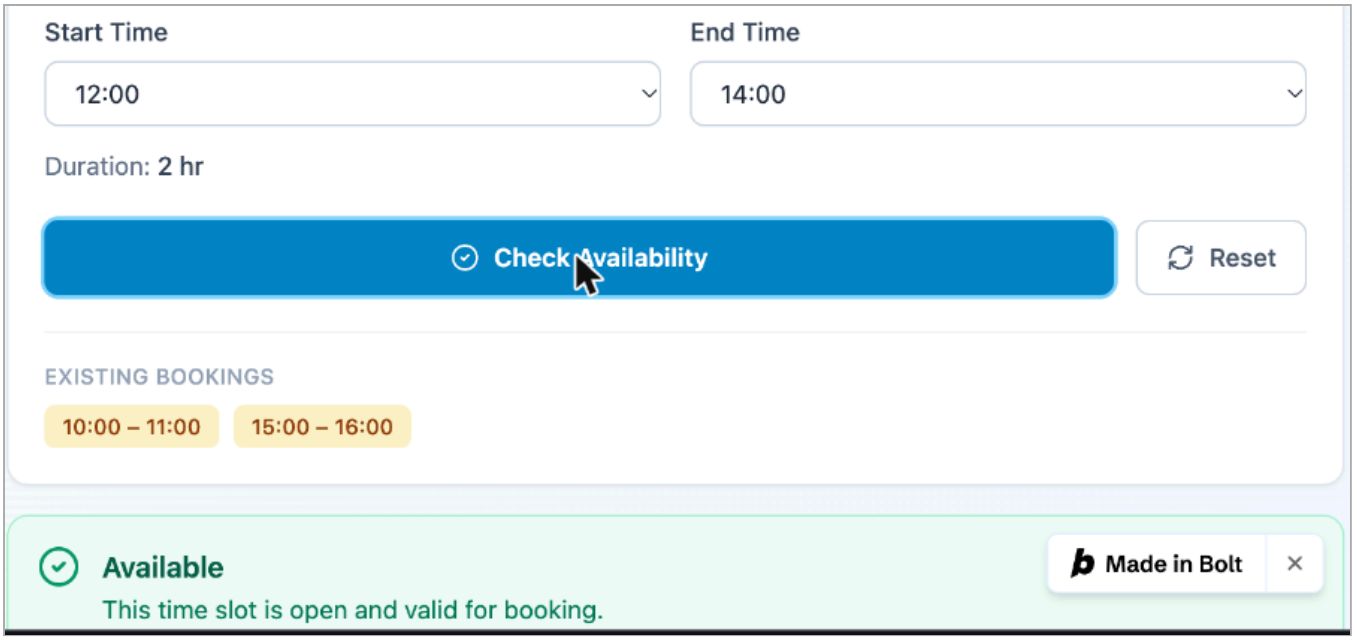
P-19 starts now (v7) — P19_final_T06_starts_now.png



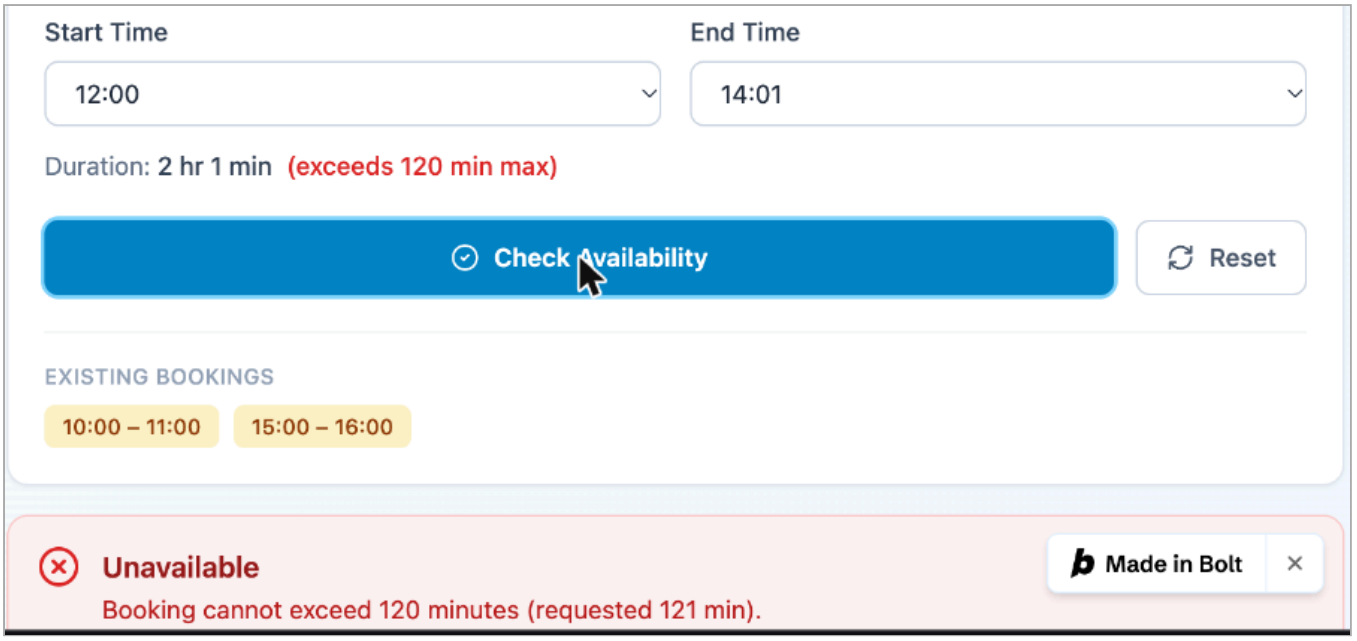
P-20 touching (v7) — P20_final_T01_touching.png



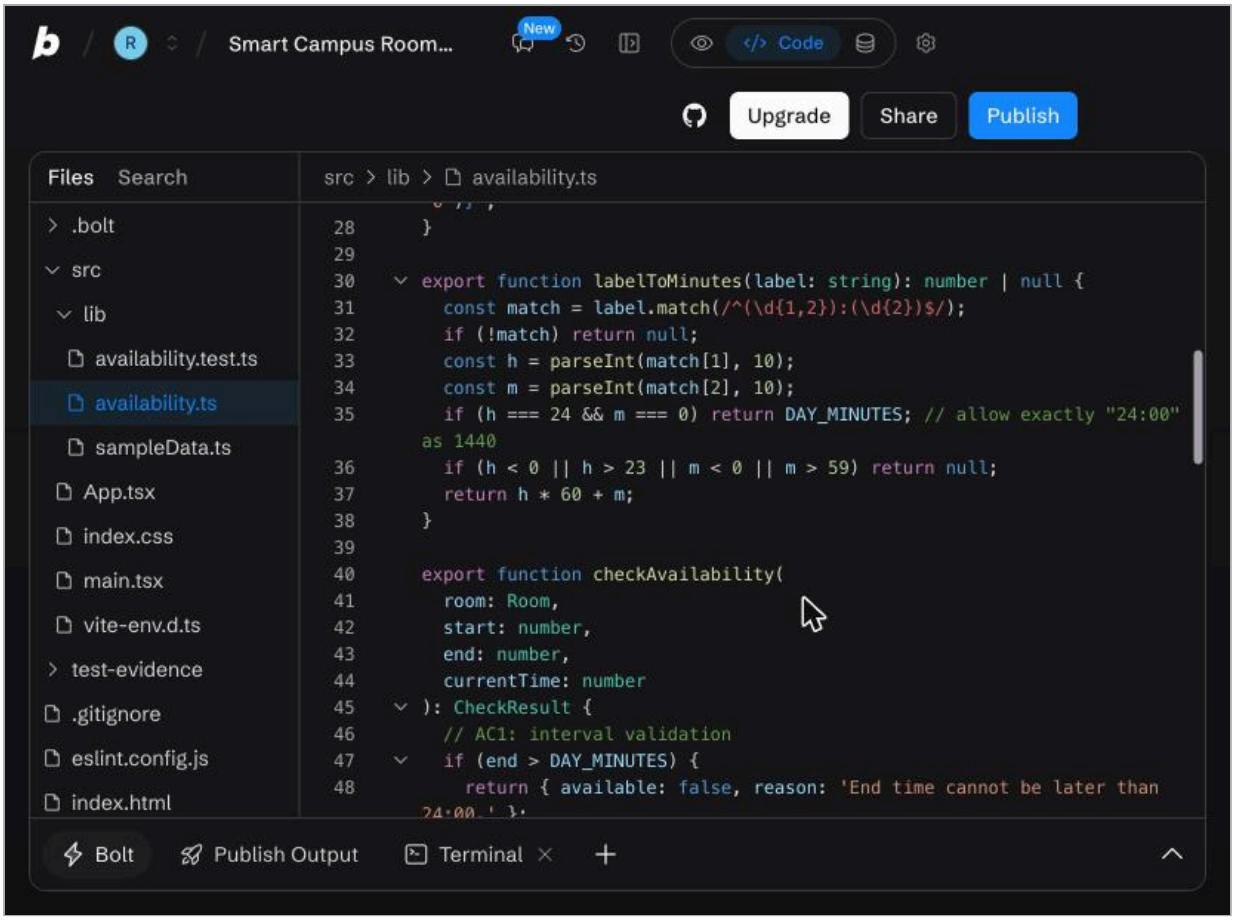
P-21 overlap (v7) — P21_final_T02_overlap.png



P-22 120 min (v7) — P22_final_T04_120min.png



P-23 121 min (v7) — P23_final_T05_121min.png



P-24 labelToMinutes code — P24_code_labelToMinutes_checkAvailability.jpg